Escuela
Politécnica
Superior

Videojuego para Game Boy en ensamblador gbz80

Grado en Ingeniería Multimedia

Trabajo Fin de Grado

Autor:
Jesús Hernández Fernández
Tutor/es:
Francisco José Gallego Durán

Septiembre 2019

Universitat d'Alacant
Universidad de Alicante

Packedtory

Un videojuego para Game Boy programado en ensamblador gbz80

Autor
Jesús Hernández Fernández

Directores
Francisco José Gallego Durán
Departamento de la Ciencia de la Computación e Inteligencia Artificial

Grado en Ingeniería Multimedia

Escuela Politécnica Superior

Universitat d'Alacant
Universidad de Alicante

ALICANTE, 6 de septiembre de 2019

Resumen

La industria de los videojuegos ha evolucionado hasta un punto en el que ya no es necesario ser una empresa grande para realizar un videojuego. Actualmente, los videojuegos que se realizan requieren de sistemas cada vez más potentes. Esto se debe en parte a que utilizan más recursos de los necesarios y no están lo suficientemente optimizados. Al contrario de los sistemas actuales, las primeras videoconsolas son sistemas que utilizan muy pocos recursos y sus juegos requieren del conocimiento en profundidad de sus límites.

El desarrollo de un videojuego de GameBoy es un proyecto pensado principalmente para el desarrollo de capacidades tecnológicas. Sacar partido a máquinas retro requiere entender su funcionamiento con detalle y trabajar más en contacto con el Hardware.

El objetivo principal de este trabajo es el desarrollo de un videojuego desde cero con ensamblador a bajo nivel para la consola GameBoy. He decidido hacer este trabajo porque es una buena forma de aplicar los conocimientos adquiridos durante la carrera y demostrar mis capacidades como ingeniero. Además, este tipo de proyecto es ideal para formar parte del portfolio, ya que son muy valorados por las empresas de videojuegos que demandan profesionales con buenas cualificaciones técnicas.

Agradecimientos

Me gustaría agradecer a todos mis compañeros de clase el apoyo que he recibido durante todos estos años de carrera. Todo este tiempo junto a ellos ha hecho que se conviertan en casi como una segunda familia para mí. En especial mi grupo de cuarto año, con el que compartí todo un año de convivencia y de historias que siempre recordaré.

También agradecer el apoyo de los profesores del grado. Esos profesores que dejan a un lado la monotonía y no dejan de evolucionar. Esos profesores que consiguen que los estudiantes den lo mejor de sí mismos y se involucren al máximo en lo que les apasiona.

Por último, nada de esto habría sido posible sin la constancia, el cariño y la paciencia de mi familia. Gracias a ellos pude conseguir estudiar lo que me apasiona y ser capaz de venir a Alicante.

A mis padres y mi hermano por haber estado siempre ahí,
a mis amigos y compañeros de clase por haber hecho que sea quien soy,
y a mi tutor y profesores por haberme orientado a lo largo de la carrera.

Una vez que dejas de aprender
es cuando empiezas a morir.

Albert Einstein.

El placer más noble es el júbilo de comprender.

Leonardo Da Vinci.

Índice general

1. Introducción y objetivos **1**

2. Marco Teórico **3**
2.1. Historia de la Game Boy . . . 3
2.2. Accesorios . . . 5
2.3. Game Boy en su lanzamiento . . . 10
2.4. Legado . . . 12

3. Terminología **13**

4. Metodología **15**
4.1. Herramientas utilizadas para el desarrollo . . . 15

5. Manual de Game Boy **17**
5.1. Herramientas de desarrollo . . . 17
5.2. Crear una ROM . . . 24
5.3. Tiles . . . 26
5.4. Sprites . . . 33
5.5. Interrupciones . . . 38
5.5.1. Timer . . . 38

6. Diseño **41**
6.1. Introducción . . . 41
6.2. Historia . . . 41
6.3. Características del Gameplay . . . 41
6.3.1. Puntos de entrada y salida . . . 41
6.3.2. Cintas transportadoras . . . 42
6.3.3. Objetivo . . . 43
6.3.4. Pantallas . . . 43
6.4. Adaptación a Game Boy . . . 44
6.4.1. Puntos de entrada y salida . . . 44
6.4.2. Cintas . . . 46
6.4.3. Pantallas y menús . . . 47
6.4.4. Controles . . . 48

7. Implementación **49**
7.1. Prototipo de aprendizaje . . . 49
7.2. Diseño del mapa . . . 50
7.3. Selector . . . 51
7.4. Gestión de las cajas . . . 53
7.4.1. Creación . . . 53
7.4.2. Borrado . . . 54
7.4.3. Dibujado . . . 54
7.5. Creación de cintas . . . 55
7.6. Movimiento de cajas . . . 56
7.6.1. Colisiones . . . 57

7.7. Puntos de entrada y salida . . . 57
7.8. Puntuación . . . 60
7.9. Rondas de juego . . . 61

Conclusiones **63**

Lista de Acrónimos **65**

Bibliografía **68**

A. Anexo I: Hardware de la Game Boy **69**
A.1. Especificaciones técnicas . . . 69
A.2. Mapa de memoria de la Game Boy . . . 69
A.3. Mapa de memoria de una ROM . . . 70
A.4. Input . . . 71
A.5. Timer . . . 71
A.6. Interrupciones . . . 71
A.7. Pixel Processing Unit (PPU) . . . 73
A.7.1. LCD Control (LCDC) . . . 73
A.7.2. STAT . . . 74
A.7.3. Mapa de fondo . . . 75
A.7.4. Ventana . . . 75
A.7.5. Sprites . . . 76
A.7.6. Mapa de la memoria VRAM . . . 78
A.7.7. Timing y acceso a VRAM y OAM . . . 79

Índice de figuras

2.1. Game Boy . . . 3
2.2. Atari 2600 . . . 3
2.3. NES . . . 3
2.4. Juegos de cartas de Nintendo . . . 3
2.5. Juegos de la NES . . . 4
2.6. Game & Watch (Ball) . . . 4
2.7. G&W (Donkey Kong) . . . 4
2.8. Tetris . . . 5
2.9. Juegos de la Game Boy . . . 5
2.10. Super Game Boy . . . 6
2.11. Hardware de Super Game Boy . . . 6
2.12. Opciones de la Super Game Boy . . . 6
2.13. Game Boy Link Cable . . . 7
2.14. Adaptador de 4 jugadores . . . 7
2.15. Batería recargable . . . 7
2.16. Cargador solar . . . 7
2.17. Nuby Light, Light Boy y Nuby Amplifier . . . 7
2.18. Handy Boy . . . 8
2.19. Booster . . . 8
2.20. Game Boy Camera . . . 9
2.21. Game Boy Printer . . . 9
2.22. Game Boy Radio FM . . . 9
2.23. Work Boy . . . 9
2.24. Barcode Boy Set . . . 9
2.25. Pocket Sonar . . . 10
2.26. Máquina de coser Singer . . . 10
2.27. PediSedate . . . 10
2.28. Game Boy funcionando después de una explosión . . . 11
2.29. Sega Game Gear . . . 11
2.30. Atari Lynx . . . 11
2.31. Dibujo en detalle de una Game Boy . . . 12

4.1. Feature Driven Development . . . 15
4.2. Git . . . 15

5.1. Código sin estilo (izq.) y con estilo (der.) en Sublime Text 3 . . . 18
5.2. Emulador BGB ejecutando The Legend of Zelda: Oracle of Ages . . . 18
5.3. Visor de VRAM: Mapa . . . 19
5.4. Visor de VRAM: Tiles . . . 19
5.5. Visor de VRAM: Sprites . . . 20
5.6. Visor de VRAM: Paletas de colores . . . 20
5.7. Debugger de BGB . . . 21
5.8. Game Boy Tile Designer . . . 22
5.9. Aseprite . . . 22
5.10. Game Boy Map Builder . . . 23

5.11. Everdrive para Game Boy . . . 23
5.12. ROM de una Game Boy vacía . . . 25
5.13. Visor de la VRAM del emulador . . . 25
5.14. Tiles que componen el sprite de mario en Super Mario Land . . . 26
5.15. Tiles que componen un mapa de Super Mario Land . . . 26
5.16. Colores codificados de Game Boy . . . 26
5.17. Índices de los tiles de una ROM vacía . . . 27
5.18. Mapa con los índices de una ROM vacía . . . 27
5.19. Fila de un tile codificada a hexadecimal . . . 28
5.20. Tile completo codificado a hexadecimal . . . 28
5.21. Mapa inundado al modificar el tile $00 . . . 30
5.22. Resultado del mapa al modificar el tile $19 . . . 30
5.23. Resultado del mapa al inicializar la paleta de colores . . . 31
5.24. Segundo tile codificado a hexadecimal . . . 32
5.25. Resultado final . . . 33
5.26. Sprites usados en Super Mario Land . . . 33
5.27. OAM . . . 34
5.28. Bit de prioridad desactivado (izq.) y bit de prioridad activado (der.) . . . 34
5.29. Funcionamiento de del espejado de los sprites . . . 35
5.30. Funcionamiento de las paletas de los sprites . . . 35
5.31. Memoria de objetos sin limpiar (izq.) y memoria inicializada a cero (der.) . . . 37
5.32. Sprite dibujado en la parte superior izquierda . . . 37

6.1. Mecánica principal del juego . . . 41
6.2. Posibles posiciones para los puntos de entrada o salida . . . 42
6.3. Diferentes tipos de puntos de entrada, salida y cajas . . . 42
6.4. Puntos de entrada (arriba) y de salida (abajo) . . . 42
6.5. Diferentes tipos de cinta y sus teclas de activación . . . 43
6.6. Pantalla del juego de Compactory . . . 43
6.7. Boceto del juego . . . 44
6.8. Adaptación de las cajas a una Game Boy . . . 45
6.9. Adaptación de los puntos de entrada a una Game Boy . . . 45
6.10. Estados de los nuevos puntos de entrada (arriba) y salida (abajo) . . . 45
6.11. Elementos secundarios y tiles que se editan a mano . . . 46
6.12. Selector del del mapa en la primera casilla . . . 46
6.13. Comportamiento de la cinta transportadora . . . 46
6.14. Comportamiento de la cinta de lanzamiento rápido . . . 46
6.15. Comportamiento de la cinta separadora . . . 46
6.16. Adaptación de la selección de cinta . . . 47
6.17. Adaptación de la orientación de las cintas . . . 47
6.18. Menú principal y Game Over . . . 47
6.19. Pantalla del juego . . . 48
6.20. Adaptación de los controles . . . 48

7.1. Prototipo de aprendizaje . . . 49
7.2. Boceto de las cintas . . . 50
7.3. Boceto modificado de las cintas . . . 50
7.4. Tipos de cajas . . . 51
7.5. Boceto del mapa . . . 51
7.6. Celda del mapa . . . 51
7.7. Posiciones en hexadecimal del sprite del selector (color) e índices de la rejilla (negro) . . . 52

7.8. Detalle de una caja . . . 53
7.9. Código de cada una de las cintas . . . 55
7.10. Significado del código de las cintas . . . 55
7.11. Mapa con cintas en el mapa y sus códigos en el array . . . 56
7.12. Esquema con los pasos del movimiento de las cajas . . . 57
7.13. Esquema del array de los puntos de entrada y salida . . . 58
7.14. Índices del estado de los puntos entrada (arriba) y salida (abajo) . . . 59
7.15. Posiciones de los puntos en orden del array . . . 59
7.16. Índices de los tiles de los puntos . . . 60
7.17. Esquema de la barra del tiempo . . . 62
7.18. Packedtory funcionando en una Game Boy Color . . . 64

A.1. Mapa de la memoria . . . 69
A.2. Cabecera de la ROM . . . 70
A.3. Registros del input y distribución de los botones . . . 71
A.4. Registros del timer y frecuencias . . . 71
A.5. Registros Interrupt Enable (izq.) e Interrupt Flag (der.) . . . 72
A.6. Todos los registros que forman la PPU . . . 73
A.7. Bits del registro LCDC . . . 74
A.8. Bits del registro STAT . . . 75
A.9. Mapa de fondo y viewport en Game Boy Tennis . . . 75
A.10. Ejemplo de uso de ventana en Mega Man II . . . 76
A.11. Mapa de fondo (izq.) y ventana (der.) en Mega Man II . . . 76
A.12. Sprite de Mega Man . . . 76
A.13. Entrada de la OAM . . . 77
A.14. Estructura de la OAM . . . 77
A.15. Tamaño de los componentes de la VRAM . . . 78
A.16. Bit 6 del LCDC desactivado (izq.) y activado (der.) . . . 78
A.17. Mapa de la memoria de la VRAM . . . 79
A.18. Desglose de lo que se hace mientras se dibuja una línea . . . 80
A.19. Registro de la OAM . . . 80
A.20. Esquema con el acceso desde la CPU a la VRAM . . . 81
A.21. Momento cuando la VRAM y la OAM es accesible . . . 81

Índice de tablas

A.1. Especificaciones técnicas de la Game Boy 69
A.2. Estructura de una ROM 70

Índice de Listados

5.1. Código mínimo para crear una ROM 24
5.2. Función para esperar al V-Blank 29
5.3. Función para cargar un tile en la posición $8000 29
5.4. Modificación para cargar el tile en la posición $8190 30
5.5. Código para inicializar paleta de colores 31
5.6. Código con los dos tiles juntos 32
5.7. Pequeño truco para calcular el tamaño de los bytes automáticamente 32
5.8. Código para pintar en el mapa un tile nuevo 32
5.9. Código para definir un sprite 36
5.10. Código para dibujar un sprite 36
5.11. Función para iniciar el contador 39
5.12. Código para definir la función para la interrupción del timer 39
5.13. Función para actualizar los segundos 39
5.14. Importante no olvidar guardar los registros 39

7.1. Función de mover el selector abajo 52
7.2. Función de mover el selector a la derecha 53
7.3. Código para separar los valores de 1 registro en 2 54
7.4. Código para copiar los tiles al mapa de fondo 60
7.5. Incremento de la puntuación 61
7.6. Dibujado de la puntuación 61

1. Introducción y objetivos

Desde hace unos cuantos años, hacer videojuegos dejó de estar únicamente en manos de grandes empresas con muchos recursos y empleados. En la actualidad, gracias a los motores de videojuegos, pequeños equipos sin apenas presupuesto alguno pueden aventurarse a desarrollar un videojuego de muchísima calidad sin la preocupación de tener que programar todo el motor gráfico u otras funcionalidades que son necesarias y tienen en común la mayoría de juegos.

Sin embargo, esta despreocupación no siempre es beneficiosa. Si queremos explotar al máximo una plataforma y tener el control total, es necesario conocer a fondo la arquitectura y el hardware. Las videoconsolas y ordenadores actuales tienen una arquitectura demasiado compleja y avanzada como para ser estudiadas por una sola persona. Es por esto que las consolas de hace unos años son ideales para empezar.

Con el nacimiento de Atari 2600 en 1976 hubo una revolución para las videoconsolas y muchas empresas empezaron a lanzar sus propias plataformas. Cada una de ellas exploraba diferentes territorios y contribuía al desarrollo del campo de los videojuegos. Entre las precursoras, destacaban notablemente la **Nintendo Entertainment System (NES)** sus grandes juegos y la **Sega Master System** por ser su competencia. Aunque tuviera mejor hardware y buenos juegos, la NES acabó ganando, aun así, en Europa se vendió la Master System mucho más que en el resto del mundo. De entre todas las videoconsolas, en este trabajo se desarrollará la Game Boy.

¿Por qué Game Boy? La Game Boy (GB) junto a la Game Boy Color (GBC) vendieron 118 millones de copias y, si contamos las de Game Boy Advance (GBA), se vendieron unas 200 millones en total. Se hicieron unos 1050 juegos para GB, unos 575 para GBC y se produjeron sistemas de 8-bits desde el 1989 hasta el 2003. Además, al ser los juegos retrocompatibles con la GBA, se puede decir que los juegos de GB perduraron hasta 2009, lo cual hace una trayectoria de nada más y nada menos que 20 años.

La Game Boy influenció a millones de personas en todo el mundo. Apareció en cientos de noticias y se hablaba de ella constantemente. Personajes famosos la jugaban, se llegó a ver a Hillary Clinton con una Gameboy en el avión presidencial e incluso astronautas en la estación espacial enviaron fotos de ellos jugando. La Gameboy dió la oportunidad de llevar sus grandes juegos a cualquier parte, cambiando la industria de los videojuegos y el concepto de tiempo libre para siempre.

El objetivo principal de este trabajo es el desarrollo de un videojuego desde cero para la consola GameBoy a bajo nivel con ensamblador. A la vez, se pretende expandir los conocimientos técnicos y de programación comprendiendo el funcionamiento en detalle del hardware de esta consola.

Los objetivos más específicos que se pretenden alcanzar con este proyecto son:

- Aprender ensamblador de Game Boy y profundizar en el procesador de la consola.
- Diseñar y desarrollar un videojuego que funcione en una Game Boy real.
- Estudiar la arquitectura de la Game Boy, entender cómo se organiza y todas sus características.
- Hacer una pequeña guía que pueda servir como referencia a futuros programadores que quieran introducirse en la programación de videojuegos para esta Game Boy.
- Explicar todas las técnicas aplicadas y conocimientos aprendidos.

2. Marco Teórico

2.1. Historia de la Game Boy

La Game Boy es la **consola portátil más popular de todos los tiempos** y **cambió la trayectoria de la historia de los videojuegos**. La primera Game Boy salió a la venta el 21 de Abril de 1989 en Japón, lo que significa que este año, 2019, es el 30 aniversario de esta consola. Después de su lanzamiento en japón, 3 meses más tarde salió en Norte América y, casi un año más tarde, en Europa.

Los años 80 fueron una época de un cambio masivo tanto en cultura como en tecnología. Fue cuando empezó el nacimiento de la tecnología de uso personal: ordenadores, teléfonos móviles y fue cuando apareció el *Walkman*. Había muchísimo menos entretenimiento a tu disposición, no había televisión cuando querías y la vida para los niños era un poco aburrida.

Figura 2.1.: Game Boy

Los videojuegos principalmente se encontraban en recreativas, las cuales eran muy populares entre adolescentes. Entonces, de repente aparecieron las videoconsolas de interiores que revolucionaron todo. La *Atari 2600* fue un fenómeno tan grande que otras empresas quisieron saltar a la acción y empezar a hacer videojuegos.

Figura 2.2.: Atari 2600

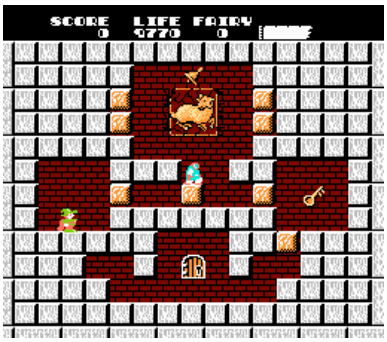
Figura 2.3.: NES

Un par de años más tarde, Nintendo se une a la fiesta y presenta un nuevo sistema de videojuegos, pero la gente fue escéptica al principio. Esto ocurrió porque Nintendo empezó como fabricante de juegos de cartas, cerca del año 1890. No fue hasta el comienzo de los años 80 cuando Nintendo empezó a definirse como una empresa de tecnología.

Figura 2.4.: Juegos de cartas de Nintendo

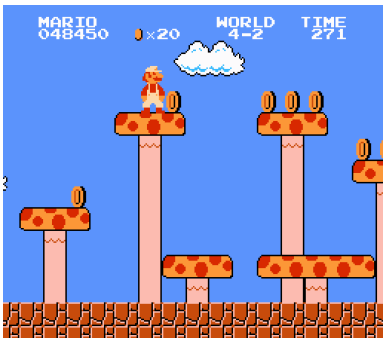
Tras el lanzamiento de la *Nintendo Entertainment System (NES)*, Nintendo se hizo con el

control de la industria del videojuego y se puso en cabeza. Nintendo se volvió el rey de los videojuegos en ese momento con grandes títulos como Duck Hunt, Solomon's Key, Castlevania y, las estrellas, Super Mario y Zelda. Gracias a todos estos grandes juegos, cuando se lanzó la Game Boy al mercado **no hubo duda alguna de que iba a ser algo bueno**.

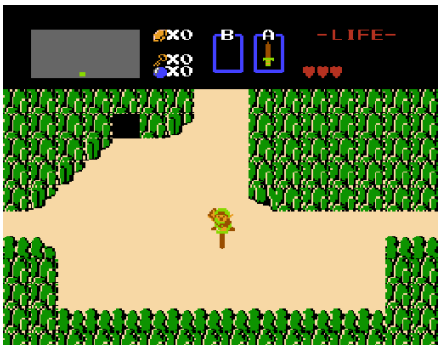


(a) Solomon's Key

(b) Duck Hunt



(c) Super Mario Bros



(d) The Legend of Zelda

Figura 2.5.: Juegos de la NES

El diseñador de la Game Boy fue el japonés *Gunpei Yokoi*. Como es bien sabido, un día se encontraba en un tren en Japón y se cruzó con un empresario pulsando los botones de una calculadora para matar el tiempo. Este acontecimiento le hizo pensar en qué pasaría si existiera un juego portátil que pudiera llevarse en los trayectos de tren. Fue en ese momento cuando Gunpei se dio cuenta del potencial de las videoconsolas portátiles.

Una de las grandes cosas acerca de Nintendo al principio de los años 80 es que verdaderamente creían en I+D (Investigación más desarrollo), en experimentar con lo nunca visto. Entonces, en 1980, Gunpei Yokoi inventó el primer juego de la famosa saga *Game & Watch*. Esto fue un paso intermedio muy interesante entre las recreativas, las videoconsolas de interiores y los juegos portátiles.

Figura 2.6.: Game & Watch (Ball)

Figura 2.7.: G&W (Donkey Kong)

Usaron una pantalla LCD impresa a medida, diseñada para mostrar personajes y objetos

predefinidos. Las versiones plegables de G&W con dos pantallas sirvieron como inspiración para futuras videoconsolas como la gran *Nintendo DS*. Se llegaron a producir 59 juegos diferentes de G&W y fue para el juego de *Donkey Kong* cuando inventaron la famosa cruceta o D-Pad (Figura 2.7, a la izquierda de la pantalla inferior). G&W no fue nada extraordinario pero a la gente le gustó y eso fue el principio de Nintendo en los videojuegos portátiles.

Entonces, Gunpei desarrolló ese concepto que nació con Game & Watch y diseñó la Game Boy. Nadie consiguió motivar tanto a los jugadores como lo hizo Nintendo. Les prometía todos los juegos que tanto amaban, pero ahora, con la posibilidad de llevárselos a todas partes. Además, uno de los grandes motivos que favoreció al éxito de la Game Boy es que fue emparejada con el juego perfecto: el **Tetris**.

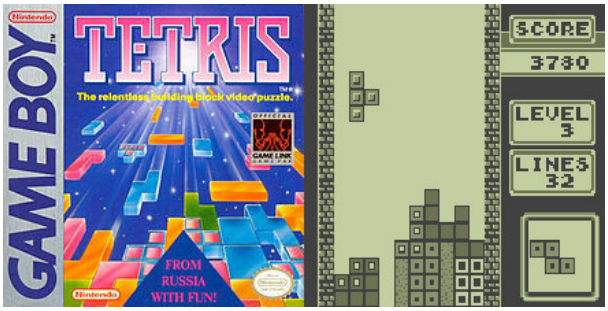


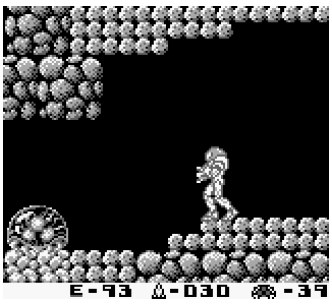
Figura 2.8.: Tetris

Tetris era el juego al que todo el mundo quería jugar. Era **muy adictivo**, **muy fácil de empezar a jugar** y **difícil de dominar** al completo. Viene de la Unión Soviética de principios de los años 80 donde un ingeniero informático llamado Alexey Pajitnov salió de su laboratorio con un juego al que llamó Tetris. Hizo pequeñas copias del juego, empezó a venderlas a escondidas e intentó retomar los derechos del juego, los cuales eran de los rusos. Esto desencadenó en una serie de demandas y peleas en juicios. Millones de dolares fueron cambiando de manos y, de alguna manera, el Tetris acabó jugándose en todas las plataformas imaginables, formando parte de la vida de millones de personas alrededor del mundo.

Además del Tetris, se lanzaron más juegos que fueron un éxito tras otro. Juegos como *The Legend of Zelda: Link's Awakening*, *Super Mario Land* y *Metroid II* fueron muy representativos para Game Boy y son considerados juegos muy buenos a día de hoy.

(a) TLoZ: Link's Awakening

(b) Super Mario Land



(c) Metroid II

Figura 2.9.: Juegos de la Game Boy

2.2. Accesorios

Por si la Game Boy no fuera suficiente, existía una gran cantidad de extraños, absurdos y maravillosos accesorios. La mayoría de ellos no llegaron a triunfar mucho, pero la Game Boy

tenía la fama y la cantidad de fans suficiente para apoyar un mercado de accesorios de terceras empresas.

Probablemente uno de los accesorios más útiles para la Game Boy fue el **Super Game Boy**. Realmente es un accesorio para la Super Nintendo Entertainment System (SNES) y permite jugar a los juegos de Game Boy en una televisión. Algo interesante de este accesorio es que la SNES no cuenta con la función de emular juegos de Game Boy, sino que es el propio cartucho el que contiene casi todo el hardware que está en la propia Game Boy, menos la pantalla y los botones.

Figura 2.10.: Super Game Boy

Figura 2.11.: Hardware de Super Game Boy

La SNES reproduce los juegos de Game Boy en su resolución nativa, 160x144 píxeles, dentro de su propia pantalla de 256x224, lo cual obliga a que haya un marco alrededor de la ventana del juego. Tiene la opción de poder cambiar las paletas de color originales a unas ya prediseñadas, o incluso elegir los 4 colores de la paleta entre los 12 colores disponibles. También permitía la opción de cambiar los marcos o dibujar unos nuevos y la opción de poder dibujar encima del juego, ideal para matar el tiempo.

(a) Selección de paleta de color

(b) Creación de paleta

(c) Selección de borde

(d) Dibujar por encima del juego

Figura 2.12.: Opciones de la Super Game Boy

Otro de los accesorios más populares es el **Game Boy Link Cable**. Este cable sirve para conectar dos Game Boys para que dos jugadores puedan competir en, por ejemplo, el Tetris. Además, existe un adaptador para cuatro 4 jugadores. Suena muy bien la idea pero hacen falta 4 personas, 4 Game Boys, 4 copias del mismo juego y que el juego sea compatible con 4 jugadores.

Figura 2.13.: Game Boy Link Cable

Figura 2.14.: Adaptador de 4 jugadores

Con todas las videoconsolas portátiles, si querías irte de viaje, tenías que asegurarte de llevarte pilas de sobra. Sin embargo, la Game Boy contaba con su propia batería recargable que podías utilizar para jugar sin pilas. Aquí es donde empiezan los accesorios raros porque también existía un cargador solar para Game Boy. Tras unas 8 horas de carga al sol, se puede jugar sin necesidad de pilas mientras dure la batería.

Figura 2.15.: Batería recargable

Figura 2.16.: Cargador solar

Por otro lado, están los accesorios que mejoran el hardware de la Game Boy. Como por por ejemplo, el cómo se ve la pantalla de la Game Boy o el sonido. Al tener una pantalla sin retro-iluminación, no es posible jugar si no hay mucha luz en la habitación o si es de noche.

Figura 2.17.: Nuby Light, Light Boy y Nuby Amplifier

El accesorio más sencillo que resuelve este problema es el **Nuby Light**. Se encaja en la parte superior de la Game Boy de forma que cubre el borde de la pantalla y la ilumina completamente

alrededor con unas pequeñas luces. El único inconveniente es que necesita 4 pilas AA para funcionar, lo cual suma un total de 8 pilas AA para jugar de noche. Su versión mejorada podría ser el **Light Boy**, un accesorio muy similar que requiere de solo 2 pilas y además incorpora un sistema novedoso para amplificar la pantalla, una lupa. Como los altavoces de la Game Boy no tenían mucha potencia ni calidad de sonido, Nuby creó otra solución, el **Nuby Amplifier**. Este se acopla en la parte inferior y con 2 altavoces es capaz de amplificar notablemente el sonido de la consola.

A partir de aquí, los accesorios de la Game Boy evolucionaron vertiginosamente. Aparecieron sistemas que pretendían mejorar la experiencia completa de jugar a la videoconsola. El **Handy Boy** fue de los primeros en encargarse de dicha tarea. Consiste en varios módulos encajables que contienen luces y lupa como el Light Boy, nuevos controles que se superponen a los botones de la Game Boy, e incluso altavoces estéreo. Además, para su funcionamiento no necesita pilas adicionales, sino que toma corriente de las propias pilas de la Game Boy.

Figura 2.18.: Handy Boy

Las luces del Handy Boy podían no funcionar muy bien y se preparó una segunda opción: dos láminas de aluminio que se colocaban entre las pilas y los contactos para utilizar la electricidad de 2 de las 4 pilas. Evidentemente, al ser el pionero en un proyecto que implica un cambio tan drástico, el funcionamiento de este no era ideal. Para ello, surgieron nuevos modelos que hacían las mismas funciones de manera más eficiente, como el **Booster**, una carcasa completa que implementa mejoras de pantalla, altavoces y controles todo en uno, un pequeño compartimento para guardar cartuchos y sus propias pilas (4 pilas C) de las que la Game Boy toma la corriente, al contrario del Handy Boy.

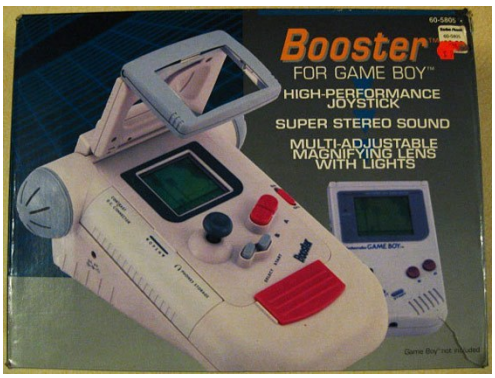


Figura 2.19.: Booster

Pero, ¿hay algún accesorio que sirva para algo que no sea mejorar las características de la Game Boy? Pues sí, aunque solo unos pocos fueron usados por un número de usuarios decente y otros ni siquiera llegaron a ver la luz y se quedaron en patentes, existe una gran variedad de auténticos accesorios.

La **Game Boy Camera**, la cual mantuvo durante un tiempo el Record Guinness de la cámara digital más pequeña del mundo, es un cartucho con una cámara integrada que se coloca como si fuera un juego cualquiera. Se pueden tomar fotos de cualquier cosa, hacer un álbum con ellas e incluso jugar a pequeños minijuegos con un personaje que puede tener tu cara. Por si fuera poco, había otro accesorio que complementaba a la perfección a la cámara: la **Game Boy Printer**. Este accesorio puede imprimir en papel con tinta negra cualquiera de las imágenes tomadas con la cámara.

Figura 2.20.: Game Boy Camera

Figura 2.21.: Game Boy Printer

También se llegaron a fabricar accesorios multimedia para la Game Boy como una radio FM o un adaptador para teclado (Work Boy) por si un monitor era demasiado grande. Hasta se hizo un lector de códigos de barras (Barcode Boy Set) con el que se podían desbloquear ciertos elementos en los juegos que venían incluidos.

Figura 2.22.: Game Boy Radio FM

Figura 2.23.: Work Boy

Figura 2.24.: Barcode Boy Set

Estos fueron probablemente los más populares con diferencia, pues había otros accesorios que, incluso en esa época podrían parecer innecesarios o hasta absurdos. Uno de ellos puede ser el **Pocket Sonar**, un sonar para la Game Boy que ayuda a localizar peces hasta 20 metros bajo el

agua. También se llegó a fabricar una máquina de coser que puede coser patrones que se pueden crear y enviar desde la consola. Por último, el accesorio que probablemente fuera más bizarro y chocante es el **PediSedate**. Este accesorio son básicamente unos auriculares y un respirador que tiene un tubo por detrás al que se puede conectar, por ejemplo, anestesia. El objetivo era calmar a los niños con la Game Boy mientras son sedados antes de una operación.

Figura 2.25.: Pocket Sonar

Figura 2.26.: Máquina de coser Singer

Figura 2.27.: PediSedate

2.3. Game Boy en su lanzamiento

La Game Boy fue una pieza única de tecnología. Sin embargo, por aquellos tiempos tuvo que competir contra otras videoconsolas que tenían más aspecto de última generación. Llegó a competir con otros sistemas como la **Sega Game Gear** o la **Atari Lynx**, las cuales tenían una pantalla mucho mejor, **con colores**, **retroiluminación** y procesadores **más potentes**. Cuando una persona quería comprar una videoconsola, la competencia siempre era más llamativa. Sin embargo, a la hora de la verdad, los jugadores querían una videoconsola portátil que durase el mayor tiempo posible. El resto de videoconsolas tenían una batería de solo 4 o 5 horas como máximo, mientras que la Game Boy podía aguantar hasta 15 horas con las mismas pilas. Adicionalmente, la Game Boy es considerada una de las videoconsolas más resistentes, podía aguantar cientos de golpes como si nada. Incluso existe una Game Boy de un soldado de la Guerra del Golfo que aguantó a la explosión de una bomba nuclear y aún seguía funcionando.

Figura 2.28.: Game Boy funcionando después de una explosión

La Game Boy fue desarrollada utilizando la filosofía con la que Nintendo se guiaba: "Innovación lateral con tecnología marchita". Lo que quiere decir es que buscaba tecnología que ya tenía unos años y que ya estuviera lo suficientemente desarrollada para saber perfectamente cómo utilizarla. Así que, aunque otras empresas tuvieran componentes más llamativos y atractivos, no sabían realmente sacarle provecho.

Figura 2.29.: Sega Game Gear

Figura 2.30.: Atari Lynx

Nintendo hizo en cierta manera lo que Steve Jobs hizo con el iPhone años más tarde. Desecharon muchos elementos que no eran importantes y no colocaron controles muy sofisticados: una cruceta (D-Pad) y 2 botones era básicamente lo que tenía. Para jugar, simplemente tenías que meter 4 pilas por la parte de atrás, insertar el juego por la ranura y darle al botón en la parte superior para encender la consola y escuchar su sonido tan característico.

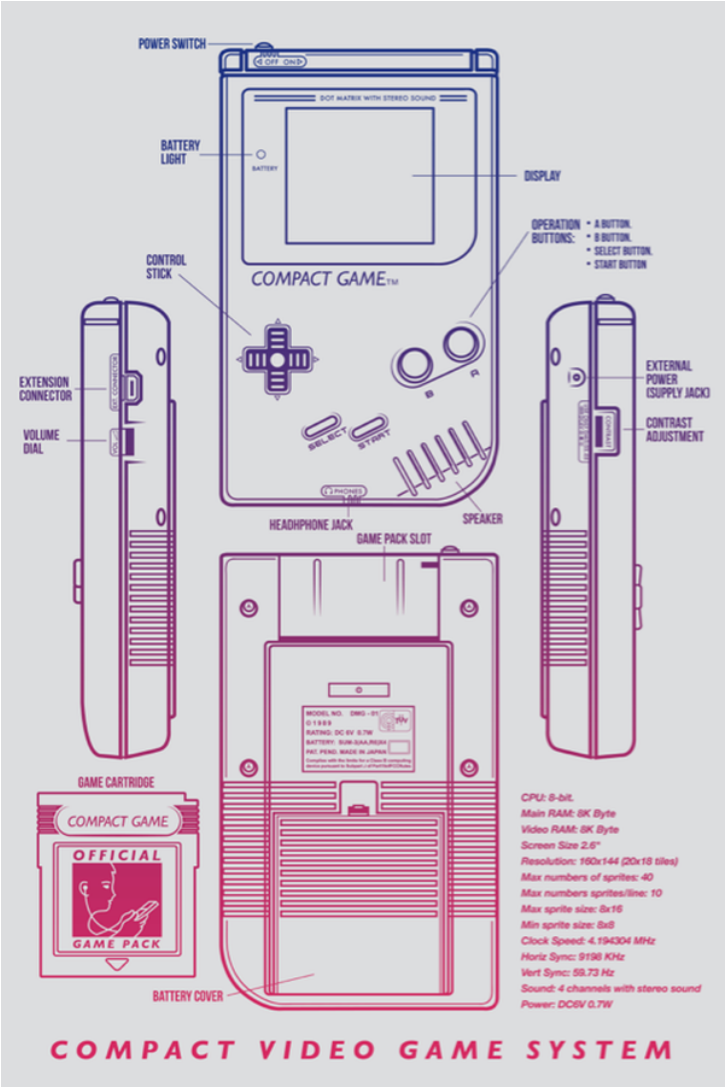


Figura 2.31.: Dibujo en detalle de una Game Boy

2.4. Legado

Es difícil exagerar el legado que dejó la Game Boy ya que se puede decir que revolucionó la tecnología **para siempre**. Sin la Game Boy no habrían smartphones, ni mucho menos videojuegos para smartphones. Fue el comienzo de la idea de, donde sea que vayas, llevar algo en el bolsillo que supiera cómo entretenerte. La Game Boy influyó en la manera de administrar el tiempo libre e hizo aparecer una nueva forma de entretenimiento.

Una de las cosa más importantes que dejó como legado es el concepto de que el minimalismo en el diseño de la tecnología es una herramienta y no un fallo. Desde la simpleza y pureza del diseño de la Game Boy se puede trazar una linea recta directamente a lo que se vio en el primer iPhone. Cuando se venden 200 millones de unidades de algo, es imposible decir que no fue un enorme éxito. Fue una videoconsola que consiguió que madres siguieran llamando a otras videoconsolas "Game Boy", aún habiendo pasado 10 años de su lanzamiento, y que consiguió convertir a millones de personas en "gamers". **Sin la Game Boy, no existiría la tecnología como la conocemos hoy en día**.

3. Terminología

A largo del documento se van a usar distintos términos, muchos de ellos abreviados para facilitar la lectura del documento.

- **Bit**: Binary digit, unidad mínima de información en informática.
- **Byte**: Unidad de información que equivale a 8 bits.
- **Hardware**: Partes físicas, tangibles, de un sistema informático.
- **Software**: Conjunto de los programas de cómputo, procedimientos, reglas, documentación y datos asociados, que forman parte de las operaciones de un sistema de computación. El Software envía instrucciones y el Hardware las ejecuta.
- **CPU**: Central Processing Unit, hardware que interpreta las instrucciones de un programa informático mediante operaciones básicas aritméticas, lógicas y de entrada/salida del sistema.
- **Opcode**: Números en hexadecimal que representan una instrucción.
- **RAM**: Random Access Memory, memoria sobre la que la CPU trabaja.
- **ROM**: Read-Only Memory, memoria en la que se guardan datos o código de solo lectura que la CPU ejecutará.
- **PPU**: Pixel Processing Unit, es el hardware encargado de dibujar los gráficos por pantalla.
- **VRAM**: Video RAM, memoria que la PPU utiliza para trabajar.
- **OAM**: Object Attribute Memory, parte de la VRAM que almacena atributos de todos los sprites.
- **HRAM**: High RAM, memoria de alta velocidad que puede acceder a la VRAM en todo momento.
- **Píxel**: Unidad básica de una imagen en una pantalla.
- **Tile**: Conjunto de píxeles de un tamaño determinado.
- **Sprite**: Elemento visual activo de la imagen en una pantalla.
- **Frame**: Fotograma, imagen que se muestra por la pantalla en un momento dado.
- **FPS**: Frames Per Second, medida de la frecuencia a la que se muestran imágenes en una pantalla.
- **LCD**: Liquid-Crystal Display, el tipo de pantalla que tiene la Game Boy.

Además, para las cifras se usarán los mismos símbolos que se utilizan en el ensamblador utilizado para el juego:

- $: Se usará para números en notación hexadecimal y la cantidad de números siempre 2 o 4 debido al tamaño de los registros ($0F, $FF00)

- Los números que empiecen con % se interpretarán en binario y generalmente tendrán una longitud de 8 o múltiplo de 8 (%00010000, %00001110)
- El resto de números se interpretarán en decimal.

Ejemplo: $3C = %00111100 = 60

4. Metodología

Puesto que se trata de un proyecto individual, no resulta necesario aplicar completamente una metodología de desarrollo, ya que estas están enfocadas al trabajo en equipo. No obstante, el uso de pautas utilizadas por las metodologías ágiles sigue siendo útil a pesar de trabajar solo.

Para la realización de este proyecto, se siguieron las pautas de una metodología ágil denominada FDD o Feature Driven Development (Desarrollo Basado en Funcionalidades). Esta metodología se basa en la realización de un diseño preliminar y global de cómo será el proyecto una vez terminado, tras lo cual se desarrollará una lista con todas y cada una de las funcionalidades que incluirá, ordenadas por prioridad de realización. Cuando se hayan planificado todas las funcionalidades se comenzará con su desarrollo realizando una a una de forma iterativa, de modo que se vaya completando el proyecto progresivamente.

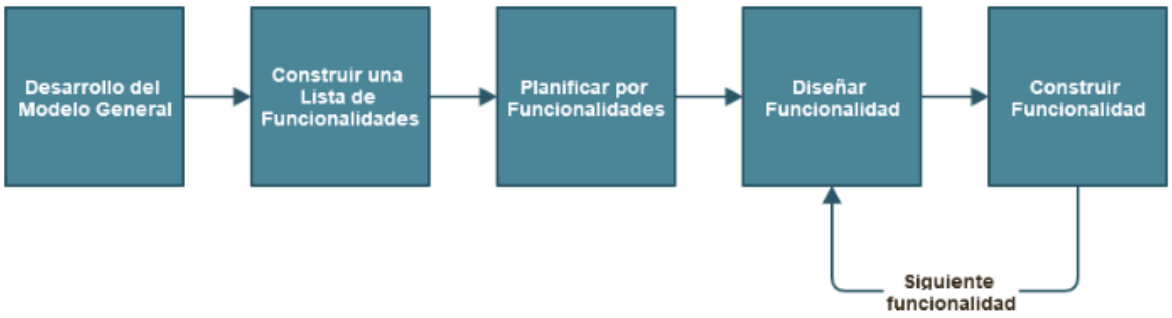


Figura 4.1.: Feature Driven Development

Esta metodología es muy útil para este tipo de proyectos, ya que requieren de muchas funcionalidades y es necesario planificarlas y priorizarlas para que no exista confusión durante su desarrollo, más aún si es un proyecto realizado por una única persona. Además, el diseño y construcción de las funcionalidades de forma individual hace que el proyecto sea más organizado y mantenga un flujo de creación continuo.

Adicionalmente, se usará un sistema de control de versiones como **Git** para evitar la pérdida de datos y para poder deshacer cambios en caso de ser necesario. Git permite un buen manejo de ficheros de texto y comprobar el historial de ediciones, así como comparar los cambios de manera muy sencilla.

Figura 4.2.: Git

4.1. Herramientas utilizadas para el desarrollo

Durante el desarrollo se van a utilizar las siguientes herramientas:

- **Sublime Text 3**: Editor de texto con resaltado editable de sintaxis del ensamblador RGBDS
- **BGB**: Emulador de Game Boy. Permite ejecutar juegos de Game Boy en los ordenadores actuales.
- **Game Boy Tile Designer** (GBTD): Herramienta open source de edición de sprites de Game Boy. Se usará para la creación de tiles tanto de fondo como de los sprites del juego para luego exportarlos.
- **Game Boy Map Builder** (GBMB): Herramienta open source de creación de mapas de Game Boy. Se utilizará para crear los mapas de cada pantalla utilizando los tiles creados con Game Boy Tile Designer.
- **Aseprite**: Editor de imágenes 2D que se utilizará para hacer bocetos antes de pasarlos a GBTD y posteriormente a formato de RGBDS.

5. Manual de Game Boy

A la hora de comenzar a programar para Game Boy, es posible que los primeros pasos lleven más tiempo de lo normal. Esto se debe en gran parte a la **escasa información** que hay y a cómo se encuentra ordenada. Es muy fácil encontrar una guía que explique el hardware completo de la Game Boy antes de enseñar a pintar un píxel. Por esto, he decidido redactar una pequeña guía técnica, para intentar evitar que nadie se estanque como yo, y dar un pequeño empujón al principio, cuando es más necesario.

La guía va a centrarse en conseguir objetivos simples de la forma más **rápida** y **directa** y explicando como funciona todo sin profundizar en lo secundario, que se verá más adelante.

5.1. Herramientas de desarrollo

Para empezar con el desarrollo en Game Boy, hay tres herramientas de las que no se pueden prescindir: un **editor de texto** con el que programar el código del juego en ensamblador, un **ensamblador** que convierta el texto escrito por una persona en un archivo que el ordenador pueda entender y un **emulador** para poder ejecutarlo. Además de estas 3 herramientas, hay otras herramientas de desarrollo no tan imprescindibles pero que facilitarán el trabajo y ahorrarán bastante tiempo y dolores de cabeza.

Ensamblador

Lo primero de todo es saber qué ensamblador se va a utilizar y cual es la sintaxis del lenguaje que es capaz de entender. Tras investigar un poco, Game Boy tiene un buen número de opciones entre las que elegir:

En primer lugar se encuentra el ensamblador oficial de Game Boy y el recomendado por Nintendo, ISAS (Intelligent Systems ASsembly) y el ISLK (Intelligent Systems LinKer). Esta es probablemente la opción más cara y menos práctica para programar en Game Boy. Para poder crear un ejecutable y probarlo es necesario comprar el kit de desarrollo (SDK) y la licencia del ensamblador. En total está estimado que cuesta unos 1000 dólares como mínimo.

Afortunadamente, hay otras opciones gratis como usar un ensamblador y emulador de código libre. Algunas son ASMotor, wla-dx y SDCC, pero el más conocido, extendido y recomendado es el ensamblador **RGBDS** (RedneX Game Boy Development System). De hecho es hasta considerado mejor que el recomendado por Nintendo, aunque sea gratis.

RGBDS es un conjunto de tres herramientas:

- RGBASM, el ensamblador que genera los objetos a partir de cada fichero en ensamblador.
- RGBLINK, el linker o enlazador que se encarga de combinar los objetos para crear la rom del juego.
- RGBFIX, el validador que corrige la cabecera de la rom.

Una vez que el código fuente haya pasado por estas tres herramientas, se habrá generado un ejecutable listo para probar en el emulador.

Editor de texto

Este es probablemente el apartado de elección libre que más opciones tiene. Hay infinitos editores para programar con distintos tipos de lenguajes, de entre los que destacan emacs, vim, visual studio code, atom... Por motivos personales y por comodidad, para este proyecto se ha elegido utilizar **Sublime Text 3**.

Todos estos editores poseen hojas de estilos que colorean la sintaxis del ensamblador, siendo de gran ayuda a la hora de programar. Además, en el caso particular de Sublime Text, fue bastante fácil editar un archivo de estilos y crear uno propio que se ajustase a lo deseado.

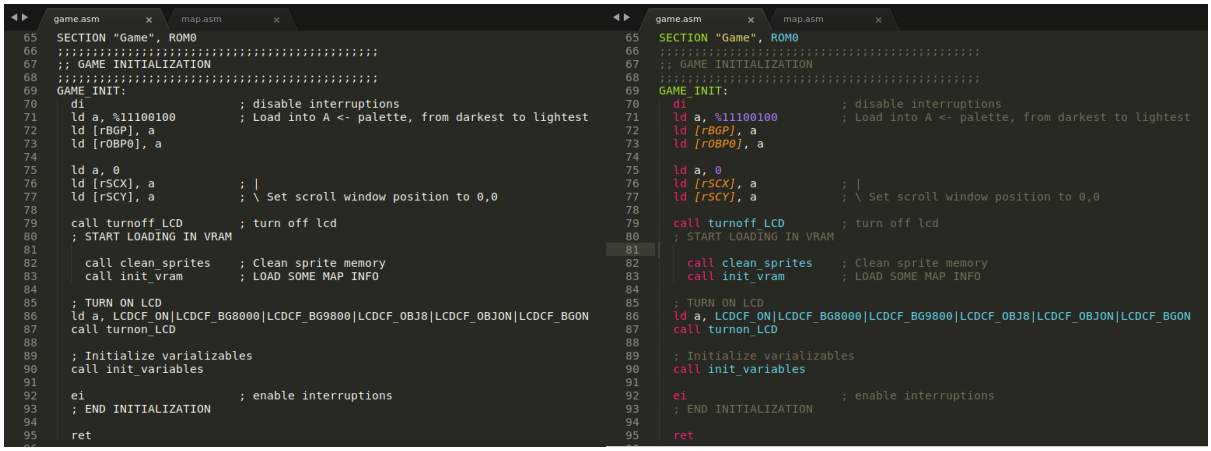


Figura 5.1.: Código sin estilo (izq.) y con estilo (der.) en Sublime Text 3

La única pega que se le podría poner a este programa es el mensaje que salta de vez en cuando en la versión gratuita, con publicidad para comprar la versión completa. Por lo general es un programa muy completo y altamente configurable.

Emulador

Al igual que con los editores de texto, al ser la Game Boy una videoconsola tan popular, hay muchísimos emuladores capaces de ejecutar la gran mayoría de juegos. Los emuladores más populares son SameBoy y BizHawk (Gambatte), pero el más recomendado en general es **BGB**. Este emulador destaca por su práctico visor de VRAM que muestra en todo momento lo que se está dibujando por la pantalla con detalle, y por su potente debugger.

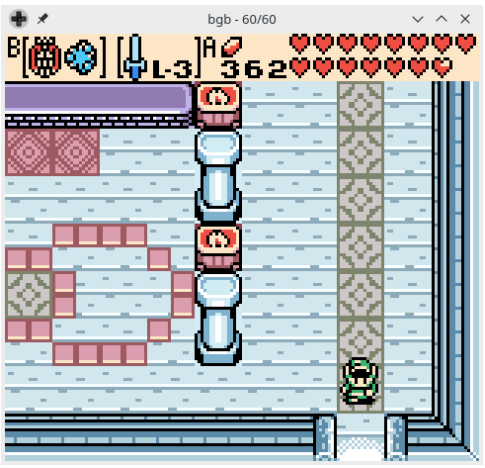


Figura 5.2.: Emulador BGB ejecutando The Legend of Zelda: Oracle of Ages

El visor de VRAM consta de 4 pestañas: mapa, memoria de tiles, memoria de sprites y paletas de color.

En la primera pestaña, se puede ver el **mapa de fondo** de Game Boy, junto al cuadrado que representa lo que se ve por la pantalla. Esto es especialmente útil para comprobar que se dibuja el fondo o que se hace el scroll de manera correcta.

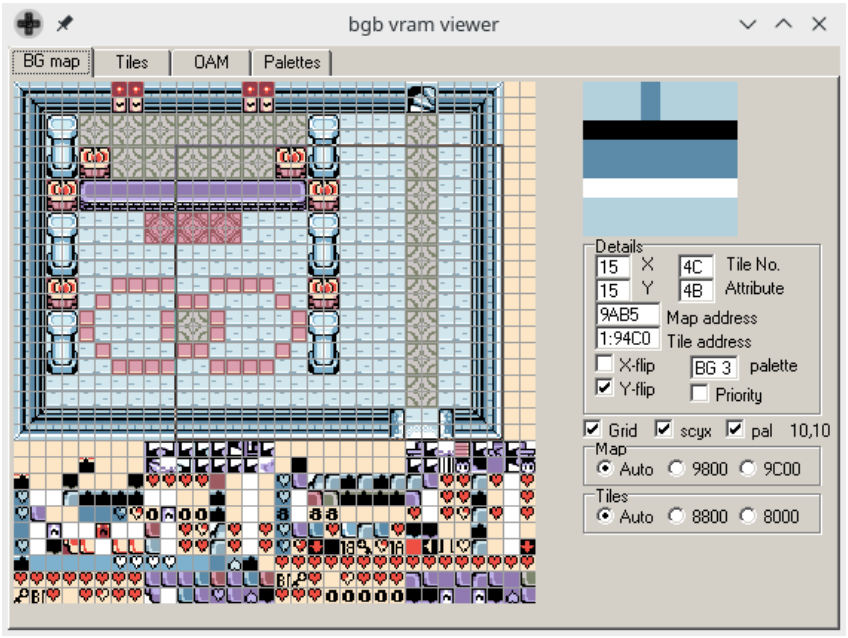


Figura 5.3.: Visor de VRAM: Mapa

En la segunda pestaña, se pueden analizar los distintos **tiles** que están cargados en memoria. Cada tile que está siendo mostrado por la pantalla se colorea para diferenciarlo de los demás. Es muy útil para comprobar que se han cargado bien los datos en memoria cuando se ha hecho alguna modificación o para consultar el número del índice de algún tile en específico.

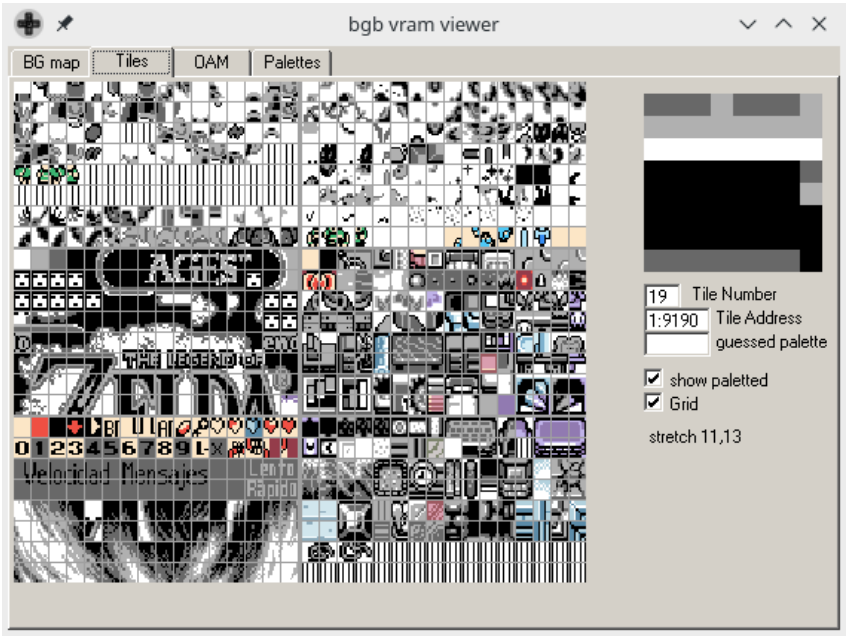


Figura 5.4.: Visor de VRAM: Tiles

En la tercera pestaña, se pueden ver los **sprites** asignados y todos sus atributos. Los sprites

que no están siendo utilizados o están fuera de la pantalla están tachados con una línea roja. Esta pestaña es muy interesante para entender cómo los juegos manejan los elementos que se desplazan por el mapa como balas o enemigos y sus animaciones.

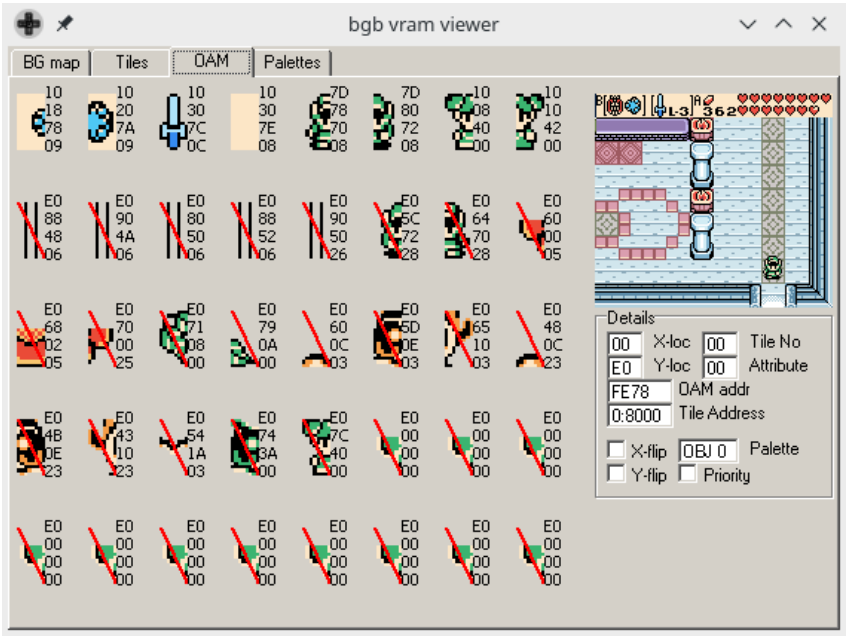


Figura 5.5.: Visor de VRAM: Sprites

Por último, la pestaña dedicada a las **paletas de colores**. Dependiendo de si estamos en un juego para Game Boy o Game Boy Color habrán más o menos. En la Game Boy clásica solo hay 3 paletas de colores: 1 para el fondo y 2 para los objetos.

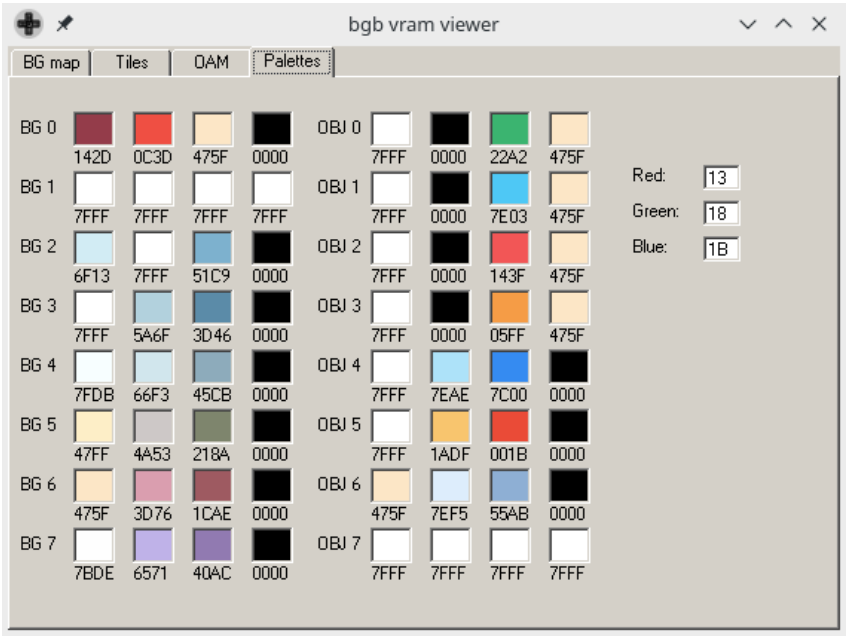


Figura 5.6.: Visor de VRAM: Paletas de colores

Además del visor de la VRAM, el debugger es sin lugar a dudas una de las funcionalidades que más caracteriza y más **valor le da al emulador**. El debugger se utiliza para examinar determinadas partes de un juego y ejecutar el código paso a paso mientras se inspeccionan todos los datos que se modifican para comprobar que se hagan las operaciones correctamente.

El debugger se puede dividir en 4 partes:

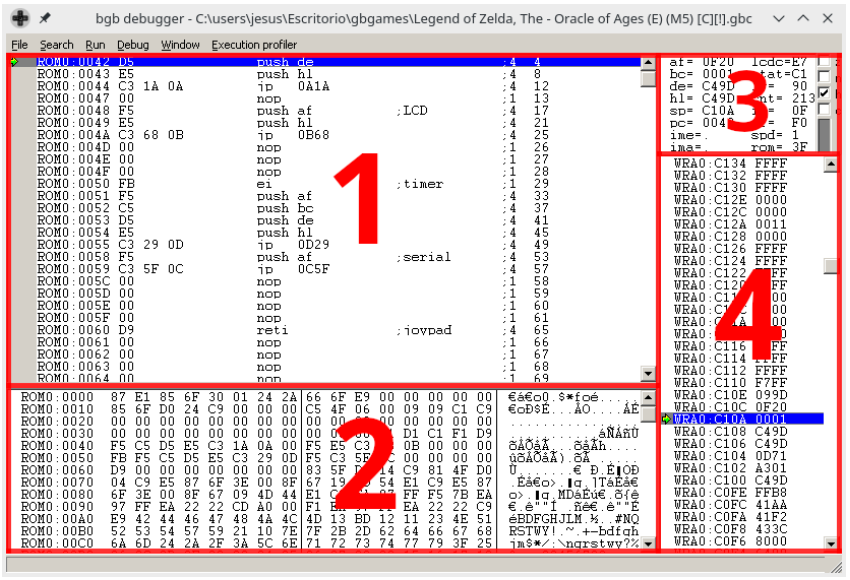


Figura 5.7.: Debugger de BGB

1. Aquí se muestra el código ensamblador y la línea que se está ejecutando en ese momento. Al pulsar **F3**, se avanza a la operación **de la siguiente línea**. Con **F7** se avanza a la siguiente linea entrando a las rutinas y con **F9** se sigue con la ejecución del programa hasta el siguiente punto de parada. Por último, al pulsar **F2** o al hacer doble click en una línea se establece un **punto de parada**. Así, cuando el ciclo del juego alcance dicha línea, el programa se detendrá.

2. En esta parte está el **visor de memoria**. Se puede inspeccionar cualquier dirección y ver los valores en hexadecimal. Es muy útil para poder inspeccionar datos en la memoria. Se suele utilizar muy frecuentemente a la misma vez que con el número 1 para ver el código y datos al mismo tiempo.

3. En esta pequeña esquina se encuentra el **visor de registros**. Esta ventana es la más usada a la hora de intentar depurar cualquier código y asegurarse de que las operaciones dan el resultado esperado.

4. Por último, la **pila**. Indica el valor de la dirección a la que está apuntando el registro SP y el del resto de direcciones si se han estado usando las operaciones PUSH y POP.

Game Boy Tile Designer

Esta es una de las herramientas que más tiempo llega a ahorrar de todo el proyecto. Es un editor de tiles muy sencillo que puede crear tiles de 8x8, 8x16, 16x16 e incluso 32x32, tanto en Game Boy clásica como en Game Boy Color. Tiene las típicas herramientas de un editor de imágenes: lápiz, borrador, cubo de pintura, cuentagotas y otras herramientas menos comunes como invertir el sprite, rotar 90 grados y scroll a una dirección.

Una vez se hayan dibujado todos los tiles, se pueden exportar fácilmente a hexadecimal para usarlos directamente en el juego.

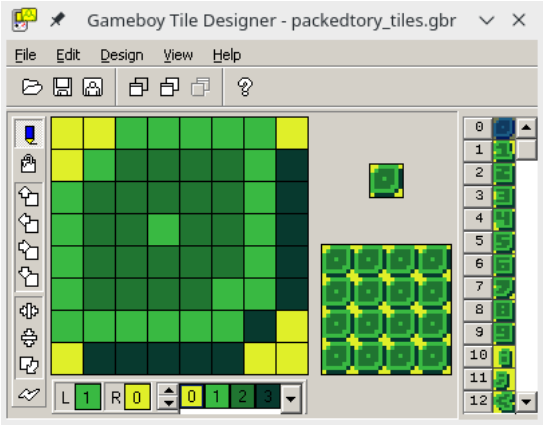


Figura 5.8.: Game Boy Tile Designer

Aseprite

Para diseñar algunos de los gráficos del juego, un editor especializado en Pixel Art como Aseprite hizo el trabajo. Aseprite es un editor de sprites con muchas funcionalidades y con una comunidad enorme de usuarios. Es muy simple de utilizar, tiene atajos de teclado muy similares a la mayoría de los programas y se pueden configurar rejillas y paletas de colores para hacerlo lo más parecido posible a Game Boy.

Principalmente fue útil para hacer bocetos rápidos del mapa situando todos los elementos y preparar las tilesheets para pasarlas a Game Boy Tile Designer más rápido.

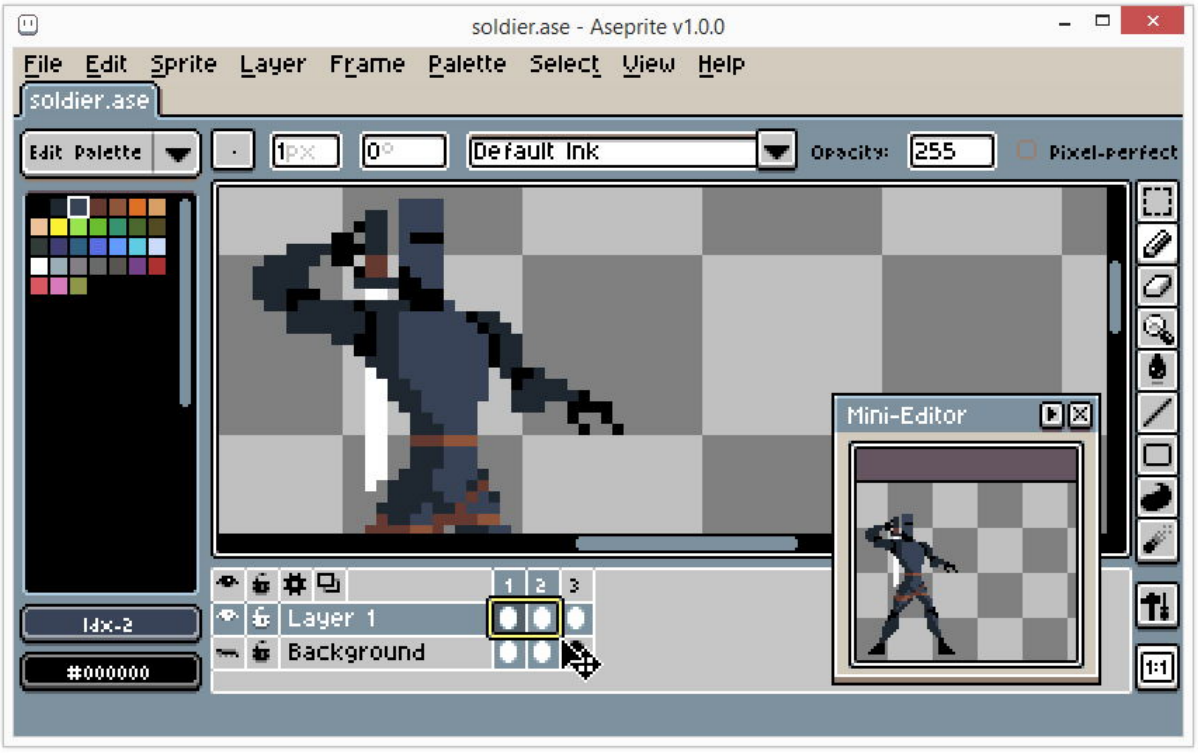


Figura 5.9.: Aseprite

Game Boy Map Builder

Game Boy Map Builder es un perfecto complemento de la herramienta anterior **GBTD!** (**GBTD!**). Sirve para construir de manera muy sencilla mapas de fondo. Solo hay que cargar el archivo creado con GBTD a modo de paleta de tiles y empezar a "pintar" en el mapa. Se puede variar el tamaño del lienzo por si el juego permite más de 32x32 tiles.

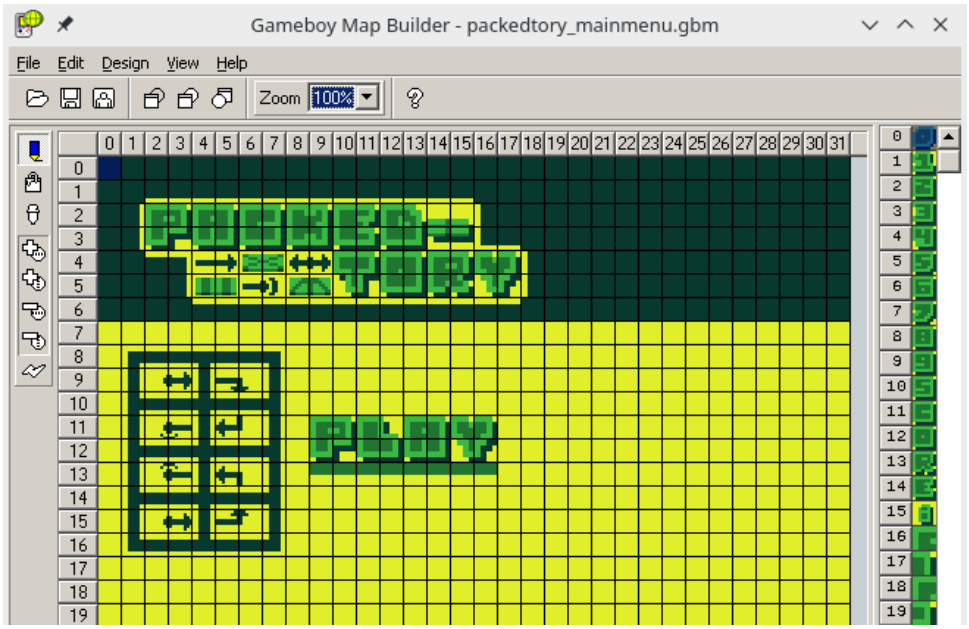


Figura 5.10.: Game Boy Map Builder

Flash Cart

El único inconveniente de trabajar con un emulador es que no es exactamente igual a una Game Boy real. La mejor solución para este problema es una Flash Cart que permita jugar directamente en una Game Boy y pueda asegurar su compatibilidad.

Hay varios tipos de cartuchos pero el más popular y facil de encontrar es el llamado Everdrive, como el de la Figura 5.11, y funciona perfectamente. Se puede utilizar en cualquier versión de Game Boy, desde la original hasta la Game Boy Advance. Solo hay que guardar las roms en una tarjeta micro SD, conectar la tarjeta al cartucho y este a la consola.

Figura 5.11.: Everdrive para Game Boy

5.2. Crear una ROM

El primer paso para comenzar a desarrollar un videojuego es la creación del archivo que va a ejecutar el código en la consola. Este archivo es conocido como memoria de solo lectura o Read-Only Memory (ROM). Este nombre se debe a que este archivo es el que va a ser copiado en el chip del cartucho que contiene datos de solo lectura.

Antes de continuar, se recomienda echar un vistazo al capítulo de metodología y a la hoja de acrónimos al final del documento por si hay algún término o acrónimo que no esté claro. A partir de ahora, los números serán representados con la sintaxis del ensamblador RGBDS: $0A es un número en hexadecimal, %101 es un número binario y 15 es un número decimal.

Si se observa el Listado 5.1, ese es el código mínimo para crear una ROM y empezar a probar con el emulador y la memoria. De momento solo se comentará muy por encima que en la linea 1 se ha incluido un archivo con muchas definiciones del hardware de la Game Boy, en la linea 3 se sitúa un salto al inicio del progama en la posición $0100 de la memoria, en la linea 9 se ha definido una cabecera con la información de la ROM del juego, y un bucle infinito al final que no hace nada.

```
1  INCLUDE "gbhw.inc"  ; archivo con macros y constantes del hardware
2  ; El programa comienza aqui:
3  SECTION "start",ROM0[$0100]
4    nop
5    jp inicio
6  ; Cabecera de la ROM (Macro de gbhw.inc)
7    ROM_HEADER ROM_NOMBC, ROM_SIZE_32KBYTE, RAM_SIZE_0KBYTE
8
9  inicio:  ; aqui empieza nuestro programa
10
11 bucle:   ; bucle infinito
12   halt
13   jr bucle
```

Listado 5.1.: Código mínimo para crear una ROM

Aunque pueda resultar un poco confuso ver todo este código sin explicar para qué sirve, todo estará aclarado con detalle más adelante. Una vez compilada la ROM, ya está lista para ser emulada y analizada.

Cómo se ve una ROM vacía

A simple vista en la Figura 5.12, solo es posible contemplar el logo de Nintendo, pero hay más detalles ocultos interesantes, como el estado de la memoria de tiles o la memoria de sprites. En la Figura 5.13 están las 4 pestañas más útiles a la hora de depurar todo lo relacionado con los gráficos. En orden son: la pestaña del mapa, memoria de tiles, OAM o memoria de atributos de los objetos y la pestaña con las paletas de colores.

Figura 5.12.: ROM de una Game Boy vacía

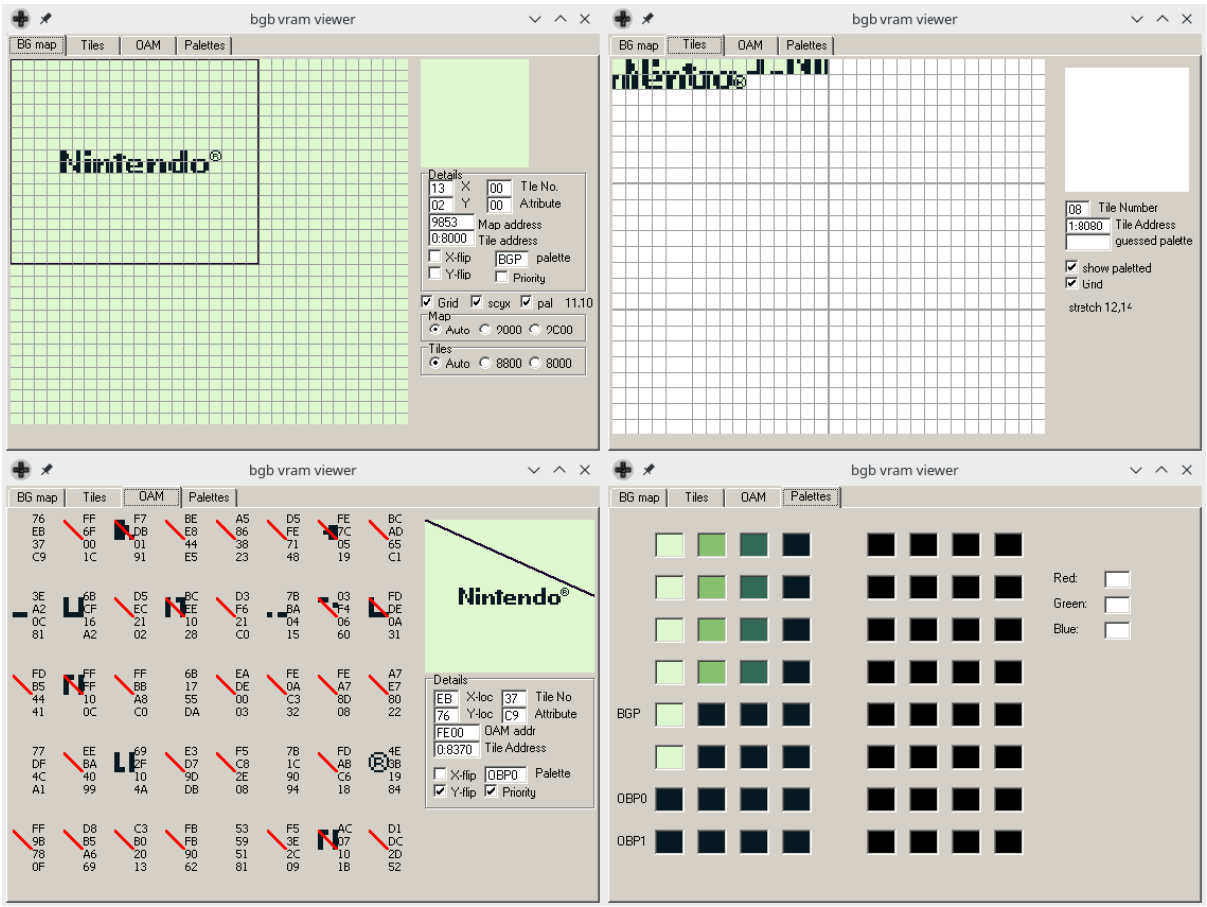


Figura 5.13.: Visor de la VRAM del emulador

Sin más dilación, vamos a centrarnos en el primer objetivo: **Pintar un Tile**.

5.3. Tiles

Qué es un tile

Un tile es la mínima unidad gráfica con la que trabaja la Game Boy. Es una imagen de 8x8 píxeles de hasta cuatro colores diferentes. Todo está compuesto de tiles, tanto el mapa, como los sprites y el Head-Up Display (HUD).

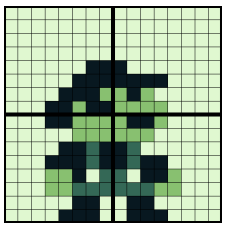


Figura 5.14.: Tiles que componen el sprite de mario en Super Mario Land

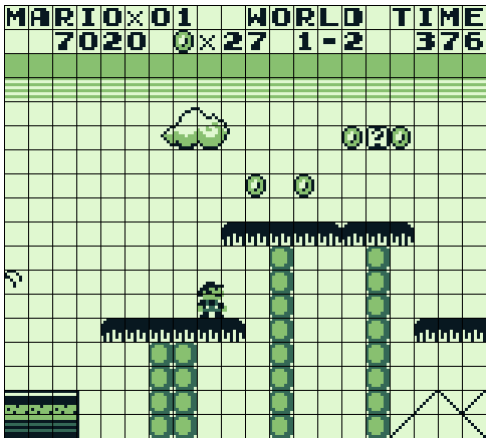


Figura 5.15.: Tiles que componen un mapa de Super Mario Land

Los colores se encuentran codificados de manera que, del más claro al más oscuro, valen respectivamente %00, %01, %10 y %11 (Figura 5.16). De esta manera, se puede representar 1 píxel con solo 2bits. Cada fila de 8 píxeles se define con 2 Bytes, por lo que cada tile ocupa en memoria $10 Bytes (16 Bytes).

Figura 5.16.: Colores codificados de Game Boy

Dónde se guardan

Para entender como funciona el proceso de pintar un Tile, es necesario entender cómo funcionan las 3 partes involucradas en el pintado del mapa y sprites: la **memoria de tiles**, la **memoria del mapa**. y el **LCD**.

La parte de la memoria dónde **se guardan los tiles** que se usarán para el juego es la memoria de tiles. Esta parte de la memoria comienza por defecto en la posición $8000[1] y acaba en la posición $8FFF de la ROM. Tiene $1000(4096) bytes de memoria, es decir, 4KB y permite almacenar hasta 256 tiles diferentes. Cada tile se almacena detrás de otro, de 16 bytes en 16

[1]Más adelante se verá que también se puede cambiar para que empiece en la posición $8800

bytes y a cada uno se le asigna un índice (ver Figura 5.17). De esta manera, en la posición $8000 estará el tile $0, en la posición $8010, el tile $01 y así sucesivamente.

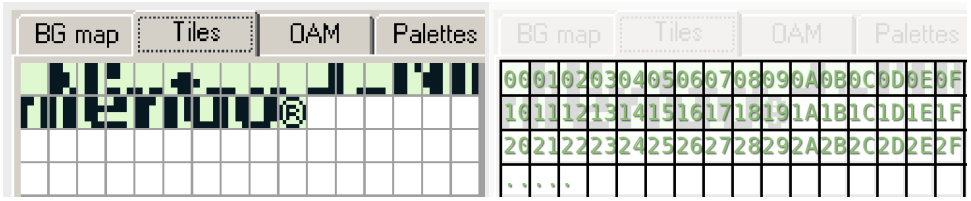


Figura 5.17.: Índices de los tiles de una ROM vacía

Cómo se dibujan

La memoria del mapa (o mapa de la Game Boy) es un mapa de 32x32 tiles que **contiene índices** a la memoria anterior. Cada tile pintado en el mapa es simplemente un byte con su numero y la Game Boy se encarga de dibujarlo automáticamente. El mapa comienza por defecto en la dirección $9800[2] y acaba en la $9BFF de la ROM. Esta parte consta de $400(1024) Bytes, es decir, 1KB. En la Figura 5.18 se pueden ver los índices que se encuentran en el mapa y directamente en la memoria.

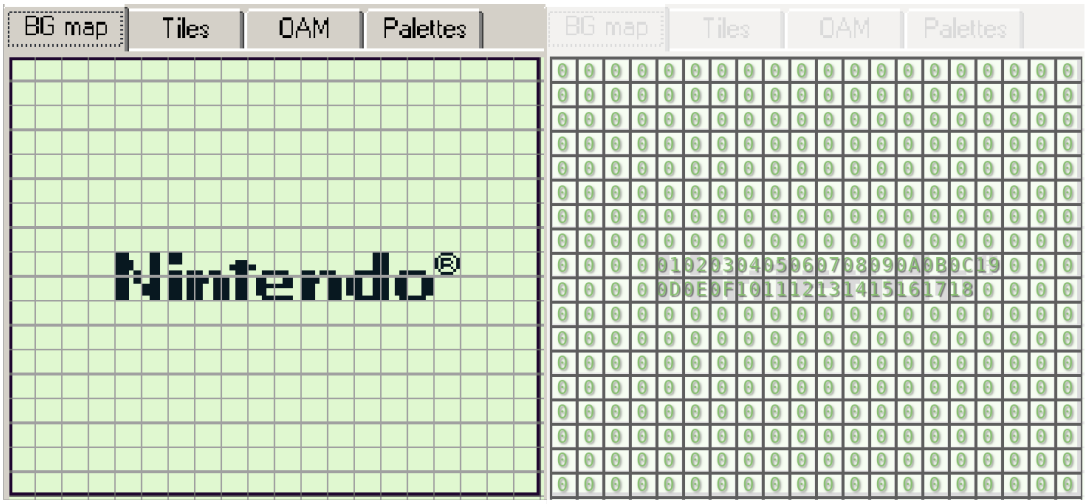


Figura 5.18.: Mapa con los índices de una ROM vacía

Por último, el LCD de la Game Boy es el encargado de controlar **todo lo que se dibuja por pantalla**. Se encarga de encender o apagar la pantalla, mapear la Video RAM (VRAM), cambiar el tamaño de sprites (8x8 o 8x16) y controlar si se dibujan el mapa, sprites, los dos o ninguno. El registro que hace este control se llama LCD Control (LCDC) y se encuentra en la posición $FF40. Cada uno de los 8bits configura todo lo anterior. Además, hay otro registro **muy importante** en la posición $FF44 que indica la posición Y actual del LCD. Este registro va desde 0 a 153 y cuando está entre 144 y 153 indica el periodo V-Blank, es decir, cuando ha terminado de dibujar la pantalla y **es posible escribir en la VRAM**[3].

En la figura 5.13, se puede observar el mapa y la memoria de tiles que se encuentran en la ROM vacía. Por defecto, a no ser que se limpie la memoria, saldrá el logo de Nintendo en primer plano, junto a los tiles que lo componen, y la Object Attribute Map (OAM) llena de datos.

[2] Al igual que la memoria de tiles, también se puede cambiar que el mapa comience en la posición $9C00. Para más información ver el Anexo A.7.6

[3] Normalmente, cuando hay que escribir una cantidad muy grande de bytes no es suficiente esperar al V-Blank, es necesario apagar el LCD y encender cuando se acabe para evitar errores de copiado.

Cómo definir un tile

Al encontrarse codificados los colores, solo son necesarios 2 bits para cada píxel. Por lo tanto, para una fila de 8 píxeles se necesitan 16bits = 2bytes y, para un tile entero, 16 bytes.

A continuación, se puede ver cómo sería la codificación de una fila de 8 píxeles (Figura 5.19) y, más abajo, la de un tile entero (Figura 5.20).

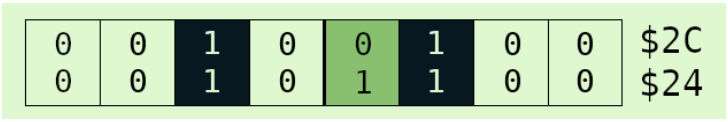


Figura 5.19.: Fila de un tile codificada a hexadecimal

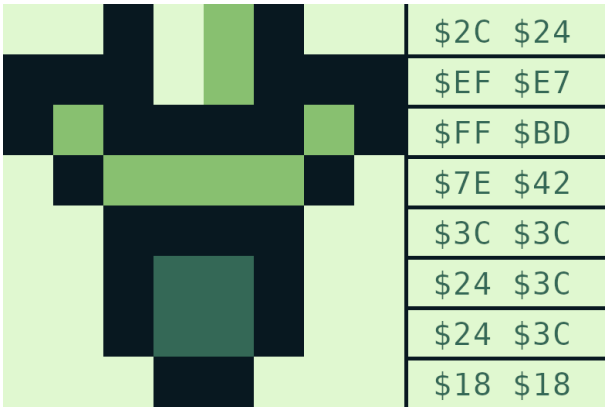


Figura 5.20.: Tile completo codificado a hexadecimal

El primer paso es cambiar los colores por su codificación en binario y sacar los bits para cada color de forma que los cuatro primeros píxeles son representados con el primer número de los dos bytes. En el ejemplo anterior en la Figura 5.19, los cuatro primeros píxeles serían $22 (%0010 y %0010) pero cada dígito está en un byte distinto ($2C y $24).

Solo hay que saber un poco de hexadecimal y seréis capaces de representar cualquier tile. Es posible hacer esto con todos los tiles de nuestro juego pero existiría el riesgo de entrar en depresión. La teoría es necesaria para entender cómo funciona, pero se usarán herramientas que faciliten el trabajo y proporcionen directamente los números en hexadecimal.

Cómo guardar un tile

Volviendo atrás, la posición donde comienza la memoria de tiles por defecto es la $8000. Desde código se hará referencia a ella con la constante `_VRAM`. Lo único que hay que hacer para guardar un tile en la memoria es copiar bytes con un simple bucle en cualquier posición entre la dirección $8000 a la $8FFF. Si se observan las Figuras 5.17 o 5.18 se puede ver que si se copia directamente en la posición $8000 se modificará el tile 0 y se llenará toda la pantalla con él.

Partiendo del código en el Listado 5.1, se van a ir añadiendo las instrucciones necesarias para llevar a cabo nuestro objetivo. El primer paso es esperar al V-Blank. No se debe escribir nada en la VRAM durante este periodo a no ser que se busquen parpadeos. Esto se hace simplemente cargando en A el contenido de la dirección $FF44 (constante `rLY`) y un bucle comprobando que sea igual a 145, justo el inicio del V-Blank.

```
..
; inicialización
  call espera_VBlank          ; esperamos a que el LCD termine de dibujar
  ..

; rutina para esperar a que vuelva al inicio el LCD
espera_VBlank:
  ld      a, [rLY]
  cp      145
  jr      nz, espera_VBlank
  ret                         ; volvemos
```

Listado 5.2.: Función para esperar al V-Blank

Ahora, en el Listado 5.3 se encuentra la función para copiar los bytes a la posición $19 de los tiles. Un bucle muy facilito sabiendo ensamblador. Se incrementa HL y DE y se copia en DE lo que hay en HL a través de A, B veces.

En la Figura 5.21 se puede ver el resultado de modificar el tile $00 que se usaba como tile de fondo. Este horror visual se puede arreglar fácilmente desplazando un poco la dirección en la que se guarda el tile.

```
..
  ; cargamos el tile en la memoria de tiles
  ld      hl, Tiles           ; HL << Dirección de los bytes del tile
  ld      de, _VRAM           ; DE << Dirección de la VRAM
  ld      b, $10              ; B << 16, numero de bytes

.bucle_carga:
  ld      a,[hl]              ; A << datos apuntado por HL
  ld      [de], a             ; [DE] << A (bytes del sprites)
  dec     b                   ; B--
  jr      z, .fin_bucle_carga ; si B = 0, terminamos
  inc     hl                  ; HL++
  inc     de                  ; DE++
  jr      .bucle_carga        ; seguimos
.fin_bucle_carga:
  ..

; Datos de nuestro tile
Tiles:
DB $2C,$24,$EF,$E7,$FF,$BD,$7E,$42
DB $3C,$3C,$24,$3C,$24,$3C,$18,$18
EndTiles:
```

Listado 5.3.: Función para cargar un tile en la posición $8000

Normalmente en cualquier juego, antes de hacer nada, lo que se suele hacer es limpiar toda la VRAM. Sin embargo, para hacerlo un poco más artístico, se puede modificar el tile de Marca Registrada (®). Este tiene el índice $19 por lo que su dirección será $8000 + $0190 = $8190 (cada tile ocupa $10 bytes). Se carga esta dirección en DE y se vuelve a ejecutar en el emulador.

Figura 5.21.: Mapa inundado al modificar el tile $00

```
..
  ld      de, _VRAM + $0190    ; DE << Dirección de la VRAM
..
```

Listado 5.4.: Modificación para cargar el tile en la posición $8190

El resultado, como es visible en la Figura 5.22, es mucho más agradable y se puede diferenciar mejor su forma.

Figura 5.22.: Resultado del mapa al modificar el tile $19

Sin embargo, si se compara esta imagen con la figura en la que se definió el tile, los colores no encajan, faltan colores. En la Figura 5.13, en la pestaña de paletas, se puede ver que solo hay 2 colores diferentes de 4. El problema es que falta **inicializar la paleta de colores de fondo**. Esto es muy sencillo y se hace a través del registro en la posición $FF47 (constante `rBGP`). Los bits de este registro son directamente los bits representados en la Figura 5.16. Los bits 0-1 son

el primer color, los bits 2-3 son el segundo, 4-5 el tercero y 6-7 el cuarto. En el Listado 5.5 está el código, simplemente dos lineas y el resultado final en la Figura 5.23.

```
..
  call espera_VBlank           ; esperamos a que el LCD termine de dibujar

  ; Cargar paleta de colores, de más oscuro a más claro
  ld a, %11100100
  ld [rBGP], a
..
```

Listado 5.5.: Código para inicializar paleta de colores

Figura 5.23.: Resultado del mapa al inicializar la paleta de colores

Cómo pintar un tile

Todo esto es lo necesario para guardar un tile, pero realmente solo se ha modificado uno que **ya estaba pintado** en el mapa. Para pintar nuevos hay que cargar simplemente un índice (de $0 a $FF) en el byte del mapa deseado. Ahora se va a codificar un nuevo sprite, la parte de arriba de la espada (Figura 5.24), y a dibujarlo justo encima del actual.

El mapa comienza en la posición número $9800 de la ROM (constante `_SCRN0`) y hay que calcular la posición a la que va a ir el nuevo tile. Si se observa el VRAM viewer (la ventana de la Figura 5.13) y se sitúa el ratón encima del tile ya pintado, se indica que su posición en la memoria es la de $9910 y justo arriba, la $98F0. Para subir una fila simplemente hay que restar $20 (una fila tiene 32 tiles). $9910 - $0020 = $98F0. Ahora solo falta el siguiente tile que se quiere dibujar. Para simplificar el código, se van a escribir los bytes de los dos tiles juntos en memoria y a duplicar el contador del bucle de copiar bytes, obteniendo así que el siguiente tile tendrá el índice de $1A.

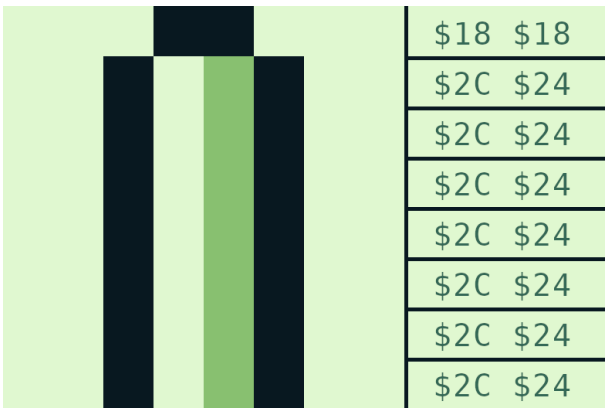


Figura 5.24.: Segundo tile codificado a hexadecimal

```
..
Tiles:
DB $2C,$24,$EF,$E7,$FF,$BD,$7E,$42
DB $3C,$3C,$24,$3C,$24,$3C,$18,$18

DB $18,$18,$2C,$24,$2C,$24,$2C,$24
DB $2C,$24,$2C,$24,$2C,$24,$2C,$24
EndTiles:
```

Listado 5.6.: Código con los dos tiles juntos

Un pequeño truco para no tener que calcular el tamaño de los bytes que se quiera copiar es cambiar el numero por una resta de directivas, es decir, de posiciones de memoria y se calculará solo automáticamente (ver Listado 5.7)

```
..
  ;ld     b,  $10               ; B << 16, numero de bytes
  ld      b, EndTiles - Tiles    ; B << 32, numero de bytes
..
```

Listado 5.7.: Pequeño truco para calcular el tamaño de los bytes automáticamente

Para pintar el nuevo tile en el mapa, en vez de escribir $98F0, se ha utilizado la constante y separado en una suma (Listado 5.8), para tener más claro que se está escribiendo en la pantalla.

El resultado se muestra en la Figura 5.25

```
..
  ld hl, _SCRN0 + $F0
  ld [hl], $1A
..
```

Listado 5.8.: Código para pintar en el mapa un tile nuevo

Figura 5.25.: Resultado final

5.4. Sprites

En Game Boy existen 3 capas de gráficos: la más baja es la del mapa de fondo, vista anteriormente. Por encima del mapa está la capa de la ventana[4], muy parecida al mapa de fondo que se puede colocar libremente y utilizarse como HUD y, por encima de todo, están los sprites.

Qué es un sprite

Los sprites son objetos en la pantalla que no encajan en los límites del mapa de 8x8 píxeles y se pueden mover libremente. Normalmente son utilizados para dibujar el personaje principal, enemigos, misiles o cualquier otro objeto que tenga movimiento.

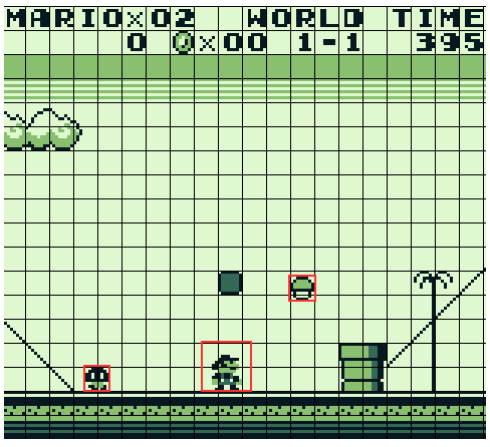


Figura 5.26.: Sprites usados en Super Mario Land

El controlador de video de Game Boy puede dibujar hasta 40 sprites a la vez, pero por limitaciones del hardware solo pueden dibujarse hasta 10 sprites en la misma linea.

[4]Ver Anexo A.7.4 para más información sobre la ventana

Dónde se guardan

La información de los sprites se encuentra en el Mapa de Atributos de Objetos, conocido como OAM. Aquí están definidos todos los sprites uno a continuación de otro. Cada sprite ocupa 4 bytes y en total, sumando los 40 sprites, la OAM ocupa $A0 (160) bytes.

Los gráficos de los sprites, se guardan en memoria en el mismo formato que los tiles de fondo y en la misma posición. Por lo tanto, a la hora de diseñar los gráficos hay que tener en cuenta que los 256 tiles[5] posibles van a ser utilizados por el mapa y los sprites.

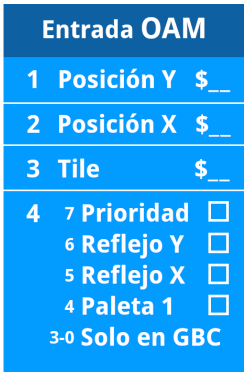


Figura 5.27.: OAM

Cada sprite cuenta con 4 Bytes que definen lo siguiente:

- Byte 1: Posición Y

  Especifica la posición Y del sprite menos 16 ($10). Esto quiere decir que el primer píxel vertical visible en pantalla es el 16, esto es así para poder poner el sprite un poco o totalmente fuera de la pantalla, dibujándolo entre las posiciones 0-15. ¿Por qué 16 en vez de 8? Porque los sprites pueden ser de 8x8 o de 8x16 y es necesaria una manera de sacar ambos de la pantalla.

- Byte 2: Posición X

  Especifica la posición X del sprite menos 8.

- Byte 3: Número de sprite

  Índice del tile guardado en la memoria de tiles que se va a dibujar.

- Byte 4: Atributos de sprite

  Define los atributos del sprite en base a los bits activados.

Los atributos que tienen los sprites en GB son los cuatro de la figura 5.27: Prioridad de dibujado, Reflejo Y, Reflejo X y Paleta 1.

El bit de prioridad define la "capa" en la que se va a dibujar el sprite. Por defecto, el sprite se dibuja por encima de todos los colores de la paleta de fondo, pero si se activa este bit, pasará a dibujarse por debajo de los colores 1, 2 y 3 de la paleta.

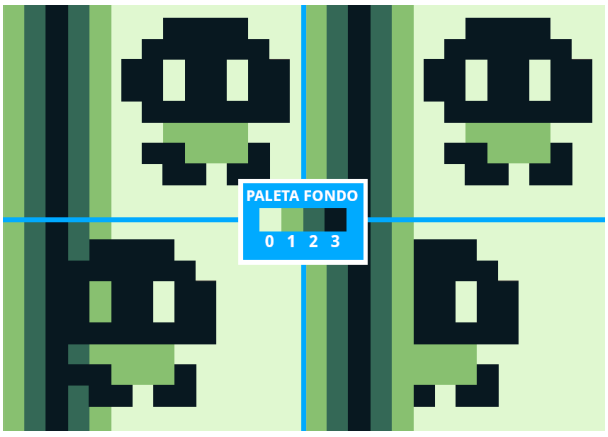


Figura 5.28.: Bit de prioridad desactivado (izq.) y bit de prioridad activado (der.)

Los bits de espejar se pueden combinar y son muy útiles para ahorrar espacio en la memoria de tiles. Con solo un tile podemos sacar **hasta 4 diferentes**, modificando solo dos bits. En la

[5]En el Anexo A.7.6 puede encontrarse más información sobre la memoria de vídeo

Figura 5.29 se puede ver su funcionamiento en el caso de un enemigo (Goomba) de Super Mario Land. Este sprite, dependiendo de la dirección a donde se mueva, se espejará horizontalmente, activando o desactivando el bit.

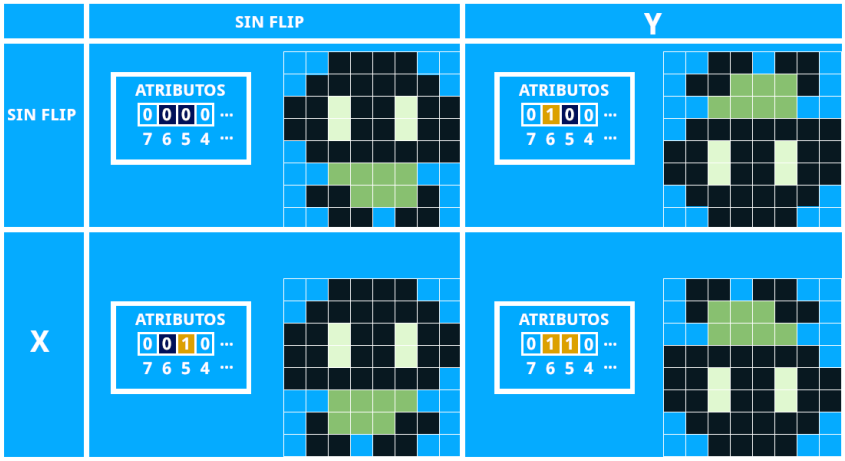


Figura 5.29.: Funcionamiento de del espejado de los sprites

El bit de la paleta de colores indica qué paleta de colores se debe usar. Por defecto todos los sprites tienen transparencia en el color que está en la posición 0 de su paleta. Esto quiere decir que los sprites solo cuentan con **3 colores + 1 transparente**. Las dos paletas de colores de sprites se encuentran situadas en las posiciones $FF48 y $FF49 (constantes `rOBP0` y `rOBP1`)

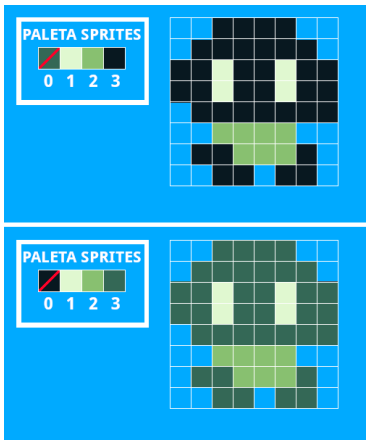


Figura 5.30.: Funcionamiento de las paletas de los sprites

Se puede encontrar más información acerca de los bits del 3 al 0 en el Anexo A.7.5.

Cómo se dibujan

Para dibujar un sprite simplemente hay que definirlo en la OAM de la misma forma que en la Figura 5.27 y con solo estar en una posición dentro de la pantalla, se dibujará automáticamente, si está activado el bit de sprites del registro del LCDC[6].

Este registro está situado en la posición $FF40 y el bit encargado de controlar si se dibujan los sprites o no es el **bit 1**. Cuando está a 0, no se dibuja ningún sprite, solo cuando está asignado a 1.

[6]El LCD Control (LCDC) se introdujo en el apartado de cómo dibujar tiles (sección 5.3, y se puede encontrar más información en el Anexo A.7.1)

Cómo definir un sprite

La memoria de atributos de los sprites está fija en la posición $FE00, consta de (40 sprites x 4 Bytes = 160 ($A0) Bytes) y acaba en $FE9F. Aquí se encuentran definidos todos los sprites uno detrás de otro. Para definir uno es tan simple como rellenar los cuatro primeros bytes de la OAM.

```
..
  ; definir sprite
  ld hl, _OAMRAM
  ld a, $10
  ld [hl+], a   ; Y = 16
  ld a, $08
  ld [hl+], a   ; X = 8
  ld a, $19
  ld [hl+], a   ; Tile = (R) tile
  ld a, $00
  ld [hl+], a   ; Atributos = 0
  ..
```

Listado 5.9.: Código para definir un sprite

En el Listado 5.9 se puede observar como se define un sprite en la posición Y=16 y X=8, lo cual significa que estará en la esquina superior izquierda y se le asigna el tile $19, por lo que tendrá el tile en la posición 25 (el tile del mango de la espada del ejemplo anterior o el tile de copyright si no se ha modificado la memoria).

Cómo dibujar un sprite

Para asegurarse de que los sprites son dibujados correctamente, hay que comprobar en primer lugar que el LCD tiene el bit de dibujar sprites asignado y que la paleta de colores de sprites está inicializada (Listado 5.10) y que, efectivamente, están los sprites definidos.

```
..
  ; Cargar paletas de colores, de más oscuro a más claro
  ld a, %11100100
  ld [rBGP], a
  ld [rOBP0], a

  call apaga_lcd              ; apagar el lcd
  ; definir sprite
  ..
  ld a, %10010011
  ld [rLCDC], a               ; encender lcd y dibujar sprites
..
```

Listado 5.10.: Código para dibujar un sprite

Una vez esté encendido el LCD de la Game Boy con los sprites activados, se debe ver correctamente el sprite en la posición superior. Sin embargo, sin haber inicializado la memoria de objetos, lo más probable que ocurra es que se vea el sprite junto a otros sprites aleatorios. Para evitar esto, simplemente habría que inicializar toda la memoria con valores '$00' (antes de definir el sprite). Se podría hacer perfectamente con la misma función que se ha utilizado para

copiar los datos a la memoria de tiles (Listado 5.3) pero con un pequeño cambio: por ejemplo, que en vez de copiar datos desde `HL` a `DE`, copie ceros desde `A` a `DE`.

En la Figura 5.31 se puede ver la diferencia entre memoria de objetos sin limpiar y memoria de objetos inicializada y, en la Figura 5.32, el resultado del sprite dibujado. Este sprite ya se puede mover por toda la pantalla, independientemente del mapa, y también se pueden cambiar sus atributos en cualquier momento (siempre que sea en un periodo que se pueda escribir en la OAM[7]).

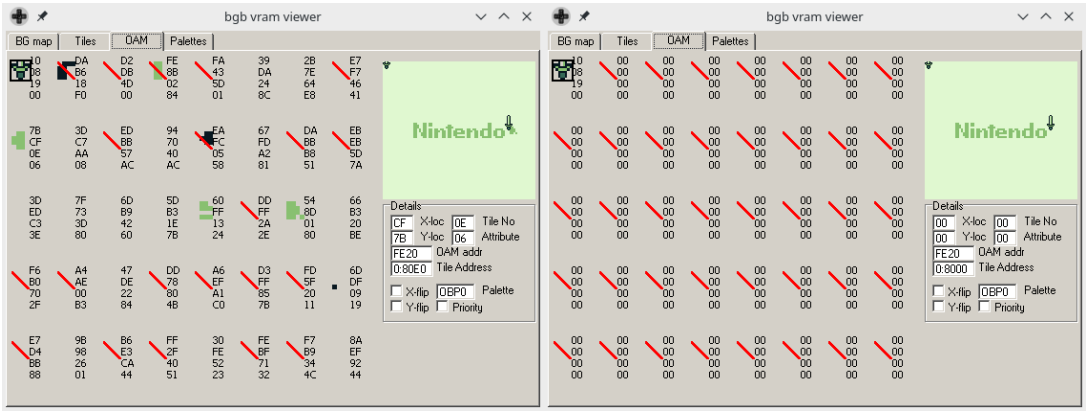


Figura 5.31.: Memoria de objetos sin limpiar (izq.) y memoria inicializada a cero (der.)

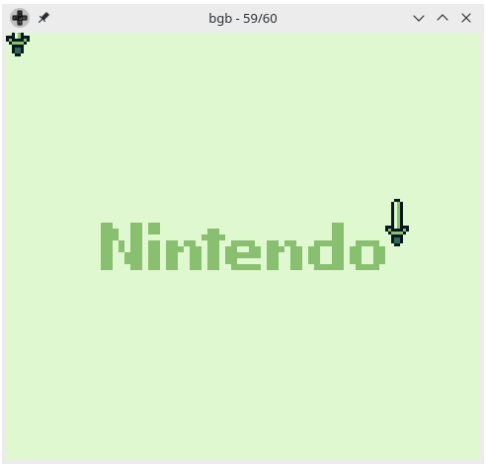


Figura 5.32.: Sprite dibujado en la parte superior izquierda

[7]En el Anexo A.7.7 se puede encontrar más información sobre la VRAM y cómo acceder a ella

5.5. Interrupciones

5.5.1. Timer

Para cualquier juego, tener el control del tiempo es algo vital para poder implementar algo tan importante como las animaciones, como algún cronómetro dentro del juego o un contador de tiempo de invulnerabilidad. En Game Boy existe un elemento llamado timer que nos ayudará.

Game Boy tiene unas posiciones reservadas desde la posición $0040 hasta la $0060 a las que va saltando en cuanto sucede cualquier condición propia de una interrupción activada (puede ser en cualquier momento del juego). Una vez en la posición, ejecuta el código de la interrupción y, al terminar, vuelve al código que se estaba ejecutando antes de la interrupción. Las interrupciones se activan asignando los bits del registro `rIE` en la posición $FFFF[8].

Cómo funciona el Timer

La interrupción del timer funciona incrementando en 1 el registro `rTIMA` que al ser un registro de 8 bits puede valer desde 0 a 255. Cuando se desborda vuelve a 0 y se produce la interrupción, saltando hasta la posición $0050. Esta interrupción se puede controlar con otros dos registros más: el `rTAC`, que controla si está encendido y la frecuencia con la que es incrementado, y el `rTMA` que es el valor inicial del timer cuando desborda.

Para controlar que ha pasado 1 segundo en la Game Boy, hay varias formas de hacerlo. La siguiente sería una de ellas:

1. Activar el timer en el registro TAC y asignar la primera frecuencia de 4096Hz (con esto se consigue que se incremente TIMA a 1/4096 veces por segundo, 1 vez cada 0,2414 milisegundos)

2. Calcular el valor inicial que debe de estar en TMA: con el valor de 255 habría una interrupción cada vez que se incremente TIMA (0,2414 ms) y con el valor de 0, una cada 62,25 ms. Es más interesante buscar un número más conveniente como 50 ms. Con una simple regla de tres se obtiene que el valor que tiene que tener TIMA cuando desborde es 204, por lo que su valor inicial tras desbordar debería ser 51 (255-204).

3. Una vez que haya una interrupción cada 50 milisegundos, solo hay que tener un contador que cuente 20 veces la interrupción y ya habrá pasado 1 segundo (20*50 = 1000ms = 1s).

Cómo definir la interrupción del Timer

Para realizar todo esto en Game Boy, lo primero que hay que hacer es iniciar tanto la interrupción del timer como el timer en sí y los valores para la velocidad.

```
timer_init:
  ld a, %00000100   ; bit 2
  ld [rIE], a       ; activar interrupcion del timer
  ld [rTAC], a      ; encender el timer
  ld a, 51
  ld [rTMA], a      ; cuando TIMA se desborde, este es su
                    ; valor inicial, (1/4096)*(255-51) = 0.049 s
  ld [rTIMA], a     ; tiempo inicial (solo primera interrupcion)
  ld a, 0
  ld [_CTIMER], a   ; Contador del timer
  ret
```

[8] Para más información de las interrupciones, ver el Anexo A.6

Listado 5.11.: Función para iniciar el contador

Una vez inicializado el timer y activada su interrupción, cada vez que se desborde el TIMA (cada 0.05s en nuestro caso) se saltará a la posición $0050 de la memoria. Sin embargo, si no se ha definido ningún código en esa posición, no hará nada la interrupción. Por lo tanto, hay que crear el código necesario para que llame a una función propia que controla los segundos. En el Listado 5.12 se define el código en la posición $0050.

```
SECTION "TimerInt",ROM0[$0050]
  call TIMER_CONTROL
  reti
```

Listado 5.12.: Código para definir la función para la interrupción del timer

Por último, para comprobar los segundos, hace falta una función que solamente incremente el contador de control del timer hasta llegar a 20 veces. Una vez que llegue a 20, habrá ocurrido 1 segundo de tiempo.

```
TIMER_CONTROL:
  ld      a, [_CTIMER]
  cp      20
  jr      z, .inc_seconds    ; if a == 20, pasa 1 segundo
    inc   a
    ld    [_CTIMER], a       ; si no, se incrementa el control
    jr    .tend
.inc_seconds
  ld      a, 0
  ld      [_CTIMER], a       ; reiniciar el contador
  ld      a, [SECONDS]
  inc     a
  ld      [SECONDS], a       ; incrementar los segundos
.tend
  ret
```

Listado 5.13.: Función para actualizar los segundos

Al poder saltar en cualquier momento del código las interrupciones, es muy importante no olvidar **hacer push y pop de todos los registros que se utilicen**. Con algo tan simple como esto se consigue que todos los registros vuelvan a su estado anterior a la interrupción y se pueden evitar errores muy grandes.

```
TIMER_CONTROL:
  push af
  ...
  pop af
  ret
```

Listado 5.14.: Importante no olvidar guardar los registros

6. Diseño

6.1. Introducción

Packedtory es un juego de gestión del espacio en una fábrica de transporte cajas para sobrevivir el máximo tiempo posible en la industria.

Packedtory es una adaptación de un juego hecho para ordenador llamado Compactory, desarrollado por *NateLargo*. Estos juego comparten muchas mecánicas, pero se tuvieron que hacer bastantes adaptaciones ya que un ordenador moderno y una Game Boy son dos plataformas con muchas diferencias.

6.2. Historia

Todo comenzó con el boom de la construcción de los años 80. Se disparó el nivel de vida durante unos años y la logística de aquella época no estaba preparada para semejante cantidad de demandas de recursos, por lo que tuvo que crearse una nueva red de transportes para satisfacer los pedidos. El jugador se encuentra en el centro de todo el problema y se tiene que poner bajo la piel del jefe de logística de una conocida empresa de paquetería, Packedtory.

6.3. Características del Gameplay

Compactory es un juego donde toda la acción sucede en una fábrica, un **mapa estático de 6x6 casillas** que el jugador puede modificar a su gusto. En este mapa van apareciendo diferentes **puntos de entrada**, que van añadiendo cajas al mapa, y **puntos de salida**, que las quitan del mapa.

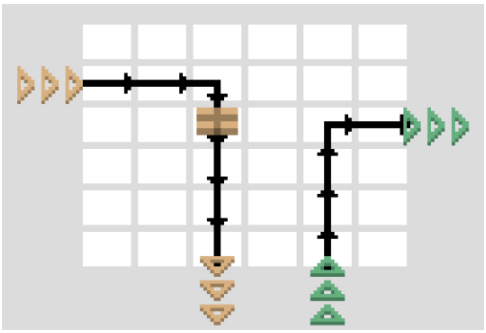


Figura 6.1.: Mecánica principal del juego

6.3.1. Puntos de entrada y salida

En la parte superior de la ventana hay situada una barra de tiempo (ver Figura 6.6) que indica cuanto falta para el comienzo de la siguiente ronda. En cada ronda aparece una nueva pareja de punto de entrada y punto de salida en una **posición al azar**. Al ser el mapa de 6x6 y haber 6 puntos por cada uno de los 4 lados, pueden haber un total de 24 puntos diferentes y, al agruparse por parejas, se puede deducir que hay un total de 12 rondas.

Cada punto tiene asignada una casilla y el jugador tiene que conectarlas creando un camino con **cintas transportadoras** en el mapa. Todos los puntos pueden tener hasta 6 colores diferentes y los puntos de salida solo aceptan cajas de su color. El espacio es limitado y el jugador tiene que llevar cuidado de diseñar rutas que conecten todos los puntos sin que se sature el mapa de cajas.

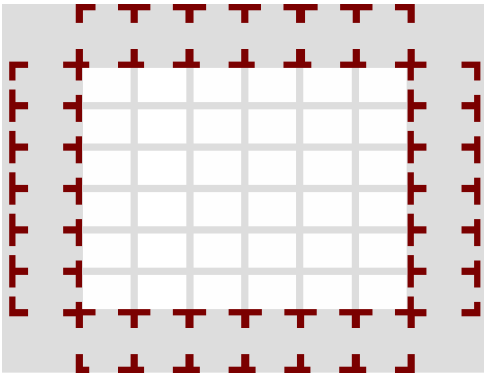


Figura 6.2.: Posibles posiciones para los puntos de entrada o salida

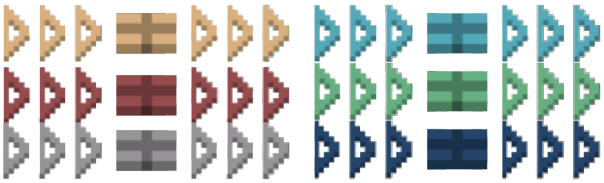


Figura 6.3.: Diferentes tipos de puntos de entrada, salida y cajas

Tanto los puntos de entrada como los puntos de salida cuentan con **3 cargas** que indican su estado. Si un punto de entrada quiere añadir una caja al mapa pero la casilla inmediata está bloqueada por otra caja, este la almacena como 1 carga de las 3 que puede mantener en su interior. De forma que, cuando se libere la casilla, suelte la caja. Si no se soluciona, llega a guardar 3 cargas y sigue bloqueada durante más tiempo, **se acaba la partida**.

Por el otro lado, un punto de salida puede coger hasta 3 cajas si se encuentra sin ninguna carga. Si tiene las 3 cargas, **deja de recoger cajas** y no hace nada hasta que terminen de procesarse. Este proceso tarda un tiempo, así que no es posible enviar todas las cajas del mismo tipo a un mismo punto de salida, lo que obliga a distribuir todo perfectamente a distintos puntos y a mantener las cajas **siempre en movimiento**.

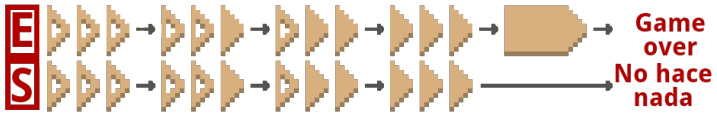


Figura 6.4.: Puntos de entrada (arriba) y de salida (abajo)

6.3.2. Cintas transportadoras

Para crear las cintas, el jugador tiene que hacer click izquierdo y arrastrar el ratón de una casilla a otra. Para borrarlas solo hay que situar el ratón encima de una casilla y pulsar el click derecho. En total hay 3 tipos de cintas:

- Cinta transportadora (conveyor): mueve una caja 1 casilla en la dirección a la que apunta.

- Cinta de lanzamiento rápido (bouncer): dispara una caja por encima de una casilla en la dirección a la que apunta hasta la siguiente.
- Cinta separadora (splitter): mueve una caja 1 casilla en una de las dos direcciones que apunta.

Para seleccionar las cintas, simplemente hay que pulsar la tecla indicada en panel de la Figura 6.5, el cual siempre se encuentra disponible en la pantalla.

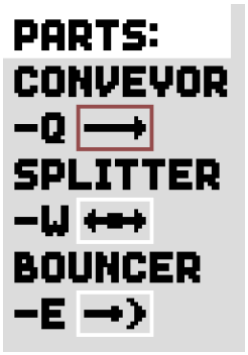


Figura 6.5.: Diferentes tipos de cinta y sus teclas de activación

6.3.3. Objetivo

El objetivo del juego es conseguir una fábrica completamente funcional y alcanzar la máxima puntuación. Una fábrica es funcional cuando llega a la última ronda sin problemas y aguanta sin saturarse durante un pequeño tiempo. La recompensa por pasarse el juego es ver un bucle con la animación de todas las cajas moviéndose por el mapa.

6.3.4. Pantallas

Compactory solo tiene la pantalla del juego que se puede ver en la Figura 6.6. En la parte de arriba se puede ver la cuenta atrás de cada ronda (explicado en el punto anterior) y la puntuación global, que es el número de cajas que se han entregado. En la parte de la izquierda está el indicador de cinta elegida junto con su tecla para seleccionarla. Por último, en la parte inferior se aprecia el indicador de pausa del juego, y algunos mensajes como el de Game Over o el mensaje de victoria también aparecen en esta zona.

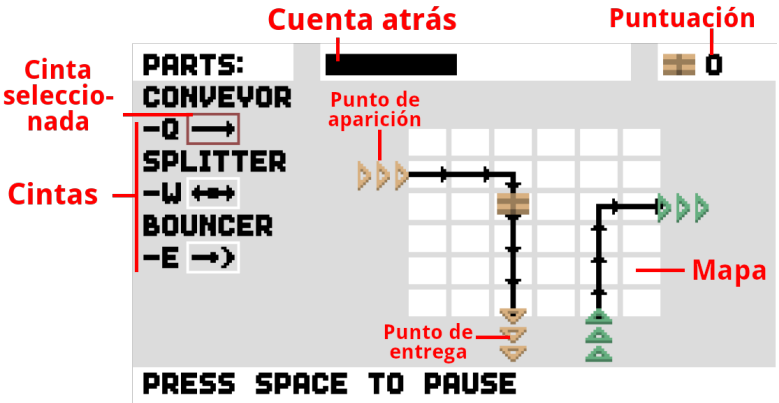


Figura 6.6.: Pantalla del juego de Compactory

6.4. Adaptación a Game Boy

Antes de decidir que este juego iba a ser adaptado a Game Boy, tuvo que analizarse si era posible. Se hizo un estudio de las mecánicas principales:

- El mapa es una rejilla 2D estática de rectángulos y se puede utilizar sin problema el mapa de fondo de Game Boy.
- Los puntos de entrada y de salida son sprites en una posición estática que se actualizan de vez en cuando.
- En total hay 4 elementos gráficos en el mapa: El mapa en sí mismo, las cintas, los puntos de entrada y salida y las cajas. Sin embargo, solo las cajas se mueven y dibujan por encima de todo y esto hace que no haya más de 2 capas de objetos. Esto significa que con la capa del mapa de fondo de la Game Boy y la capa de los sprites se pueden adaptar todos los elementos gráficos del mapa.
- Los elementos gráficos secundarios como el indicador de cinta, la pausa y la puntuación se pueden replicar en el mapa de fondo junto a la rejilla.

Una vez comprobado que todas las mecánicas principales eran adaptables a Game Boy, se comenzó con la adaptación. Fue en este momento cuando nació Packedtory.

Lo primero que se hizo fue un boceto de cómo se vería el mapa en la pantalla de la Game Boy con todos los elementos gráficos. Cada casilla del mapa tenía que tener el hueco suficiente para que cupiese en su interior el sprite de 1 caja y cualquier tipo de cinta. Se tuvo que eliminar 1 fila y 1 columna del mapa ya que un formato de 6x6 era imposible en la pantalla de 160x144 de una Game Boy.

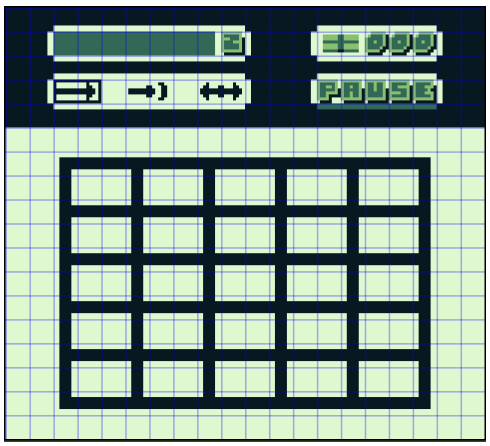


Figura 6.7.: Boceto del juego

Para las cajas, al haber solo 4 colores disponibles, no pueden cambiar de paleta de color para identificar el tipo de caja. Se tuvieron que diseñar 4 formas diferentes de cajas para poder diferenciarlas. Para el tamaño de las cajas solo había una medida posible que fuera lo suficientemente grande y poder tener un mapa de 5x5. Cada caja mide 16x8 píxeles y, para poder dibujarla por encima de todo, se necesitan 2 sprites. Ambos sprites usan el mismo tile para ahorrar espacio y el segundo sprite tiene activado el bit de espejar horizontalmente.

6.4.1. Puntos de entrada y salida

Los puntos de entrada y salida también se tuvieron que rediseñar. El espacio restante en el borde del mapa no daba mucha flexibilidad. Solo hay 1 fila tanto por arriba como por abajo y

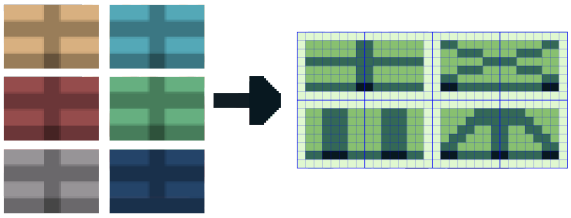


Figura 6.8.: Adaptación de las cajas a una Game Boy

2 columnas a los lados. Para poder representar el tipo de caja y su estado sin usar colores, la única opción posible utilizar 2 tiles. Un tile se encarga de representar el estado del punto con las cargas y el otro representa el tipo de caja. En la Figura 6.9 se pueden observar los puntos de entrada en los extremos del mapa. Los puntos de salida serían iguales que los de entrada pero con el tile de estado invertido.

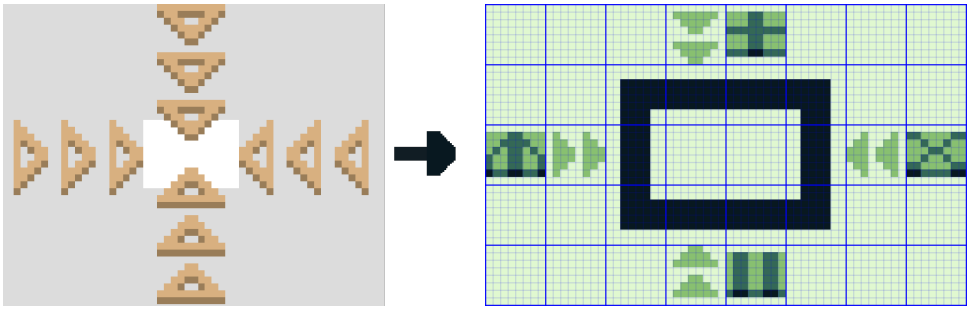


Figura 6.9.: Adaptación de los puntos de entrada a una Game Boy

Al utilizar solo 1 tile para el estado, se tuvo que disminuir la cantidad de cajas que pueden almacenar de 3 a 2 para que pudiera caber todo. En la Figura 6.10 se puede observar los 3 diferentes estados que tiene cada punto: vacío (0 cargas), medio lleno (1 carga), lleno (2 cargas). Además, en el caso del punto de entrada, se añadieron 2 frames intermedios para indicar que se está cargando y dar más información al jugador.

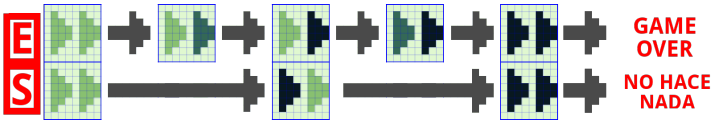


Figura 6.10.: Estados de los nuevos puntos de entrada (arriba) y salida (abajo)

Para los elementos secundarios se intentó aprovechar la ventana que tiene Game Boy para dibujar elementos por encima del mapa. Sin embargo, al ser un mapa estático y no haber scroll, no es necesario usar la ventana. Para modificar estos elementos se guardan las posiciones de la memoria del mapa y se modifican directamente a mano, obteniendo el mismo resultado.

Los elementos secundarios modifican los tiles del mapa de la siguiente manera:

- La barra del tiempo modifica los tiles de derecha a izquierda para simular que está disminuyendo. Cuando llega al límite, se redibuja entera la barra y se disminuye el número de ronda.
- La puntuación cambia según se van entregando cajas por los puntos de salida
- El indicador de cinta cambia los tiles del tipo de cinta seleccionada a otros tiles con un pequeño borde.
- El indicador de pausa cambia los tiles de la fila de abajo cada vez que se pulsa el start.

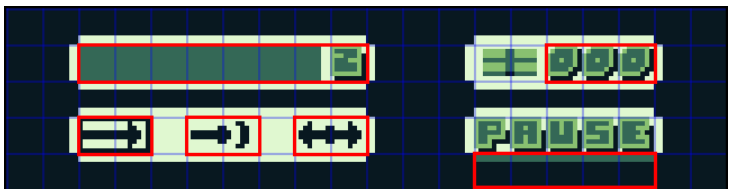


Figura 6.11.: Elementos secundarios y tiles que se editan a mano

6.4.2. Cintas

Una de las funcionalidades que se tuvo que cambiar completamente fue la de crear cintas en el mapa. En la versión de ordenador, al crear las cintas con el ratón, con calcular la colisión con el mapa era suficiente. En Game Boy, obviamente, no existe nada parecido y hay que adaptarse a los controles que ofrece. Para conseguir algo similar, surgió la idea de crear una especie de rectángulo que indica la posición donde interactuar con el mapa. El rectángulo o selector, indica dónde se va a crear una nueva cinta o se va a borrar una existente y lo controla el jugador mediante las flechas de dirección.

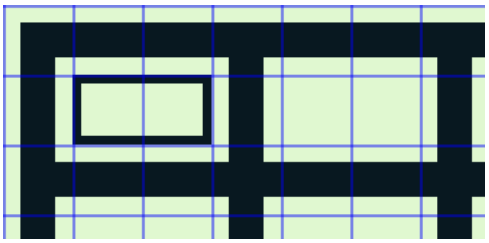


Figura 6.12.: Selector del del mapa en la primera casilla

Se mantuvieron los 3 tipos diferentes de cintas con los gráficos de Game Boy y con la misma funcionalidad:

Figura 6.13.: Comportamiento de la cinta transportadora

Figura 6.14.: Comportamiento de la cinta de lanzamiento rápido

Figura 6.15.: Comportamiento de la cinta separadora

Para seleccionar el tipo de cintas se tuvo que adaptar el panel lateral izquierdo a una barra más compacta y que indicase el tipo de cinta seleccionado. Por falta de botones, no se pudo asignar un botón a cada tipo de cinta así que se solucionó usando una única tecla que selecciona un tipo de cinta diferente cada vez que se pulsa. Cada vez que se pulsa la tecla de seleccionar tipo, el rectángulo que indica el tipo se mueve hacia la derecha y, si está en la última posición, vuelve al principio.

Para orientar las cintas, en la versión de ordenador sobra con arrastrar el ratón mientras se pulsa el botón izquierdo y la cinta apunta en la dirección a la que se mueve el ratón. En Game Boy, no es posible hacer esto y se tiene que buscar alguna forma de orientar las cintas. La opción

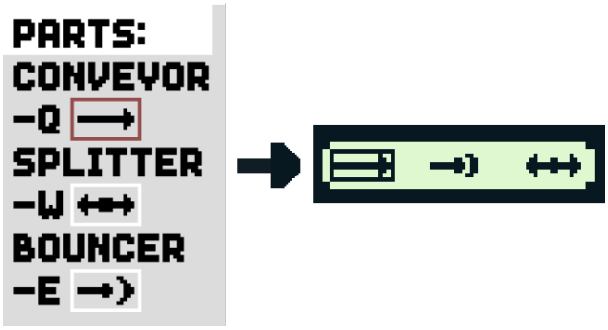


Figura 6.16.: Adaptación de la selección de cinta

por la que se optó es la siguiente: si en una casilla ya hay una cinta y se intenta crear otra cinta del mismo tipo, esta cinta rota 90º hacia la derecha.

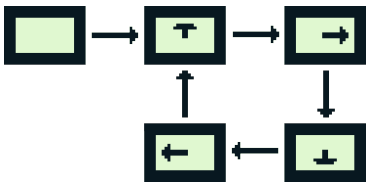


Figura 6.17.: Adaptación de la orientación de las cintas

6.4.3. Pantallas y menús

El juego original no contaba con menú principal ni pantalla de Game Over ya que con un texto en la parte inferior comunicaba todo. Según se diseñó el mapa, adaptar esto no era posible y se decidió hacer estas dos pantallas para dar información al jugador.

- Menú Principal: Contiene el botón de empezar partida y una pequeña animación de una caja moviéndose que introduce al jugador en el movimiento de las cintas.
- Game Over: Muestra una pequeña animación de la puntuación final y un botón de volver a jugar. Al pulsar cualquier tecla te lleva al menú principal.

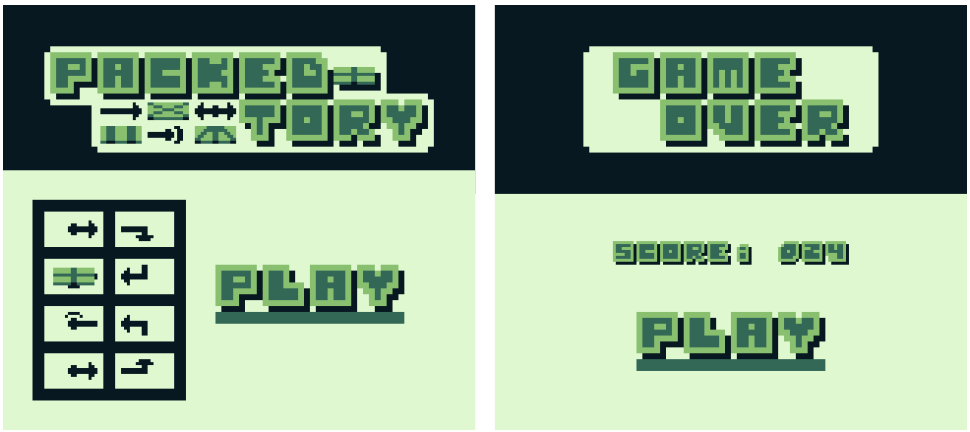


Figura 6.18.: Menú principal y Game Over

- Juego: Es donde sucede toda la acción. Contiene todos los elementos necesarios para una partida (señalados en la Figura 6.19). Cuando se completa un nivel, se carga el siguiente hasta un límite y, si se pierde, se va al menú de Game Over y se puede volver a empezar.

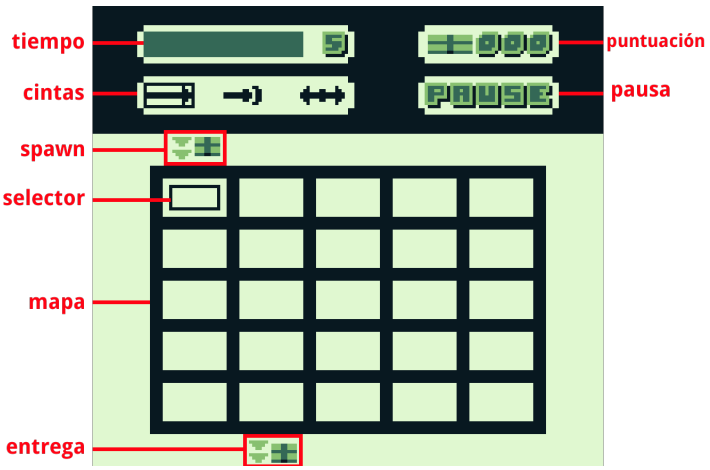


Figura 6.19.: Pantalla del juego

6.4.4. Controles

En la versión de ordenador, se utiliza el movimiento del ratón y los dos botones para crear cintas y 3 teclas más (q,w,e) para elegir la cinta. También pulsando la barra espaciadora se pausa el juego y pulsando la "z", se reinicia el juego. El problema ocurre cuando una Game Boy solo cuenta con una cruceta con 4 direcciones y 4 botones y la versión de ordenador utiliza un total de 7 botones + ratón.

Para ser capaces de jugar en una Game Boy, hay que rediseñar los controles. Al no contar con el movimiento de un ratón, la cruceta tiene que ser utilizada para controlar el selector y los botones A y B para crear una cinta y borrarla respectivamente.

Después de la creación y borrado de cintas, solo quedan 2 botones restantes y 3 acciones restantes. Para la selección de la cinta se agruparon todas las teclas en una sola que al pulsarla va alternando el tipo de cinta. Esta única tecla se asignó al botón select y la pausa, al igual que en casi todos los juegos, se le asignó al start. Una vez asignados todos los botones, la única tecla restante sería la "z" para poder reiniciar el mapa rápidamente, pero al no ser una función tan importante, se puede prescindir de ella.

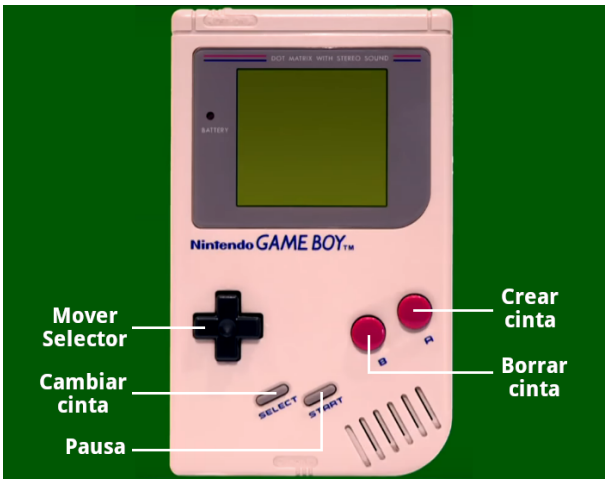


Figura 6.20.: Adaptación de los controles

7. Implementación

Antes de empezar a desarrollar un proyecto nuevo, si no estás familiarizado con la tecnología, una de las mejores cosas que se pueden hacer es un pequeño prototipo de aprendizaje. Este prototipo tiene que servir para aprender cómo hacer algunas mecánicas simples, como la de mover un sprite o hacer que salte, y para investigar cuales son las diferencias respecto a otras tecnologías.

7.1. Prototipo de aprendizaje

El prototipo consistía en un sprite de un caballero moviéndose por un mapa pequeño con unas pocas colisiones. Este prototipo sirvió para entender varias características de la Game Boy. Fue muy útil para aprender a dibujar tiles y sprites, entender cómo se almacenan en la memoria todos los tiles y los utilizan el fondo y los sprites.

En primer lugar, tras obtener los sprites de una página de recursos para videojuegos, se comenzó con el proceso de convertir los gráficos al formato propio de la Game Boy. Aquí es donde entra en acción el software específico para desarrollar gráficos para Game Boy: *Game Boy Tile Designer* para dibujar los tiles y *Game Boy Map Builder*[1] para montar los mapas.

Después de haber dibujado todos los tiles y el mapa, había que exportarlo todo a su propio archivo en hexadecimal que más adelante se incluirá dentro del juego para tener acceso a los datos que van directamente a su correspondiente memoria.

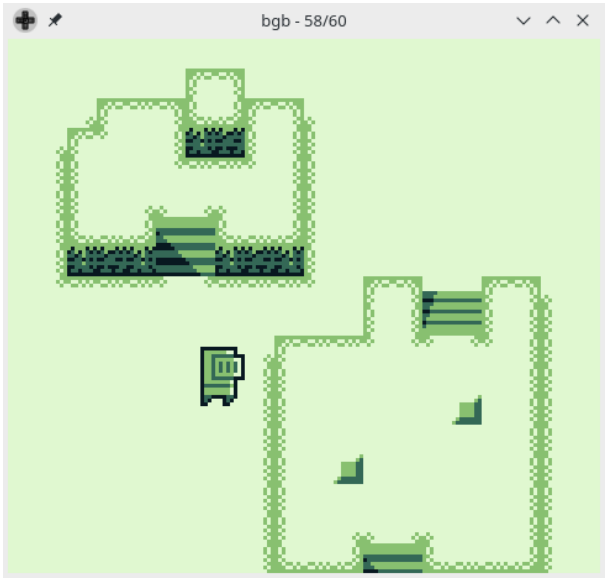


Figura 7.1.: Prototipo de aprendizaje

También se tuvieron que controlar todos los botones del joypad (las 4 direcciones y los 4 botones). Simplemente consistió en leer los valores del registro de entrada P1 (Player 1) situado en la posición $FF00. En los cuatro primeros bits (0-3) se encuentran los valores de los cuatro botones y direcciones pero hay que asignar antes el bit 5 para leer los botones o el bit 4 para leer las direcciones.

[1] Ver el apartado 5.1 para más información acerca de las herramientas de desarrollo

Tras controlar los botones, se hicieron las funciones que controlaban la posición del sprite e iban modificando su posición en base a los botones de dirección pulsados. El dibujado se hizo en un primer momento esperando al V-Blank manualmente, es decir, "congelando" el juego cuando se quería mover el sprite hasta que sea posible acceder a la VRAM. Sin embargo, esta no es la mejor manera de hacer el dibujado porque, en un juego grande, puede desencadenar en una gran perdida de tiempo que podría utilizarse para otros cálculos como las colisiones o la inteligencia artificial.

Al final, se optó por una mejor forma de realizar el dibujado: en vez de esperar a que se pueda acceder a la pantalla cada vez que se mueve el sprite, se crearon dos nuevos registros con la posición en X y en Y del sprite que iban aumentando dependiendo del botón pulsado. Además de independizar las variables de movimiento del dibujado del sprite, se utilizó la interrupción del V-Blank, que hace que se salte automáticamente al código de dibujar cuando se puede acceder a la OAM. Dentro de esta interrupción, se pueden copiar libremente los valores de las variables a la posición de los sprites en la OAM y moverlos.

7.2. Diseño del mapa

La idea de que el mapa fuera una rejilla estuvo clara desde el principio. A la hora de encajar el mapa en la pantalla de la Game Boy se tuvo que controlar que hubiera el suficiente espacio para todos los elementos del juego. Las casillas de la rejilla debían tener suficiente espacio para que cupieran sprites de 2 tiles y, además, tenían que estar alineadas correctamente con el mapa de fondo para poder tener 2 tiles vacíos para las cintas.

Se empezaron a hacer bocetos de todas las cintas y cajas en Aseprite, editor de sprites 2D, para ir calculando tamaños y ver si encajaban bien. Se dibujaron todos los tipos de cintas, en todas las direcciones con la intención de utilizar esos mismos gráficos en la Game Boy.

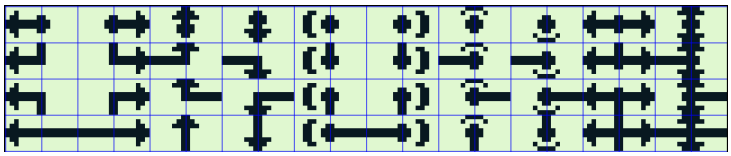


Figura 7.2.: Boceto de las cintas

Las 8 primeras columnas corresponden a la cinta transportadora, en las 4 direcciones y con las 4 conexiones entre celdas. Las columnas 9-16 son de la cinta de lanzamiento rápido y las 4 últimas son de la cinta separadora. Las horizontales tenían generalmente un aspecto un poco mejor que las verticales (ya que horizontalmente hay el doble de espacio disponible) y, dentro de las verticales, algunas flechas son de diferente tamaño debido a que el espacio son 8 píxeles de alto.

Sin embargo, no se tardó mucho en rediseñar toda la estructura y diseño de las cintas ya que era imposible guardar todas las posibles combinaciones en la memoria de la Game Boy por falta de espacio. Se hizo una versión simplificada sin conectores, con solo 4 direcciones por cinta (8*3=24 tiles de solo las cintas y otros 4 con los conectores por si se quería combinar los tiles más adelante).

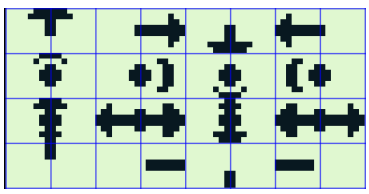


Figura 7.3.: Boceto modificado de las cintas

Por otra parte, el diseño de las cajas tuvo que realizarse teniendo en cuenta que iba a haber un selector por el mapa, por lo que tendría que ser capaz de verse bien. Para cumplir esto, los sprites de las cajas tienen que tener un pequeño margen de 1 píxel para poder ver el selector. Además, las cajas debían de indicar su tipo y tener un dibujo diferente ya que la Game Boy solo posee 3 colores para los sprites. Las 4 diferentes formas de cajas se pueden apreciar en la Figura 7.4.

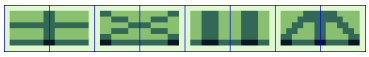


Figura 7.4.: Tipos de cajas

Hecho esto, se procedió a hacer un boceto del mapa. Debido a la temática del juego, era mejor que la ventana se quedase estática ya que, de lo contrario, podría desorientar al jugador. La pantalla tenía que contener una barra de tiempo, contador de puntuación, indicador de la cinta seleccionada e indicador de pausa, además de toda la rejilla del mapa, y todo debía caber en una ventana de 20x18 tiles y 160x144 píxeles. Entonces, se decidió mostrar la información de la partida en la parte superior de la ventana y el mapa completo en la parte inferior.

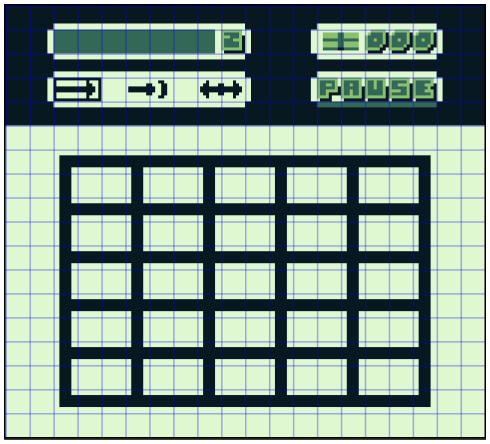


Figura 7.5.: Boceto del mapa

De esta forma, las celdas de la rejilla contienen 2 tiles en su interior y cabe cualquier caja y las cintas en su interior sin problema.

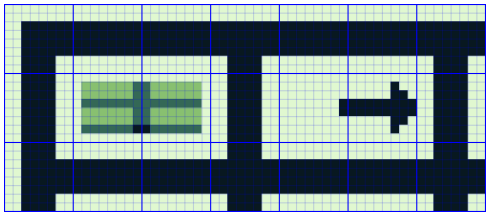


Figura 7.6.: Celda del mapa

7.3. Selector

Una de las primeras funcionalidades en ser implementada fue la del selector que sirve para crear las cintas en el mapa. Este selector es un rectángulo que sigue un movimiento fijado a la rejilla del mapa. Para que su tamaño se adapte al de una casilla, tiene que medir 16x8 y, al ser 8 píxeles el ancho máximo de un sprite, hacen falta 2 sprites para poder dibujarlo.

Para el movimiento, se crearon 2 bytes que contenían el índice de la rejilla en la que se encuentra situado. Por ejemplo, si está en la segunda fila, primera columna, los valores de los bytes serán Y=1, X=0. Cuando se pulsa el pad hacia abajo, se incrementa en 1 la posición en Y hasta un máximo de 4, y si se pulsa hacia arriba se decrementa hasta un mínimo de 0. Lo mismo en el eje horizontal con el byte de la X.

La primera posición para las coordenadas Y=0 y X=0 para los sprites del selector es y=48 y x=20. En la Figura 7.7 se pueden ver los valores en hexadecimal de las posiciones en Y en X para el primer sprite del selector. Para transformar el número de los bytes al valor que necesitan los sprites se tuvo que estudiar la relación.

Figura 7.7.: Posiciones en hexadecimal del sprite del selector (color) e índices de la rejilla (negro)

Para la Y, cuando el byte vale 0, la posición del sprite debería ser $48 (72), cuando vale 1, $58, cuando 2, $68... Al haber solo 1 tile de espacio entre fila y fila, tiene una conversión directa: hay que multiplicar por 16 los valores del byte Y y sumar todo a $48. En el ensamblador de GB no existe el concepto de multiplicar y se tiene que descomponer en sumas. Por ejemplo, para hacer algo tan simple como multiplicar por 2, se tendría que hacer `add a, a` o rotar todos los bits 1 vez hacia la izquierda. Afortunadamente, multiplicar por 16 en ensamblador es muy sencillo. Simplemente hay que rotar todos los bits 4 veces hacia la izquierda o, mejor aún, utilizar la instrucción: `swap a`, la cual intercambia los 4 bits bajos y los altos.

```
mover_abajo:
  ld a, [POSY]
  inc a
  ld [POSY], a
  swap a            ; a = $01->$10
  add $48           ; añadir posición Y inicial
  ld [_SPR0_Y], a
  ld [_SPR1_Y], a
  ret
```

Listado 7.1.: Función de mover el selector abajo

Para la X es un poco más complicado. Para 0 vale $20, para 1 vale $38, para 2, $50... No hay una relación tan directa. Hay 2 huecos entre cada inicio de celda y un incremento de $18 (24) en cada para pasar de una a otra. Se puede hacer un apaño multiplicando la posición en X (que va de 0 a 4) por $18, y sumar la posición inicial de $20.

Sin embargo, en ensamblador no existe forma de multiplicar por 24. De la misma manera que para hacer una multiplicación por 3 hay que descomponerla en una suma triple (sumar tres veces el mismo valor), para multiplicar un número por $18 hay que multiplicarlo por $10 y luego

sumarle su multiplicación por $08 o al revés. En el Listado 7.2 se puede ver el procedimiento.

```
mover_derecha:
  ld [POSX], a
  inc a                ; a++
  ld [POSX], a         ; actualizar registro
  swap a               ; a = $01->$10
  ld b, a              ; b = a
  rra                  ; a = $10->$08 (1/2)
  add b                ; a=a+b = a*$18 (a*24)
  add $20              ; añadir posición X inicial
  ld [_SPR0_X], a
  add $08              ; a+=8 (segundo sprite)
  ld [_SPR1_X], a
  ret
```

Listado 7.2.: Función de mover el selector a la derecha

7.4. Gestión de las cajas

7.4.1. Creación

Este es uno de los pilares principales del juego: gestionar la creación y borrado de las cajas. Una caja se compone de 2 sprites, por lo tanto, debido a las limitaciones del hardware de la Game Boy, solo se pueden dibujar hasta 40 sprites a la vez, lo que se traduce en 20 cajas a la vez. Sin embargo, el selector ya ocupa 2 de esos sprites, dejando disponibles un total de 19 cajas. Además, hubo que considerar que, también debido a la restricción del hardware de la Game Boy, no se pueden dibujar más de 10 sprites por línea. Sin embargo, al tener un mapa de 5x5 celdas, en el caso de que hubieran 5 cajas en una misma fila, serían exactamente 10 sprites y se podrían dibujar todas sin problemas.

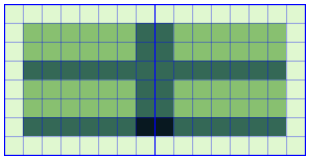


Figura 7.8.: Detalle de una caja

Al haber solo 19 cajas disponibles, hay que gestionar a la perfección la creación y el borrado de todas las cajas y sus sprites. Para poder representar la posición de todas las cajas se tuvo que crear un array con todas ellas. El objetivo de este array era agrupar todas las posiciones en un mismo sitio y hacer las comprobaciones necesarias para poder dibujar las cajas correctamente.

Como se quería reducir el número de registros usados en las comparaciones y el tamaño de los arrays, se tuvieron que guardar las posiciones utilizando un único byte. De esta forma, se usan 4 bits para la posición Y y los otros 4 para la X. Por ejemplo, la posición Y=3 y X=0 estaría representada por el valor $30 (se tiene que usar hexadecimal porque si no el compilador lo convertiría a hexadecimal, es decir de 30 a $1E). Una vez almacenados de esta forma, hay que realizar una serie de operaciones adicionales para obtener los valores de Y y X. En el listado 7.3 se puede ver la macro que se utilizó durante el desarrollo del juego con el objetivo de acelerar el proceso.

```
SPLIT_A_BC: MACRO
  ld b, a    ; b = YX
  and $0F    ; a = 0X
  ld c, a    ; C = 0X
  ld a, b    ; a = b
  and $F0    ; a = Y0
  swap  a    ; A = 0Y
  ld b, a    ; B = 0Y
  ENDM
```

Listado 7.3.: Código para separar los valores de 1 registro en 2

Se implementó una **pila** que guarda los índices de los sprites disponibles cuya finalidad era reutilizar los sprites que se iban desocupando de las cajas que se borran. Esta pila funciona de manera que cuando se necesita un nuevo sprite, se desapila el valor superior y ese es el índice del nuevo sprite que va a ser utilizado.

Para construir una pila en ensamblador son necesarios 2 elementos: los datos y el puntero al último elemento. Los datos se guardan en un array de 20 bytes, ya que nunca habrá más de 19 cajas. El puntero es simplemente un número entero almacenado en un solo byte a continuación de los datos y su función es obtener el último elemento en el array a partir de sumar su valor a la dirección de memoria del inicio de los datos. Por ejemplo, para una pila de 5 datos que comienza en la dirección $C000, el puntero vale 4 y se le suma a la dirección del principio, obteniendo de esta forma que el último elemento está en la posición $C004.

A la hora de hacer **pop** u obtener el último elemento, **se obtiene el último valor** de la forma anterior. Tras obtenerlo, se decrementa el valor del puntero en 1 y, para evitar posibles errores, el valor del último elemento anterior se borra. A la hora de hacer **push** o guardar un elemento en la última posición, se incrementa el índice, se obtiene el último elemento, se suma el valor del nuevo índice a la dirección de memoria de los datos y **se guarda el nuevo valor** en la última posición.

En resumen, cada vez que se quiere crear una caja, se hace pop en la pila de sprites y se empieza a utilizar ese sprite. Se actualiza la posición en el array de posiciones de las cajas y se actualizan la posición, el tile y los atributos del sprite que se hayan indicado (la posición de las cajas se calcula igual que la del selector).

7.4.2. Borrado

Por otro lado, a la hora de borrar una caja, se hace push del índice de dicho sprite para liberarlo y que pueda ser utilizado cuando se cree la siguiente caja. Después, en el array de posiciones de las cajas, se borra su posición y se almacena el valor $FF que indica que esa caja no está en uso. También se modifican los valores del sprite. Con mover los dos sprites a la posición Y=0,X=0 es suficiente para ocultarlos.

7.4.3. Dibujado

En un principio las cajas se dibujaban cuando se creaban, directamente actualizando los atributos del sprite. Una vez que se quisieran mover las cajas, aparte de actualizar los valores en el vector, se actualizaba la posición sumando o restando el número correspondiente (16 si se mueve verticalmente y 24 si se mueve horizontalmente) directamente en las coordenadas del sprite. Dibujar las cajas de esta manera implicaría tener un valor en el array de cajas y otro en la posición del sprite que están sincronizados en teoría pero que en la práctica se pueden desincronizar de manera muy fácil.

Para solucionar este problema, la mejor opción es crear una rutina de dibujado que transforme cada posición en el array a la correspondiente del sprite. Para hacer esta transformación, se tienen que hacer exactamente los mismos cálculos que se utilizaron para el selector, pero esta vez para cada caja.

Para relacionar las posiciones guardadas en el array con los sprites en memoria se hizo que cada posición del array fuera la misma que su sprite. Es decir, la posición 0 del array utiliza los sprites 2 y 3 (0 y 1 están ocupados por el selector), la caja de la posición 1 del array utilizará el sprite 4 y 5, y así sucesivamente. Para mantener la coherencia, como puede eliminarse cualquiera de las cajas en cualquier momento, las posiciones con el valor $FF representan cajas que no se encuentran en uso, y por lo tanto su correspondiente sprite se encuentra disponible para su uso.

7.5. Creación de cintas

Para crear todas las cintas en juego se comenzó por diseñar un código, una forma de representar cada cinta en un solo byte. Tras varios diseños la mejor opción fue utilizar los 4 bits bajos (0-3) para la dirección de la cinta y los 4 altos (4-7) para el tipo de cinta. De esta forma, el tipo de cinta también puede representar la cantidad de celdas que las cajas serán desplazadas. Las direcciones de la cinta son 1 = Arriba, 2=Derecha, 3=Abajo, 4=Izquierda.

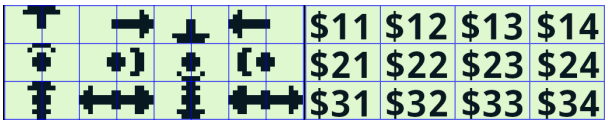


Figura 7.9.: Código de cada una de las cintas

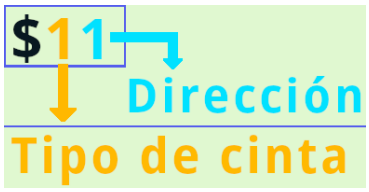


Figura 7.10.: Significado del código de las cintas

Lo mínimo que hay que conocer a la hora de crear una cinta es: la posición en la que se quiere crear la cinta y el tipo de cinta. La posición se obtiene directamente de los registros de posición del selector y para el tipo de cinta se necesita un registro adicional. Implementar la selección de cinta no fue una tarea muy complicada, simplemente es un registro que va incrementando cuando se pulsa un botón (el select) que vuelve a 0 cuando llega al límite (para 3 tipos de cinta: tipo 1, 2 y 3) se vuelva a 1.

Para almacenar todas las cintas del mapa es necesario un array de 25 bytes. El array se inicializa con valores $00 y se va rellenando de los códigos de las cintas que se van construyendo. Cuando se pulsa A, se calcula la dirección de memoria de la cinta de la casilla en la está el selector y se modifica el valor dependiendo de:

- Si la casilla está vacía, se añade el primer valor del tipo de cinta seleccionado ($11, apuntando hacia arriba, para la primera cinta)
- Si ya hay una cinta del mismo tipo que el seleccionado, se rota 90º el tile de la cinta. Cuando el tile pasa de apuntar hasta la izquierda, se reinicia el contador y pasa a apuntar hacia arriba de nuevo.

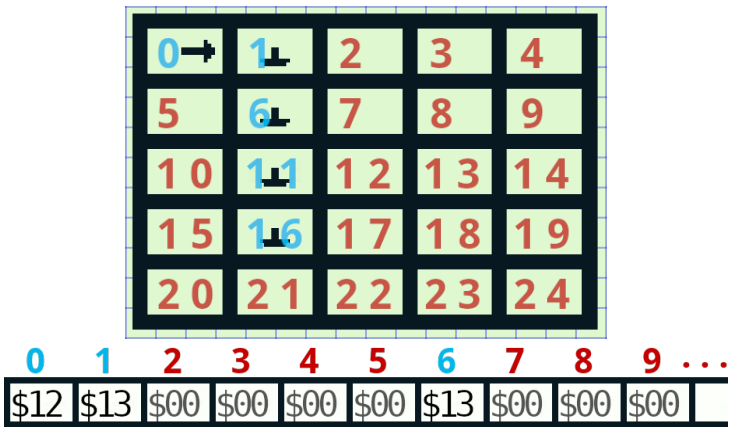


Figura 7.11.: Mapa con cintas en el mapa y sus códigos en el array

7.6. Movimiento de cajas

El movimiento de las cajas básicamente consiste en que cada una de las cintas que hay en el mapa desplace, si es posible, la caja que se sitúe encima de ellas. Esta acción se realizará una vez cada segundo para que la persona jugando tenga el suficiente tiempo de reacción y no se sature el mapa de cajas.

Para conseguir el movimiento hay 2 arrays de vital importancia: el que contiene las posiciones de las cajas y el que contiene los códigos de las cintas. Para cada caja, se obtiene su posición en el mapa del array de las cajas. Esta posición es necesaria para obtener el código de su cinta del array de las cintas y poder mover la caja correctamente.

Para obtener el código de una cinta en una determinada posición solo hay que hacer un pequeño cálculo: multiplicar la fila por 5 y sumar la columna. Por ejemplo, para la posición \$12 sale $1 * 5 + 2 = 7$, lo que significa que el código correspondiente está en el séptimo elemento del array de las cintas.

Una vez se ha obtenido el código, se comprueba la dirección a la que se va a mover y si se puede mover. En la Figura 7.9 se pueden ver todos los códigos visualmente, pero esencialmente el segundo dígito es el que indica la dirección y tiene 4 posibles valores: 1 = Arriba, 2=Derecha, 3=Abajo, 4=Izquierda.

Cuando ya se sabe la dirección, hay que comprobar que esté lo suficientemente alejado del límite. Esta comprobación es muy simple, al saber ya la dirección, se suma o se resta 1 en Y o en X y se obtiene la nueva posición de la caja. Si, en el eje que se mueve, el nuevo valor está entre 0 y 4, está dentro del mapa y se podría realizar el movimiento. Si no, se movería únicamente hasta el borde. Una vez se tiene la posición final, lo único que falta por comprobar es que esa casilla esté vacía, es decir, colisiones.

En la figura 7.12 se puede ver un esquema con todos los pasos para el movimiento. En el mapa hay 2 cajas junto a unas cintas. La primera caja en el array, la que está en la posición \$02, no se encuentra encima de ninguna caja. Se obtiene \$00 tras calcular la posición de su cinta en el array y no se sigue con el bucle.

La segunda caja, al contrario, si que se encuentra situada encima de una cinta. Primero, se extrae su posición del array \$11. Tras hacer el cálculo $1 * 5 + 1 = 6$ y acceder al valor del sexto elemento del array de cintas, se obtiene el código \$13. Este código, al separar los dígitos, el primero nos dice que se va a mover 1 casilla, y el segundo, nos dice que se mueve hacia abajo. Para mover una caja hacia abajo hay que sumar \$10 a su posición: \$11 + \$10 = \$21. Al hacer esta suma, a falta de comprobar las colisiones, el resultado está dentro de los límites del mapa y se puede mover.

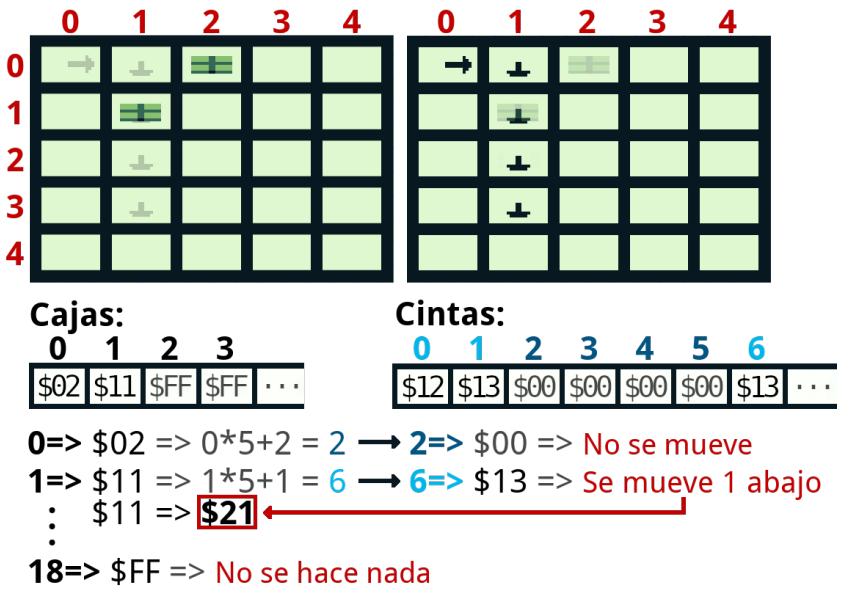



Figura 7.12.: Esquema con los pasos del movimiento de las cajas

7.6.1. Colisiones

Una vez el movimiento está hecho, aparece un gran problema, las cajas pueden superponerse en la misma casilla. Para solucionar esto hay que realizar un sistema de colisiones que compruebe, antes de que se mueva cada una de las cajas, si es posible moverse a esa casilla porque está libre.

Hay muchas formas de comprobar las colisiones en el mapa pero las que se tuvieron en cuenta son las siguientes. Una de las que se estuvo a punto de llevar a cabo es comprobar, una a una, todas las cajas que hay en el mapa con la nueva posición, de forma que se tiene que recorrer todo el array por cada movimiento de una caja. Los puntos a favor de hacerlo de esta forma es no tener que crear ningún otro array y, por lo tanto, se evitan problemas de coherencia y duplicado. Por otro lado, los puntos en contra son que tiene un coste muy alto si hay muchas cajas moviéndose a la vez y se tiene que consultar muy a menudo.

Otra manera de hacer colisiones es creando un array que contenga la información de cada una de las cajas que hay en las celdas. Así, están representadas todas las posiciones que están ocupadas en el mapa y se puede acceder directamente a una posición del mapa y comprobar si está libre. Los inconvenientes de este array es que hay que actualizarlo constantemente, cada vez que se crea, borra y mueve una caja. Los beneficios son que se puede obtener la caja de una posición directamente y tiene un coste menor.

Tras realizar varias comparaciones y estudiar las ventajas y desventajas, se optó por crear el array con la información de todas las cajas. Aunque tenga una dificultad mayor, tiene más opciones para almacenar datos y se pueden consultar mucho más rápida y cómodamente.

7.7. Puntos de entrada y salida

Además de poder crear cintas en el mapa y gestionar y mover todas las cajas, hace falta un elemento que cierre el círculo. Los puntos de entrada y salida son los que se encargan de crear y eliminar cajas cada cierto tiempo, forzando al jugador a mantenerse alerta.

Aunque ya estén implementadas la creación y el borrado de las cajas, estos puntos tienen más trabajo de lo que parece. Para gestionar los puntos correctamente, hace falta un array que almacene todas las características de los puntos. Un punto tiene que contener los siguientes elementos:

- Saber con que casilla del mapa interactúa, es decir, la casilla donde crea las cajas o donde las recoge.
- Debe ser de un tipo: de entrada o de salida.
- Tiene que tener sólo 1 de los 4 tipos de cajas.
- Tiene que controlar su propio estado, lo cual influye en el juego y en su apariencia.

Los puntos se sitúan junto a las casillas del borde, en el exterior del mapa. Al ser el mapa de 5 casillas de largo y haber 4 bordes, hay en total 20 posiciones posibles para los puntos. Para poder representar todos sus datos hay que analizar qué hay que representar y cuanto ocupa.

Para establecer la posición de la casilla de un punto, hacen falta 2 números que pueden ir de 0 hasta 5. Se pueden guardar en solamente **1 byte**, utilizando los 4 primeros bits para la posición en Y y los 4 últimos para la posición en X.

Para representar el tipo entre 2 opciones: entrada o salida, con **1 bit** es suficiente, ya que puede tomar los valores de 0 y 1. En cuanto a las cajas, los 4 tipos disponibles se pueden representar perfectamente con un número de 0 a 3, es decir, con **2 bits**.

Por último, el número que representa el estado. En la Figura 7.14 se puede ver que los puntos de entrada cuando están bloqueados tienen 11 estados diferentes: del 0 al 4 son los estados normales, y del 5 al 10 es una cuenta atrás para el fin del juego. Al haber 11 estados hacen falta **4 bits** como mínimo y, si se junta el tipo de punto, el tipo de caja y el estado, en total hay 7 bits que se pueden agrupar en un mismo byte.

En la Figura 7.13 está el formato que tiene cada elemento del array y, en especial, el primer byte. El primer byte corresponde al tipo, número de caja y estado, y el segundo byte corresponde a la posición del mapa que se va a comprobar en cada actualización. Para el primer punto, sus características son: tipo 1, tipo de caja 2 y estado 0. Esto significa que este punto es un punto de salida que acepta el tercer tipo de cajas y que, al estar en estado 0, puede recoger 2 cajas. El segundo punto, es un punto de entrada de cajas de tipo 2 que está bloqueado y no puede añadir cajas al mapa.

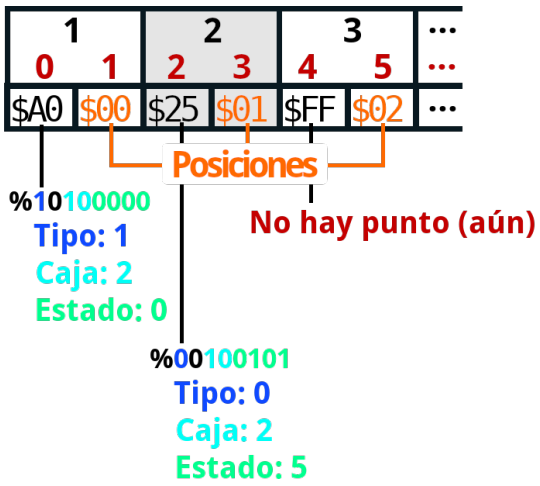


Figura 7.13.: Esquema del array de los puntos de entrada y salida

Los puntos se actualizan constantemente para que vayan creando y recogiendo cajas y haya movimiento en el juego. Las cajas se mueven cada segundo y los puntos se actualizan cada 3, de forma que hay tiempo suficiente para que se alejen las cajas de su punto de entrada y se creen nuevas antes de que se entreguen las primeras.

La actualización de los puntos de entrada y salida es muy simple. Es un bucle que comprueba las casillas de todos los puntos para ver si están ocupadas por alguna caja. Depende del tipo

del punto se hace una acción u otra y se actualiza su estado. Por último, se dibujan todos los puntos representando su nuevo estado.

Para que los puntos de entrada puedan crear una caja, se comprueba que **no haya ninguna caja** en su casilla correspondiente. Si está bloqueada la entrada, se incrementa su estado, acumulando cajas en su interior. Del 0 al 4 representa visualmente las cajas que faltan por añadir. Cuando el estado es 2, el punto tiene 1 caja pendiente y cuando es 4, 2. A partir del 4, no se siguen acumulando cajas, sino que se entra en estado de alerta y cambia su tile a un signo de exclamación. Además de cambiar el tile, se hace una pequeña animación de dos frames para llamar la atención del jugador y que lo solucione lo antes posible. Si se consigue liberar la casilla del punto de entrada, el punto suministra 1 caja y pasa a tener un 2 de estado, ya que todavía tiene otra caja pendiente.

Por otro lado, para los puntos de salida, se comprueba que **haya una caja** en su casilla y que tanto el punto de salida como la caja sean del mismo tipo. Si esto ocurre, se toma la caja borrándola del mapa. Inmediatamente se incrementan la puntuación del juego y el estado del punto de salida en 1. Si el estado del punto alcanza el máximo de 2, no podrá seguir recogiendo cajas aunque estén en su casilla y sean del mismo tipo, sino que no podrá hacer nada durante un tiempo. Cuando el punto de salida toma una caja comienza una cuenta de 6 segundos que simboliza el tiempo que se tarda en procesar la caja, y desaparece del almacén del punto. Esto hace que gane de nuevo un hueco que vuelve a estar disponible para recoger otra caja. Para ello, el estado del punto de salida se reduce en 1.

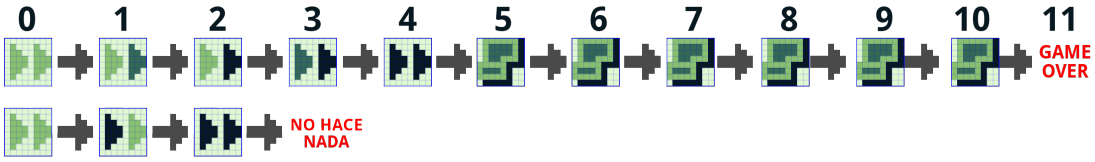


Figura 7.14.: Índices del estado de los puntos entrada (arriba) y salida (abajo)

Para dibujar todos los puntos en su sitio correspondiente, cada uno se dibuja en una posición diferente en base a su orden. En la Figura 7.15 se ven todas las posiciones de los puntos dependiendo del orden que tengan en el mapa. Por ejemplo, los 5 primeros elementos del array (10 primeros bytes) representan los 5 puntos en el límite superior del mapa.

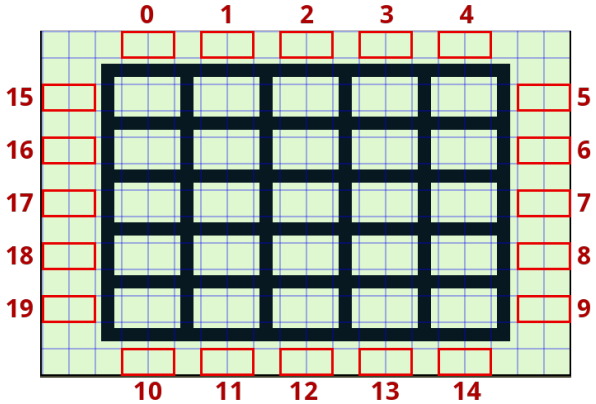


Figura 7.15.: Posiciones de los puntos en orden del array

Para ahorrar tiempo de ejecución y poder dibujar todos los tiles en el mismo V-Blank, el dibujado se divide en 2 pasos. En primer lugar, se recorren los estados de todos los puntos y se guardan en un array (`SPAWNERS_TILES`) todos los tiles que deben tener esos estados. Después, simplemente se copian los tiles desde el array a la posición en memoria correspondiente de cada

punto. En el Listado 7.4 se puede ver una pequeña parte de la función utilizada para copiar los puntos. Se carga en `HL` el inicio del array para utilizarlo como puntero, se copia desde el array a las direcciones fijas y se incrementa el puntero manualmente después de copiarlo.

```
write_all_tiles_mem:
  call wait_VBlank
  ld hl, SPAWNERS_TILES
  ; dibujar las 5 cajas superiores
  ld a, [hl+]
  ld [$98A0+03], a
  ld a, [hl+]
  ld [$98A0+06], a
  ld a, [hl+]
  ld [$98A0+09], a
  ld a, [hl+]
  ld [$98A0+12], a
  ld a, [hl+]
  ld [$98A0+15], a
```

Listado 7.4.: Código para copiar los tiles al mapa de fondo

Para obtener el tile correspondiente de cada punto, hay que utilizar los estados y hacer unas pequeñas operaciones. Si el punto es de entrega, se parte del estado actual y se suma un múltiplo de 8 para conseguir el tile correcto: 0 para los sprites del borde de arriba, 8 para los de la derecha, 16 para los de abajo y 24 para los de la izquierda. Si el punto es de salida, hay que sumarle además un desplazamiento de 5 para poder obtener el tile del punto.

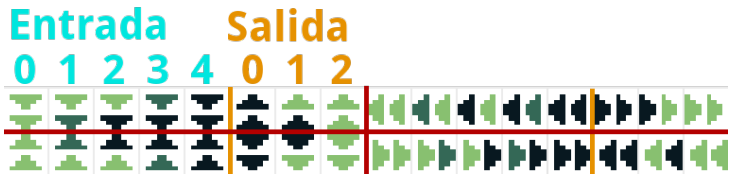


Figura 7.16.: Índices de los tiles de los puntos

7.8. Puntuación

La puntuación es el objetivo secundario del juego ya que para conseguir procesar el mayor número de cajas es necesario centrarse en construir una fábrica funcional.

Para representar la puntuación, se diseñaron 3 tiles en la parte superior derecha del mapa, como se puede apreciar en la Figura 6.11. Para poder representar 3 números diferentes de 0 a 9, está claro que hace falta un tamaño mayor de 1 byte, y si se quiere incrementar un registro constantemente y tener 2 números diferentes en un mismo registro se complica un poco la cosa. Al estar los registros en hexadecimal, si se incrementa $09 en 1, se obtiene $0A.

Sin embargo, existe una forma muy útil de representar los números en ensamblador llamada Binary Coded Decimal (BCD). Esta permite que los números binarios en lugar de guardarse tal cual, se guarden en un byte de manera que cada 4 bits representan un numero del 0 al 9, y no del 0 al 15 como pasa en base 2. Existe una instrucción encargada de convertir un número a BCD: `daa`. Se utiliza principalmente justo después de incrementar un número para convertirlo a BCD y evitar números hexadecimales. En el Listado 7.5 se puede ver el código utilizado para la función.

```
GAME_SCORE_INCREMENT:
  ld a, [GAME_SCORE+1]
  inc a
  daa      ; convertir a BCD
  cp $99               ; if a==99
  jr z, .add_100       ; se pone a 0 y se suma 100
    ld [GAME_SCORE+1], a
    ret
.add_100
  ld a, [GAME_SCORE]
  inc a
  ld [GAME_SCORE], a
  ld a, 0
  ld [GAME_SCORE+1], a
  ret
```

Listado 7.5.: Incremento de la puntuación

Gracias a que en la memoria de tiles están guardados los números en orden, se pueden cargar directamente números en el mapa y que se dibujen correctamente. Una vez los registros tienen los números en decimal, solo hay que obtener cada una de las cifras y copiarlas a sus posiciones correspondientes en el mapa.

```
drawScore:
  ld a, [GAME_SCORE]
  ld [_SCORE_POS +0], a    ; centenas
  ld a, [GAME_SCORE+1]
  and $F0
  swap a
  ld [_SCORE_POS +1], a    ; decenas
  ld a, [GAME_SCORE+1]
  and $0F
  ld [_SCORE_POS +2], a    ; unidades
  ret
```

Listado 7.6.: Dibujado de la puntuación

7.9. Rondas de juego

Una vez creados los puntos de entrada y de salida, falta un mecanismo que se encargue de añadirlos al mapa progresivamente para el jugador tenga que rediseñar la distribución de las cintas y no deje de pensar. Una ronda es básicamente la aparición de de una nueva pareja de 1 punto de entrada y 1 de salida.

La barra del tiempo de la parte superior indica el tiempo restante para la siguiente ronda. Este es un elemento muy importante para avisar al jugador del tiempo restante para la siguiente ronda. Tiene un tamaño de 7 tiles y cada uno de ellos tiene 5 frames diferentes para obtener una animación más suave.

En la Figura 7.17 puede verse un esquema en el que aparecen todos los elementos que influyen en la barra del tiempo. En la parte superior se encuentran los 7 tiles que van a indicar el tiempo

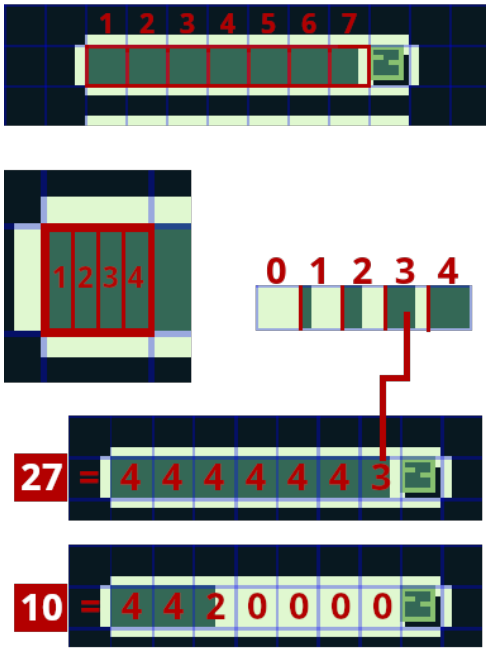


Figura 7.17.: Esquema de la barra del tiempo

restante para la siguiente ronda y justo debajo están las 5 diferentes opciones que puede contener cada tile.

Cada tile tiene tiene un valor entre 0 y 4 y al sumarlos todos se obtiene el tiempo restante para la siguiente ronda. Los números del 0 al 4 dibujan el tile al 0 %, 25 %, 50 % y 75 % y 100 % respectivamente.

Cada ronda comienza con todos los tiles al máximo y por tanto, 7 tiles por 4 da un tiempo de 28 segundos. Cada segundo el contador disminuye en 1 y se decrementa el valor del tile más a la derecha distinto de 0. La ronda se acaba cuando todos los tiles de la barra están vacíos y el contador llega a 0. A continuación, se vuelven a llenar todos los tiles, comienza la siguiente ronda y se decrementa el número que hay al lado de la barra del tiempo que indica el número de rondas restantes.

Cuando comienza una nueva ronda, se obtiene una nueva pareja de puntos de entrada y salida y se añade al mapa. Estos puntos se encuentran almacenados en un array que contiene los elementos suficientes para todas las rondas. Uno de los puntos de pareja puede ser representado utilizando simplemente 2 bytes: uno para la posición en la que se encuentra el punto (los que hay en la Figura 7.15) y otro byte para toda información del punto (Figura 7.13). De esta forma, solo hay que leer los 4 bytes siguientes de la respectiva ronda y añadir los dos puntos a su posición correspondiente.

Conclusiones

Tras varios meses de esfuerzo he llegado al final de este trabajo de fin de grado. Durante todo este tiempo he disfrutado y aprendido mucho sobre el mundo de los videojuegos, algo que siempre me ha apasionado desde pequeño.

He buscado información y documentación sobre el hardware de la consola, las características que tiene, cómo funciona, la historia de los videojuegos y las videoconsolas, influencia tecnológica y cultural en la época y en los años posteriores y he consultado numerosos manuales de hardware de Game Boy y de ensamblador. Me alegro de haber adquirido todos estos conocimientos e información, ya que sé que me acompañarán durante mucho tiempo y me serán de utilidad en mi vida profesional.

La satisfacción y el deseo de unir todas las piezas y que todo encaje y funcione como debería han hecho posible que este proyecto haya llegado a su etapa final. Además, ver que una consola que surgió cuando yo todavía no había nacido tiene tanto potencial aún hoy en día me ha abierto los ojos. Me ha hecho darme cuenta de la enorme cantidad de trabajo que tuvieron que realizar los primeros programadores y la ingeniería que requiere hasta el más simple de los videojuegos.

Si tuviera que empezar de nuevo este trabajo, hay una serie de cosas que habría priorizado sobre otras, como las animaciones de los objetos. Habría comenzado creando todas las animaciones deseadas antes de empezar a programar funcionalidades adicionales, y empezar a construirlas desde ahí. Ya que, hacia el final del proyecto, tuve la intención meter algunas animaciones pero al tener ya diseñado todo el juego era demasiado complejo integrarlas, hasta el punto que no merecía la pena.

Por último, he de nombrar unas posibles mejoras que se podrían haber realizado también al juego tal y como está en su estado actual. Me hubiera gustado añadirle música y efectos de sonido creados por mí, así como un mayor número de niveles que escalen en dificultad. Sin embargo, no pude llevar estas ideas a cabo debido a que al comienzo de este proyecto no tuve en cuenta que podría tener un tamaño tan inmenso y me tomaría tanto tiempo darle forma al propio juego y a la memoria descriptiva.

Para dar este trabajo por finalizado, a continuación se muestran unas imágenes del juego funcionando correctamente en una consola Game Boy Color.

Figura 7.18.: Packedtory funcionando en una Game Boy Color

Lista de Acrónimos

TFG: Trabajo Final de Grado
TFM: Trabajo Final de Master
GB: Game Boy.
GBC: Game Boy Color.
GBA: Game Boy Advance.
CPU: Central Processing Unit.
PPU: Pixel Processing Unit.
ROM: Read-Only Memory.
RAM: Random Access Memory.
VRAM: Video RAM.
HRAM: High RAM.
I/O: In/Out.
OAM: Object Attribute Map.
HUD: Head-Up Display.
LCD: Liquid-Crystal Display
LCDC: LCD Control
CRT: Cathode-Ray Tube
NES: Nintendo Entertainment System
SNES: Super Nintendo Entertainment System
RGBDS: Rednex Game Boy Development System
BCD: Binary Coded Decimal

Bibliografía

[1] *Game Boy™CPU Manual (Inglés).* Comandos de lenguaje ensamblador, tiempos de ejecución y opcodes, y todo lo que querías saber de Game Boy pero tenías miedo de preguntar. Disponible en `http://marc.rawer.de/Gameboy/Docs/GBCPUman.pdf`.

[2] Nate Largo. Compactory. `https://natelargo.itch.io/compactory`. Página web del videojuego adaptado.

[3] David Pello. Tutorial de desarrollo para la gameboy. `http://wiki.ladecadence.net/doku.php?id=tutorial_de_ensamblador`. Web muy útil con información del hardware de la Gameboy y varios ejemplos.

[4] Documentación de rednex game boy development system (inglés). `https://rednex.github.io/rgbds/`. Página web con toda la documentación del ensamblador.

[5] Game boy developers (inglés). `http://www.devrs.com/gb/`. El antiguo foro de desarrolladores más famoso de Gameboy, con muchos ejemplos, tutoriales, código y herramientas.

[6] Basic hello-world example for gameboy using rgbds (inglés). `https://github.com/nezticle/rgbds-template/blob/master/src/hello-world.asm`. Hola mundo para entender la sintaxis y empezar con el ensamblador RGBDS.

[7] Gameboy opcode summary (inglés). `http://www.devrs.com/gb/files/opcodes.html`. Resumen de todos los comandos del ensamblador para Game Boy y su funcionamiento utilizando elementos gráficos como flechas.

[8] Awesome game boy development list (inglés). `https://gbdev.github.io/list.html`. Una lista enfocada en el desarrollo en Game Boy con muchos recursos, herramientas, documentos, proyectos relacionados y homebrews.

[9] The ultimate game boy talk (inglés). `https://youtu.be/HyzD8pNlpwI`. Una charla que intenta comunicar todo lo posible de la Gameboy, incluyendo hardware, sus peculiaridades así como los trucos que los juegos han utilizado, reviviendo el espíritu de aquellos tiempos cuando los programadores contaban los ciclos del reloj y las limitaciones del hardware eran todo un reto.

[10] Retro tech: Game boy (inglés). `https://youtu.be/Oy8zSYKkczI`. Un corto documental acerca de la historia de la Gameboy donde se explora lo que llegó a ser y su impacto en la sociedad.

[11] Nintendo game boy - did you know gaming? (inglés). `https://youtu.be/PFjie61oyb8`. Un vídeo que comenta algunos datos interesantes, secretos y *Easter Eggs* de la Gameboy.

[12] Game boy accessories - angry video game nerd (inglés). `https://youtu.be/EEzJH90h3aA`. Un vídeo que habla un poco acerca de la historia y comenta múltiples accesorios que salieron para la Gameboy.

[13] Gameboy en wikipedia (inglés). `https://en.wikipedia.org/wiki/Game_Boy`. Página de wikipedia de la Gameboy. También está en español pero hay más información en la versión inglesa.

[14] Gunpei yokoi en wikipedia. `https://en.wikipedia.org/wiki/Gunpei_Yokoi`. Página de wikipedia del diseñador de la Gameboy. También está en español pero hay más información en la versión inglesa.

[15] 1980s in video gaming (inglés). `https://en.wikipedia.org/wiki/1980s_in_video_gaming`. Página de wikipedia del mundo de los videojuegos en los años 80.

A. Anexo I: Hardware de la Game Boy

A.1. Especificaciones técnicas

Componentes	Características
CPU	8-bit Sharp LR35902 (Similar al Z80) a 4.19 MHz
RAM	8KB
VRAM	8KB
Display	2.6“ LCD 160×144, 4 Tonos
Sonido	Estéreo, 4 Canales
Voltaje	6V, 0.7A
Dimensiones	90 mm x 148 x 32 mm

Tabla A.1.: Especificaciones técnicas de la Game Boy

A.2. Mapa de memoria de la Game Boy

La Game Boy es una consola de 8bits pero con un espacio de direcciones de 16bits. Esto significa que puede acceder a posiciones de hasta 2$\hat{1}$6 bytes (65536 bytes, 64K), es decir, la dirección máxima es $FFFF. La primera mitad del espacio ($0000-$7FFF) es el espacio que viene definido por el cartucho del videojuego. A continuación se encuentra la Memoria de Vídeo (VRAM) y RAM externa, la cual también puede venir opcionalmente del cartucho. Después se encuentra la RAM interna de la Game Boy y, por último, en una pequeña parte cerca del final, se encuentra la Memoria de Sprites, los puertos de Entrada/Salida y RAM Alta interna.

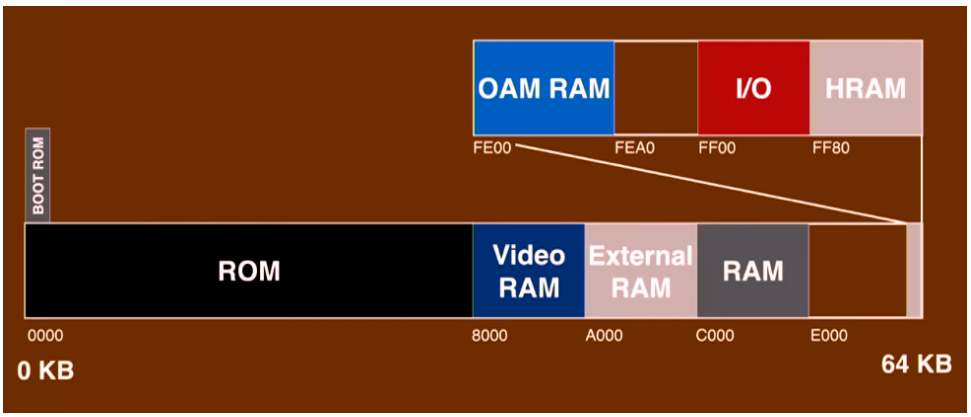


Figura A.1.: Mapa de la memoria

Que la ROM tenga un espacio de 32KB no significa que los juegos tengan un máximo de 32KB. Algunos juegos como el tetris solo necesitan 32KB pero hay juegos que pueden llegar a ocupar hasta 2MB. Esto se consigue utilizando un chip que tienen determinados cartuchos,

conocido como mapper. La ROM del cartucho se divide en dos partes: la primera mitad ($0000-$3FFF) siempre está siempre fija y contiene la lógica principal del juego y la segunda mitad que es intercambiable por otros bancos que haya disponibles gracias al mapper. De esa forma, se puede conseguir más espacio para almacenar niveles, gráficos y otros datos que solo sean necesarios durante ciertas partes del juego.

Con la RAM externa pasa lo mismo, se puede intercambiar por otros bancos y obtener más espacio si fuera necesario. Esta puede ser una buena forma de guardar todos los datos de las partidas de los juegos.

A.3. Mapa de memoria de una ROM

Las ROMs de GameBoy tienen que guardar cierta estructura para que la GameBoy las acepte, especialmente en el tema de la cabecera. A continuación las partes de una ROM y su función:

Dirección	Descripción
$0000-$00FF	Vectores de interrupción
$0100-$0103	Punto de entrada a la ejecución del programa
$0104-$014F	Cabecera del cartucho
$0150-$3FFF	Código del juego. Este es el banco 0 de 16K, y es fijo
$3FFF-$7FFF	Segundo banco de 16K intercambiable

Tabla A.2.: Estructura de una ROM

Los vectores de interrupción son unas posiciones reservadas a las que se salta cuando hay una interrupción de cualquier tipo (se explican en más detalle en el Anexo A.6).

La posición $0100 es el punto de entrada a la ejecución del programa. Normalmente se suele poner un nop, seguido de un salto al punto de inicio del juego (1 byte del nop + 3 del jump) para que se pueda definir la cabecera justo a continuación.

ROM Header		
0100 - 0103	Entry Point	nop, jp $0150
0104 - 0133	Nintendo Logo	$CE, $ED, $66, $66, $CC, ...
0134 - 0143	Title	“SUPER MARIOLAND”
013F - 0142	Manufacturer Code	$00
0143	CGB Flag	$00
0144 - 0145	New Licensee Code	$00, $00
0146	SGB Flag	$00
0147	Cartridge Type	$01
0148	ROM Size	$01
0149	RAM Size	$00
014A	Destination Code	$00
014B	Old Licensee Code	$01
014C	Mask ROM Version number	$00
014D	Header Checksum	$9E
014E - 014F	Global Checksum	$41, $6B

Figura A.2.: Cabecera de la ROM

La cabecera contiene el logo de nintendo, ($0104-$0133) que es comparado con el existente en la ROM justo al arranque de la consola, si no coinciden, se detiene la ejecución. Además, contiene datos con información del juego y tiene una estructura predefinida:

A.4. Input

Para controlar los botones pulsados en la Game Boy es muy simple: hay un solo registro que contiene toda la información de los 8 botones. En vez de utilizar los 8 bits para representar cada uno de los botones pulsados, utiliza 6 bits. Los botones se encuentran divididos en 2 columnas, el pad de direcciones y los otros 4 botones. Hay que activar el bit 4 para poder leer los 4 botones (Start, Select, B y A) y el bit 5 para las 4 direcciones.

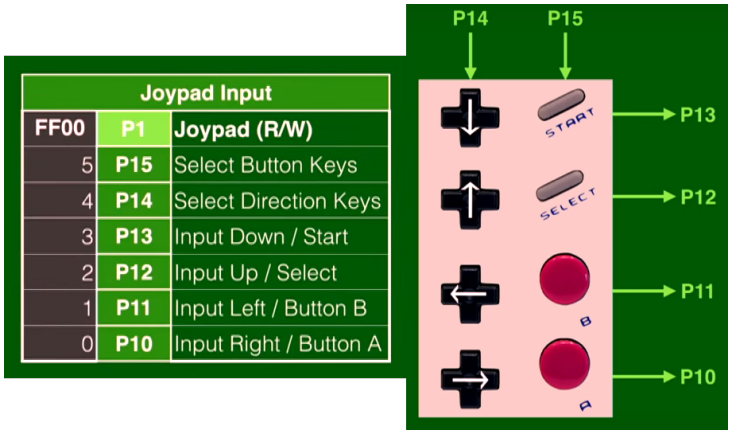


Figura A.3.: Registros del input y distribución de los botones

A.5. Timer

Como en casi todos los sistemas, hay un timer o cronómetro que ayuda a controlar el tiempo de manera precisa. Hay solo 1 timer en la Game Boy y se controla principalmente con 3 registros: TIMA, TMA y TAC.

- TIMA (): Es el registro que está incrementando constantemente en una determinada frecuencia especificada en el registro TAC (bits 0-1). Cuando este byte llega a su valor máximo, produce una interrupción y vuelve a su valor inicial.
- TMA: Es el valor inicial con el que comienza el registro TIMA cuando se desborda.
- TAC: Es el registro que controla el timer. El bit 2 enciende o apaga el timer y los bits 1-0 representan una de las 4 velocidades posibles (Figura A.4).

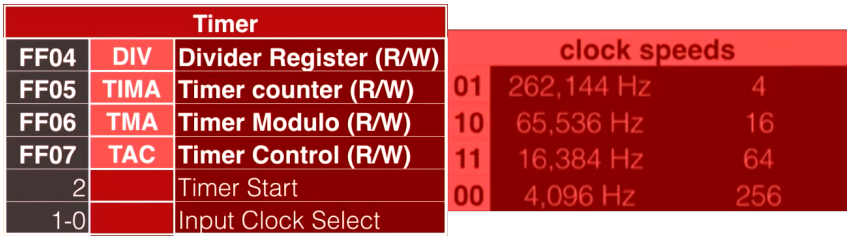


Figura A.4.: Registros del timer y frecuencias

A.6. Interrupciones

Una interrupción consiste en una función que se ejecuta independientemente del hilo principal del juego. Una interrupción puede ser llamada en cualquier momento del programa. Se deja en

espera el código que se estaba ejecutando, se realiza la interrupción y al acabar se vuelve al código anterior.

En Game Boy existen 5 tipos de interrupciones:

- Joypad: se ejecuta cada vez que se pulsa un botón
- Serial: cada vez que se recibe un byte por transferencia
- Timer: cada vez que se desborda el registro TIMA[1]
- LCDC STAT: hay varias razones para esta interrupción dependiendo de los bits que estén asignados en el registro LCDC STAT ($FF41). Una razón muy popular es la de indicar al usuario cuando se va a redibujar una línea, para por ejemplo cambiar el scroll del fondo y producir algún efecto.
- V-Blank: ocurre cuando se redibuja la última linea de la pantalla, al principio del periodo V-Blank.

Interrupt Controller						
FFFF	IE	Interrupt Enable (R/W)	FF0F	IF	Interrupt Flag (R/W)	
4		Joypad	4		Joypad ⟶	RST $60
3		Serial	3		Serial ⟶	RST $58
2		Timer	2		Timer ⟶	RST $50
1		LCD STAT	1		LCD STAT ⟶	RST $48
0		V-Blank	0		V-Blank ⟶	RST $40

Figura A.5.: Registros Interrupt Enable (izq.) e Interrupt Flag (der.)

Todas las interrupciones se activan o desactivan en el registro IE (Interrupt Enable) en la posición $FFFF. Además, hay un registro llamado IF (Interrupt Flag) en la posición $FF0F que indica las interrupciones que están pendientes de hacer. Afortunadamente, no hace falta leer este registro para controlar las interrupciones ya que salta automáticamente a unas posiciones reservadas en memoria. En la Figura A.5 se puede ver a las posiciones a las que salta cada interrupción.

[1]Más información sobre el timer en el apartado A.5

A.7. Pixel Processing Unit (PPU)

La PPU es la unidad que hace todos los gráficos. Contiene 12 registros en total y están agrupados en la Figura A.6.

Las especificaciones de la PPU son las siguientes:

- Pantalla: 160x144 píxeles
- Colores: 4 colores (tonos de gris/tonos de verde)
- Basado en tiles de 8x8 píxeles, 20x18 tiles,
- 40 sprites (10 por línea)
- 4KB de VRAM

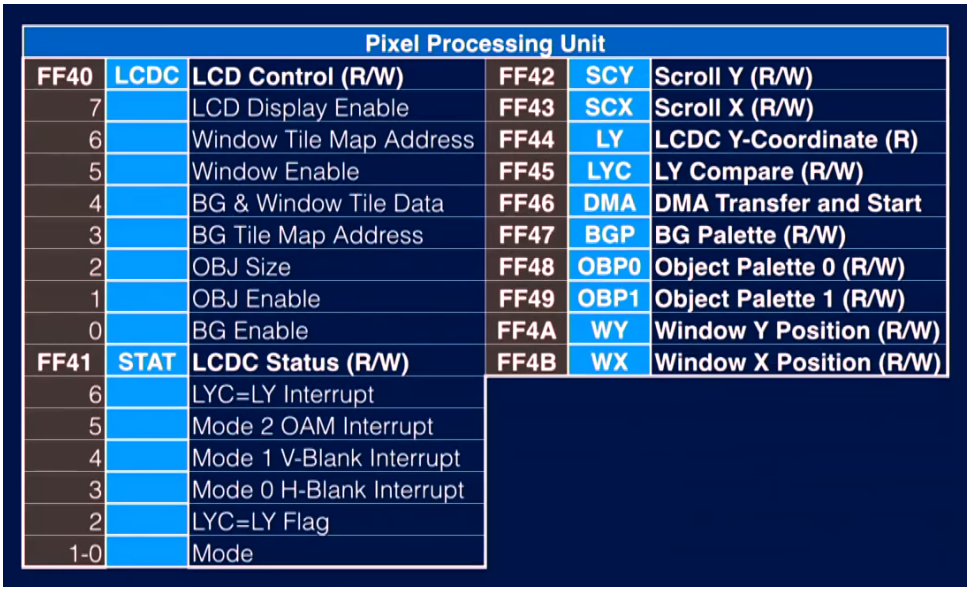


Figura A.6.: Todos los registros que forman la PPU

A.7.1. LCD Control (LCDC)

El LCD Control (LCDC) es el registro encargado de controlar todo lo que se dibuja por pantalla. Controla si se dibujan fondo, ventana y sprites y donde se sitúan los tiles de fondo y de ventana. Los bits de este registro (Figura A.7) son en orden, los siguientes:

- bit 7: Enciende o apaga el lcd. Si está apagado no se ve nada por pantalla (0=apagado, 1=encendido).
- bit 6: Selecciona la posición del mapa de la ventana (0=$9800-$9BFF, 1=$9C00-$9FFF).
- bit 5: Dibuja o no la ventana (0=apagado, 1=encendido).
- bit 4: Selecciona la posición de los tiles de fondo y ventana (0=$8800-$97FF, 1=$8000-$87FF).
- bit 3: Selecciona la posición del mapa de fondo (0=$9800-$9BFF, 1=$9C00-$9FFF).

- bit 2: Tamaño de los sprites (0=8x8, 1=8x16). Si está activado, todos los sprites del juego pasan a ser de 8x16 y utilizan 2 tiles (el que tiene asignado y el siguiente en memoria) en vez de 1.
- bit 1: Dibuja o no los sprites (0=apagado, 1=encendido).
- bit 0: Dibuja o no el fondo y la ventana (0=apagado, 1=encendido).

$FF40	LCDC (R/W)
7	Control del display
6	Selección del mapa de la ventana
5	Control de la ventana
4	Selección de los tiles mapa de fondo y ventana
3	Selección del mapa de fondo
2	Tamaño de los sprites
1	Control de los sprites
0	Control del fondo y ventana

Figura A.7.: Bits del registro LCDC

A.7.2. STAT

El LCDC STAT es el registro encargado de controlar todas las interrupciones relacionadas con la pantalla. Controla si hay interrupciones cuando la coordenada LY[2] es igual LYC y también cuando comienzan los periodos V-Blank y H-Blank. Los bits de este registro (Figura A.7) son en orden, los siguientes:

- bit 6: Interrupción cuando LYC es igual a LY (LY es la linea que se dibuja y LYC es seleccionada por el usuario)
- bit 5: Modo %10
- bit 4: Modo %01
- bit 3: Modo %00
- bit 2: Flag de coincidencia de LYC y LY (indica cuando son iguales, independientemente de la interrupción)
- bit 1-0: Modo del flag

STAT muestra el estado actual del controlador LCD. Los modos son:

- Modo 00: indica que es el periodo H-Blank y la CPU puede acceder la VRAM
- Modo 01: lo mismo que el modo 00 pero con el periodo V-Blank (más tiempo)
- Modo 10: indica que la OAM se está usando y no puede ser usada
- Modo 11: indica que la OAM y la VRAM se están usando y la CPU no puede acceder a ninguna

[2]Más información sobre la pantalla en el Anexo A.7.7

$FF41	STAT (R/W)
6	Interrupción LYC = LY
5	Interrupción modo 2 OAM
4	Interrupción modo 1 V-Blank
3	Interrupción modo 0 H-Blank
2	Flag LYC=LY
1-0	Modo

Figura A.8.: Bits del registro STAT

A.7.3. Mapa de fondo

El mapa de fondo es la capa que se encuentra por debajo de todo lo que está dibujado por pantalla. Tiene un tamaño en ventana de 20x18 pero realmente tiene un tamaño de 32x32 tiles y 256x256 píxeles.

La pantalla de Game Boy tiene un funcionamiento similar al de una cámara. Es un viewport del tamaño de la ventana que se puede desplazar libremente por el mapa de fondo. Cuando llega al límite del mapa tanto horizontal como verticalmente, vuelve al principio. De hecho, el efecto de scroll se produce si se va cargando el mapa dinámicamente justo antes de que el viewport lo visualice.

En algunos juegos se puede apreciar con un emulador cómo está dibujado este mapa de fondo y cómo se mueve el viewport por encima del mapa. Por ejemplo, en Game Boy Tennis se puede ver perfectamente cómo se desplaza el viewport por el mapa estático, siguiendo la pelota. Los registros para controlar la posición son: $FF42 para la Y y $FF43 para la X (constantes `rSCY` y `rSCX`)

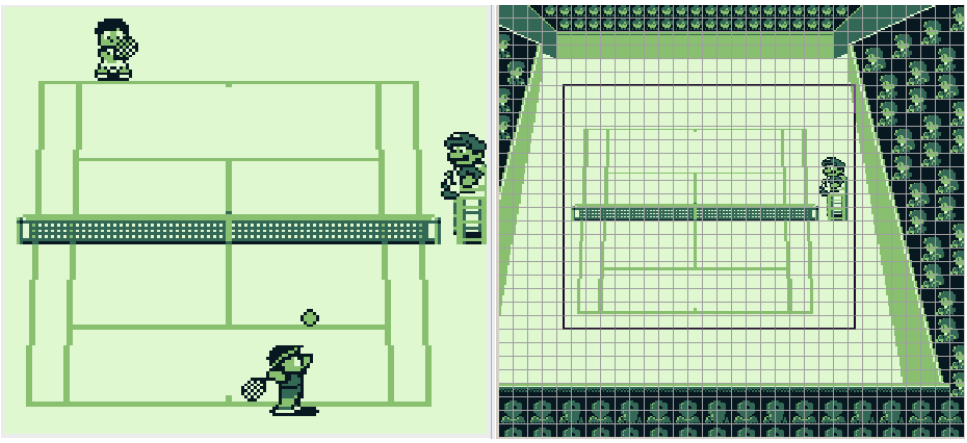


Figura A.9.: Mapa de fondo y viewport en Game Boy Tennis

A.7.4. Ventana

Encima de la capa del fondo existe una capa adicional, la ventana. Tiene el mismo tamaño que el mapa de fondo (32x32 tiles) pero tiene la peculiaridad de que se puede elegir la posición en la que empieza. La posición indica el primer tile de la capa y se dibujará hacia la derecha y hacia abajo. No hay transparencia con la capa de fondo, por lo tanto, lo que se suele hacer es situarla en la parte inferior o derecha de la pantalla.

En la Figura A.11 se puede observar que la ventana se superpone al mapa y, aunque el jugador se mueva por el mapa y se modifique el fondo, va a pintarse por encima. La ventana se configura en el registro del LCDC (A.7.1) y se mueve con los registros $FF4A (`rWY`) para la Y y $FF4B

(`rWX`) para la X.

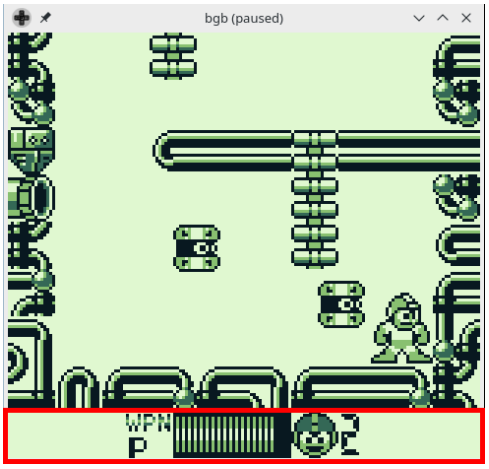


Figura A.10.: Ejemplo de uso de ventana en Mega Man II

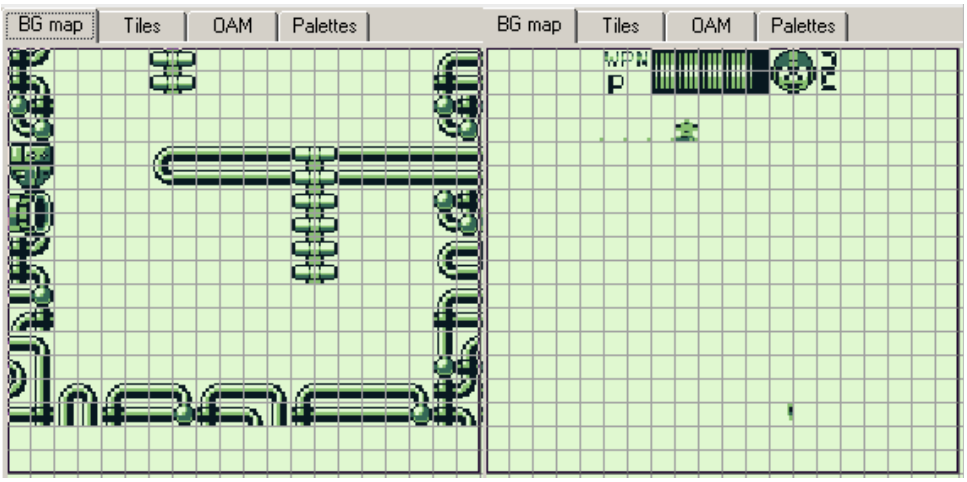


Figura A.11.: Mapa de fondo (izq.) y ventana (der.) en Mega Man II

A.7.5. Sprites

Los sprites son el núcleo de todo juego. Son los objetos se que pueden mover por todo el mapa y se dibujan por encima de todo. La Game Boy tiene un máximo de 40 sprites que se pueden dibujar a la vez y un límite de 10 sprites por línea. En cuanto hayan 10 sprites en una misma línea y se añada otro sprite, la PPU se encarga de borrar las filas de píxeles que coincidan.

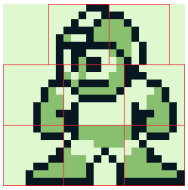


Figura A.12.: Sprite de Mega Man

Toda la información de los sprites se encuentra en el Mapa de Atributos de Objetos, conocido como OAM. Aquí están definidos todos los sprites uno a continuación de otro. Cada sprite ocupa 4 bytes y, sumando los 40 sprites, la OAM ocupa $A0 (160) bytes.

Los gráficos de los sprites, se guardan en memoria en el mismo formato que los tiles de fondo y en la misma posición. Cada sprite tiene los siguientes atributos:

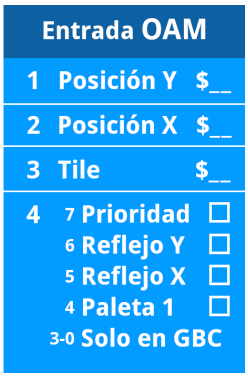


Figura A.13.: Entrada de la OAM

- Byte 1: Posición Y

  Especifica la posición Y del sprite menos 16 ($10). Esto quiere decir que el primer píxel vertical visible en pantalla es el 16, esto es así para poder poner el sprite un poco o totalmente fuera de la pantalla, dibujándolo entre las posiciones 0-15. ¿Por qué 16 en vez de 8? Porque los sprites pueden ser de 8x8 o de 8x16 y es necesaria una manera de sacar ambos de la pantalla.

- Byte 2: Posición X

  Especifica la posición X del sprite menos 8.

- Byte 3: Número de sprite

  Índice del tile guardado en la memoria de tiles que se va a dibujar. Si los sprites son de 8x16, dibuja el tile del índice y el siguiente.

- Byte 4: Atributos de sprite

  Define los atributos del sprite en base a los bits activados[3].

Todos los sprites se encuentran almacenados uno detrás de otro de la siguiente forma:

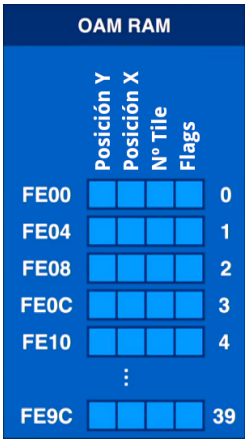


Figura A.14.: Estructura de la OAM

[3]En la sección de sprites (5.4) se pueden encontrar más información sobre el byte de prioridad

A.7.6. Mapa de la memoria VRAM

La Video RAM (VRAM) de la Game Boy contiene todos los gráficos del juego: los tiles del fondo, de los sprites y los índices del mapa y de la ventana. Los tiles del fondo ocupan 4KB (256 tiles x 16 bytes/tile), los tiles de los sprites otros 4KB y los índices del mapa y de la ventana ocupan 1KB cada uno (32x32 índices).

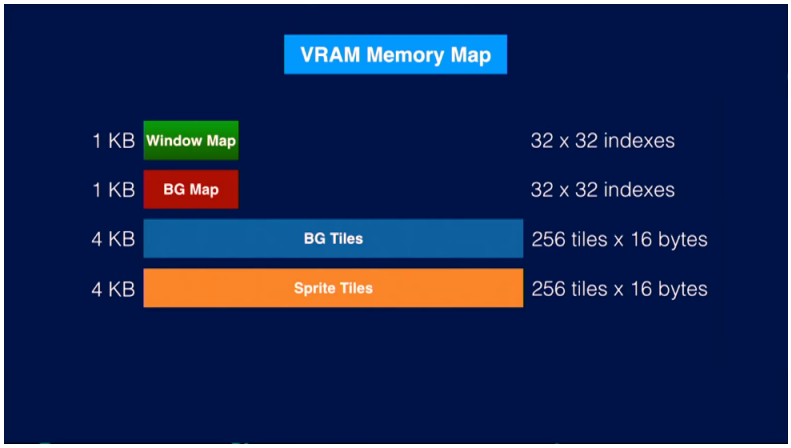
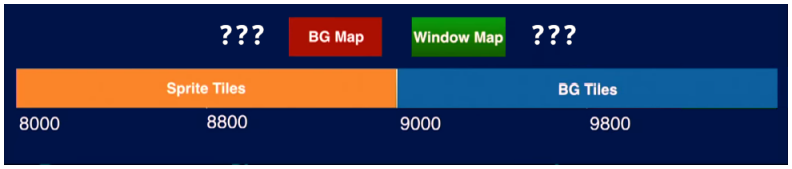


Figura A.15.: Tamaño de los componentes de la VRAM

Pero hay un problema con todo esto: el tamaño total de la VRAM es de 8KB (de $8000 a $9FFF) y solo con los tiles ya se ocupan los 8KB de almacenamiento. ¿Cómo es capaz de almacenar todo la Game Boy?



La respuesta es que los tiles de fondo se superponen con los tiles de los sprites. Con el **bit 6** del LCDC se puede controlar la posición de los tiles de fondo. Si está desactivado, los tiles de fondo y los de los sprites se comparten totalmente y si está a activado, se solapan la mitad de los sprites.

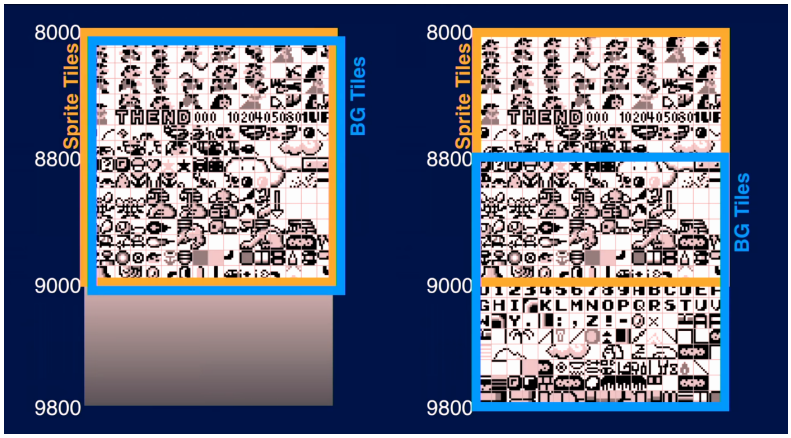


Figura A.16.: Bit 6 del LCDC desactivado (izq.) y activado (der.)

Los tiles tanto de fondo como los de sprites tienen exactamente el mismo formato (tiles de 8x8 píxeles y 2 bits por píxel) y se pueden repartir de las dos diferentes formas representadas en

la Figura A.16. De esta forma, los primeros 128 tiles serían exclusivos de los sprites, los últimos 128 serían exclusivos del fondo, y los 128 del centro serían utilizables por ambos.

En la Figura A.17 se puede el mapa resumido de la VRAM. Además, los índices del fondo y ventana son totalmente intercambiables. En este caso, se controlan utilizando los bits 3 y 4 del LCDC.

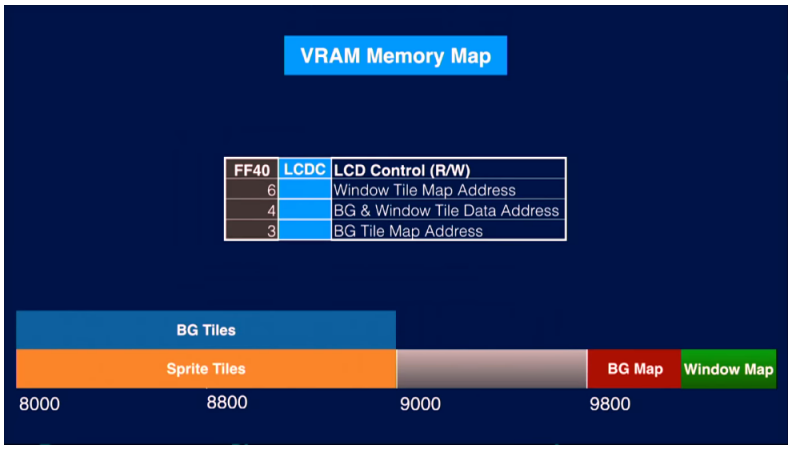


Figura A.17.: Mapa de la memoria de la VRAM

A.7.7. Timing y acceso a VRAM y OAM

La pantalla de la Game Boy, al igual que muchos ordenadores basados en Cathode-Ray Tube (CRT), dibuja los píxeles de uno en uno, de arriba abajo y de izquierda a derecha. La frecuencia de refresco de la pantalla es de 60Hz aproximadamente y eso significa que dibuja de arriba a abajo, de derecha a izquierda, 160x144 píxeles 60 veces por segundo.

Timing Vertical

El timing vertical consiste en controlar a qué altura o línea de la pantalla se está dibujando en todo momento. Esto es muy importante para poder realizar algunos efectos visuales de algunos juegos, como los de carreras.

Existen 3 registros a tener en cuenta:

- $FF44 - LY, coordenada Y del LCDC (`rLY`): informa de la línea vertical en la que se encuentra dibujando. Este registro incrementa desde 0 hasta 153, el máximo de líneas. Cuando se encuentra entre la posición 144 y la 153, indica que se ha dibujado toda la pantalla y hay un espacio donde la PPU no hace nada.
- $FF45 - LYC, comparación con LY (`rLYC`): el número de este registro se compara con el valor de LY y asigna el flag (bit 2) del registro del STAT.
- $FF41 - LCDC STAT (`rSTAT`): define el estado del LCDC. Su bit 6 controla si están activadas las interrupciones de comparación con LYC y el bit 2 indica el momento en el son iguales.

Timing Horizontal

Profundizando un poco más, el timing horizontal es muy importante para entender lo que ocurre en cada línea y comprender el acceso a la VRAM y la OAM.

En cada línea se produce lo más importante para ver algo, la transferencia de píxeles a la pantalla en base al mapa y sprites definidos en la VRAM y OAM. Esto tarda sobre unos 43

ciclos de reloj y se hace para 144 lineas. Cuando se termina la transferencia de píxeles de una línea, no se empieza a dibujar el primer píxel de la línea siguiente instantáneamente.

Para cada línea, la PPU realiza: un **registro de la OAM**, que tarda 20 ciclos, la **transferencia de los píxeles**, que tarda 43 ciclos, y hay un **espacio de 51 ciclos** en el que PPU no hace nada y se conoce H-Blank (Espacio en blanco horizontal). Verticalmente pasa algo parecido, tampoco comienza con la primera línea instantáneamente al acabar la última. Hay un espacio en blanco de 10 líneas en las que la PPU se encuentra en espera. Este espacio es cuando la PPU está más tiempo seguido en espera y se denomina V-BLank.

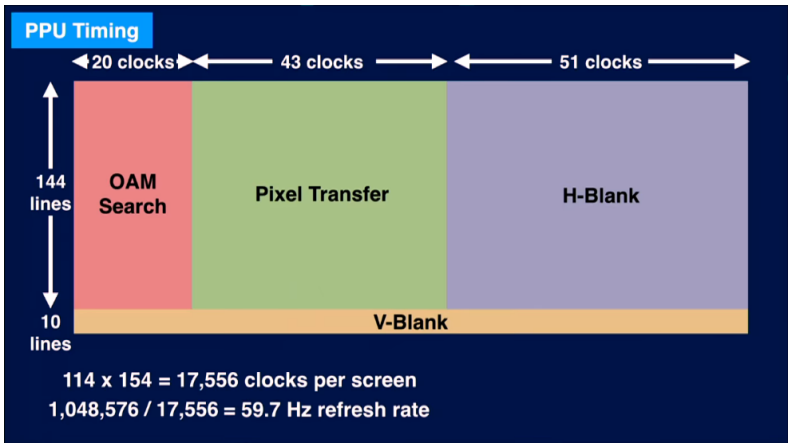


Figura A.18.: Desglose de lo que se hace mientras se dibuja una línea

El registro de la OAM consiste en (por cada línea) hacer una comparación de todos los sprites que se encuentran en la línea actual y añadirlos a un array donde caben hasta un máximo de 10 sprites. La lógica para añadirlos al array es que la posición del sprite debe ser distinta de 0 y que la línea se contenga entre el inicio del sprite y su altura.

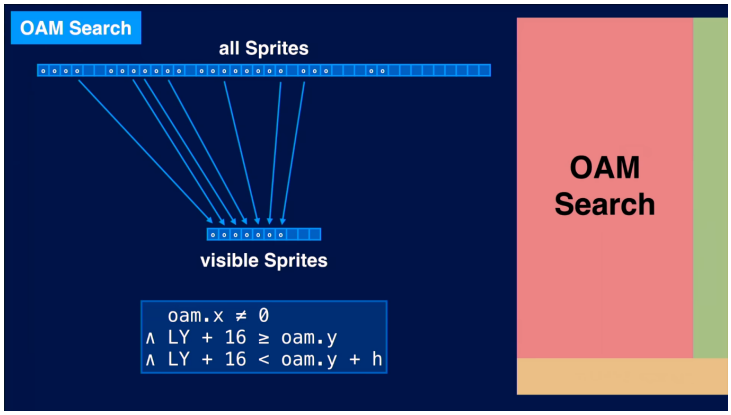


Figura A.19.: Registro de la OAM

Acceso a la VRAM

La CPU está conectada a la RAM y, a su vez, la PPU está conectada a la VRAM y a la OAM. La CPU no se encuentra conectada directamente a la VRAM, tiene que pasar a través de la PPU para poder escribir o leer algo. La PPU actúa como un interruptor que controla cuando es accesible y si la CPU intenta acceder igualmente, escribir no cambia nada y al leer solo obtiene $FF. No pasa nada malo pero tampoco hace nada útil.

Durante la transferencia de píxeles, no se puede acceder para nada a la VRAM pero si que se puede acceder durante el registro de la OAM o durante los espacios H-Blank y V-Blank. Por

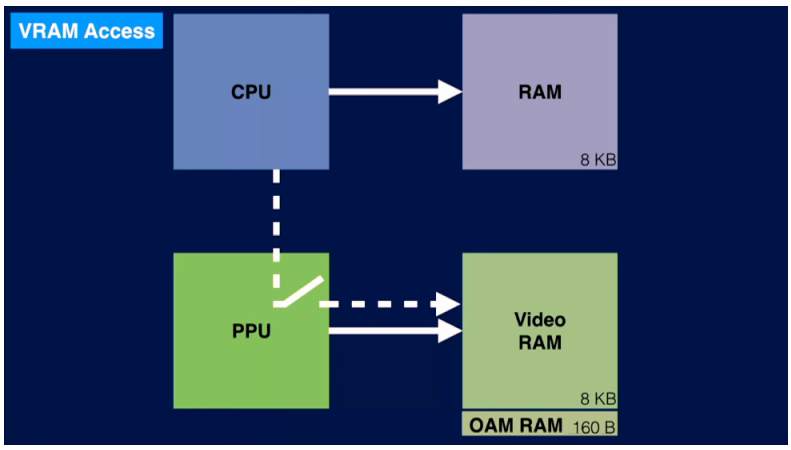


Figura A.20.: Esquema con el acceso desde la CPU a la VRAM

otra parte, la OAM no es accesible durante la transferencia de píxeles ni durante el registro de la OAM. Solo durante los espacios H-Blank y V-Blank.

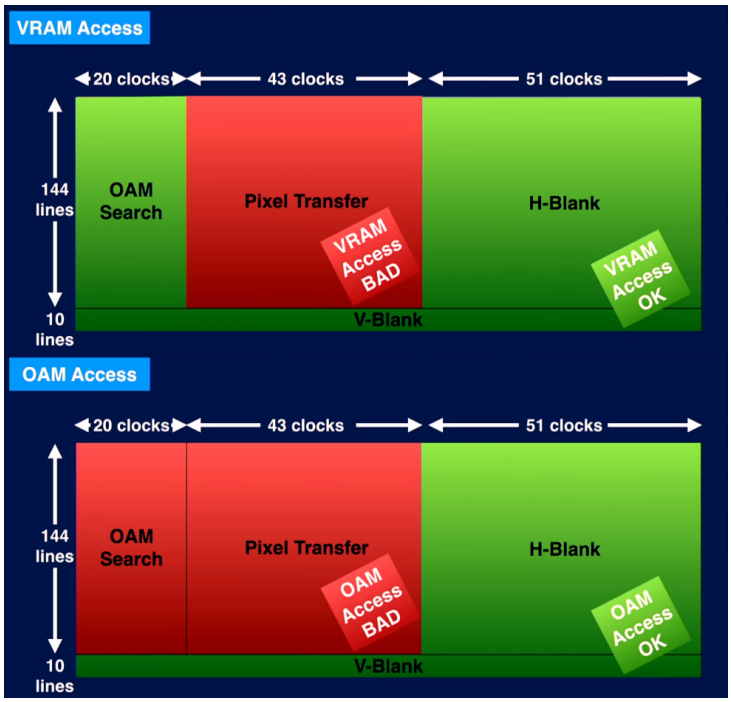


Figura A.21.: Momento cuando la VRAM y la OAM es accesible

Resumiendo, durante el registro de la OAM para decidir qué sprites son visibles y la transferencia de píxeles, es una zona peligrosa para la VRAM y no se debería hacer nada. Normalmente, para modificar algo relacionado con el fondo se suele esperar al V-Blank o si es algo bastante grande (como por ejemplo cargar 100 tiles que se van a usar en el juego) se recomienda apagar el LCD.

Para mover sprites, tambíen es posible esperar al V-Blank para actualizar la OAM pero existe una técnica llamada "Shadow OAM" que consiste en reservar una zona de 160 bytes (tamaño de la OAM) en la RAM, pudiendo acceder a ella y modificarla en cualquier momento y luego copiar toda esa zona a la OAM en el periodo V-Blank.